



DIGITAL LOGIC DESIGN

G JAYA KRISHNA
ASSISTANT PROFESSOR
DEPARTMENT OF CSE
RGUKT NUZVID

M VEDAVYAS
CSE-05, 2020 BATCH
RGUKT NUZVID



COURSE OF CONTENT

S NO.	TOPIC NAME	PAGE NO.	LINK
1.	<u>UNIT-1</u>	1	Click here
1.1	NUMBER SYSTEMS	1	
1.2	CONVERSION OF NUMBER SYSTEMS	2	
1.3	PROBLEM SOLVING	6	
1.4	CLASSIFICATION OF NUMBERS	9	
1.5	PROBLEM SOLVING	14	
1.6	BINARY CODES	15	
1.7	ADDITION & SUBTRACTION ON BCD AND EXCESS-3	19	
1.8	ERROR DETECTION AND ERROR CORRECTION	24	
1.9	HAMMING CODES	27	
1.10	LOGIC GATES	28	
1.11	ALGEBRAIC PROPERTIES	32	
1.12	CANONICAL REPRESENTATION FOR SWITCHING OF EXPRES- SIONS	34	
1.13	REALIZATION OF LOGIC GATES USING UNIVERSAL GATES	36	
2.	<u>UNIT-2</u>	42	Click here
2.1	SYNTHESIS OF LOGIC GATES	42	
2.2	K-MAPS(Karnaugh Map)	48	
2.3	VARIOUS IMPLICANTS IN K-MAPS	57	
2.4	VARIABLE ENTRANT MAP	60	
2.5	COMBINATIONAL AND SEQUENTIAL CIRCUITS	63	
2.6	ARITHMETIC CIRCUITS	64	
2.7	BOOTH'S ALGORITHM	73	
2.8	COMPARATOR	80	
2.9	NOISE MARGINS	89	
3.	<u>UNIT-3</u>	90	Click here
3.1	MULTIPLEXER	90	
3.2	DE-MULTIPLEXERS	104	
3.3	DECODERS	109	
3.4	ENCODERS	117	
3.5	PRIORITY ENCODER	120	
3.6	IC 74X148	121	
4.	<u>UNIT-4</u>	122	Click here
4.1	SEQUENTIAL CIRCUITS	122	
4.2	COMBINATIONAL vs SEQUENTIAL CIRCUITS	122	
4.3	TYPES OF SEQUENTIAL CIRCUITS	123	
4.4	CLOCK SIGNAL AND TRIGGERING	123	
4.5	MEMORY ELEMENTS IN SEQUENTIAL CIRCUITS	125	
4.6	SR LATCH/FLIPFLOP	126	
4.7	JK FLIPFLOP	129	
4.8	MASTER SLAVE FLIPFLOP	132	
4.9	T-FLIPFLOP	133	
4.10	D-FLIPFLOP	135	
4.11	CONVERSION OF FLIPFLOPS	136	
4.12	STATE DIAGRAMS FOR FLIPFLOPS	140	



S NO.	TOPIC NAME	PAGE NO.1	LINK
5.	<u>UNIT-5</u>	141	Click here
5.1	COUNTERS	141	
5.2	DIFFERENCE BETWEEN SYNCHRONOUS AND ASYN- CHRONOUS COUNTERS	142	
5.3	THREE-BIT ASYNCHRONOUS UP COUNTER	143	
5.4	THREE-BIT ASYNCHRONOUS DOWN COUNTER	144	
5.5	THREE-BIT ASYNCHRONOUS UP/DOWN COUNTER	145	
5.6	MOD-6 UP COUNTER	146	
5.7	MOD-6 DOWN COUNTER	147	
5.8	RANDOM COUNTER	148	
5.9	TWO-BIT SYNCHRONOUS UP COUNTER	149	
5.10	TWO-BIT SYNCHRONOUS DOWN COUNTER	151	
5.11	TWO-BIT SYNCHRONOUS UP-DOWN COUNTER	152	
5.12	SELF-STARTING COUNTER	154	
5.13	FREE RUNNING COUNTER	155	
5.14	SHIFT COUNTERS	155	



UNIT-1

INTRODUCTION

- **DIGITAL** : Computeralizing something is called digitalization.
- **LOGIC** : Useful to work something.
- **DESIGN** : Design means structure. In design phase we are going to pictorially draw the actual representation of the requirement.

Examples :

1. In any kind of software industry, first the client give requirements then we are going to analyse them and moves to design phase accordingly.
2. In olden days, for 64 GB of memory the size of chip was large (10cmx10cm) and computer size is also very vast. Now a days size of the same 64 GB of memory is 1cmx1cm.
3. In olden days, there are two switches for ON and OFF. But now a days there is only one switch for ON/OFF this is the advantage of design.
4. For calculator we require any programming language like Java, C etc along with a hardware component (Mother board). Motherboard is responsible for processing of the data. Internally the hardware process the data and stores it in the form of 0s and 1s [Binary language].

1.1 NUMBER SYSTEMS

[-For Video](#)

- The way in which information is going to be stored or processed in the system is called Number system.
- As human beings we are following decimal number system in our daily life.
- There are various types of number systems. The four most common number system types are :
 1. Binary number system (Base- 2)
 2. Decimal number system (Base- 10)
 3. Octal number system (Base-8)
 4. Hexadecimal number system (Base-16)

1. Binary Number System (Base 2 Number System)

- The base 2 number system is also known as the Binary number system in which only two binary digits exist, i.e., 0 and 1. Specifically, the usual base-2 is a radix of 2.

For example : $(1010)_2$, $0b11$ are binary numbers.

- Here the values inside the brackets must be less than the radix i.e., 2.

For example : $(1023)_2$ is not a binary number.

2. Decimal Number System (Base 10 Number System)

- The decimal number system has a base of 10 because it uses ten digits from 0 to 9.
- In the decimal number system, the positions successive to the left of the decimal point represent units, tens, hundreds, thousands and so on.

For Example : $(905)_{10}$ is a decimal number.

3. Octal Number System (Base 8 Number System)

- In the octal number system, the base is 8 and it uses numbers from 0 to 7 to represent numbers.

For Example : $(765)_8$, 011 are octal numbers.

- Here the values inside the brackets must be less than 8.

For Example : $(795)_8$ is not an octal number.



4. Hexadecimal Number System (Base 16 Number System)

- In the hexadecimal system, numbers are written or represented with base 16.
- In the hexadecimal system, the numbers are first represented just like in the decimal system, i.e. from 0 to 9 and 10 to 15.
- To avoid ambiguity we represent 10 as 'A', 11 as 'B', 12 as 'C', 13 as 'D', 14 as 'E', 15 as 'F'.
- Here the values inside the brackets must be less than 16.
For Example : $(1F7)_{16}$, $0X11$ are Hexadecimal numbers.

1.2 CONVERSION OF NUMBER SYSTEMS

[-For Video](#)

1. Converting any number system into Decimal number system :

- General form to convert any number system into decimal number system is $(xyz)_n \Rightarrow (x \cdot n^2 + y \cdot n^1 + z \cdot n^0)_{10}$
- Examples :
 1. $(11)_2 \Rightarrow (1 \cdot 2^1 + 1 \cdot 2^0)_{10} \Rightarrow (3)_{10}$ [converting binary to decimal number system]
 2. $(11)_8 \Rightarrow (1 \cdot 8^1 + 1 \cdot 8^0)_{10} \Rightarrow (9)_{10}$ [converting octal to decimal number system]
 3. $(11)_{10} \Rightarrow (11)_{10}$
 4. $(11)_{16} \Rightarrow (1 \cdot 16^1 + 1 \cdot 16^0)_{10} \Rightarrow (17)_{10}$ [converting hexadecimal to decimal system]

• Note :

General form to convert fractional number into decimal number system is $(xy.z)_n \Rightarrow (x \cdot n^1 + y \cdot n^0 + z \cdot n^{-1})_{10}$

Example : $(11.11)_2 \Rightarrow (1 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 1 \cdot 2^{-2})_{10} \Rightarrow (3.75)_{10}$

2. Converting Decimal number system into any other number system :

- To convert decimal number system into any other number systems we need to divide the decimal number with respective base value and we need to write all the remainders from bottom to top.
- **Decimal to Binary Number :**
Suppose if we have to convert decimal to binary, then divide the decimal number by 2.
Example : $(25)_{10} = (11001)_2$

OPERATION	OUTPUT	REMAINDER
$25 \div 2$	12	1
$12 \div 2$	6	0
$6 \div 2$	3	0
$3 \div 2$	1	1
$1 \div 2$	0	1

• Decimal to hexa decimal Number :

To convert decimal to hexadecimal number we have to divide the given decimal number by 16 such that base 10 changes to base 16.

EXample : $(128)_{10} = (80)_{16}$

OPERATION	OUTPUT	REMAINDER
$128 \div 16$	8	0
$8 \div 16$	0	8



- **Decimal to Octal Number :**

To convert decimal to octal number we have to divide the given decimal number by 8 such that base 10 changes to base 8.

Example : $(128)_{10} = (200)_8$

OPERATION	OUTPUT	REMAINDER
$128 \div 8$	16	0
$16 \div 8$	2	0
$2 \div 8$	0	2

- **Note : Conversion of Fractional number into binary :**

= > To convert fraction to binary, multiply it by 2 keeping notice of the resulting integer and fractional part. Continue multiplying by 2 until you get a resulting fractional part equal to zero. Then just write out the integer parts from the results of each multiplication.

Example : converting the fraction $(5.28)_{10}$ into binary.

OPERATION	OUTPUT	REMAINDER
$5 \div 2$	2	1
$2 \div 2$	1	0
$1 \div 2$	1	1

For integer part, the answer is 101 and fractional part is to be calculated as follows :

OPERATION	OUTPUT	VALUE
$0.28 * 2$	0.56	0
$0.56 * 2$	1.12	1
$0.12 * 2$	0.24	0
$0.24 * 2$	0.48	0

Now, we need to write all the values in the table from top to bottom

Therefore the binary number for $(5.28)_{10}$ is $(101.0100)_2$

3. Converting **Octal number system** into **Binary number system** :

Method-1 :

- We can convert the **octal number into decimal and then convert** the decimal number into its equivalent binary number.

Example : Converting $(756)_8$ to decimal first then into binary.

$(756)_8 \Rightarrow (7*8^2 + 5*8^1 + 6*8^0)_{10} \Rightarrow (494)_{10}$

OPERATION	OUTPUT	REMAINDER
$494 \div 2$	247	0
$247 \div 2$	123	1
$123 \div 2$	61	1
$61 \div 2$	30	1
$30 \div 2$	15	0
$15 \div 2$	7	1
$7 \div 2$	3	1
$3 \div 2$	1	1
$1 \div 2$	0	1

Therefore $(756)_8 = (111101110)_2$

Method-2 :

- We can also use the octal number table to convert a number with **base 8 to a number with base 2.**



- Using this table we can also convert a binary number to an octal number.

OCTAL NUMBER	BINARY NUMBER (2^2 2^1 2^0)
0	0 0 0
1	0 0 1
2	0 1 0
3	0 1 1
4	1 0 0
5	1 0 1
6	1 1 0
7	1 1 1

• **Example :**

1. Convert $(756)_8$ into a binary number.

Now with the help of the table, we can write as follows :

For **7** Binary number is **111**

For **5** Binary number is **101**

For **6** Binary number is **110**

Therefore $(756)_8 = (111101110)_2$

2. Convert $(111110)_2$ into octal number.

Separate the given number with 3 digits each as follows :

|111|110

As we know that octal number for 111 is 7 and for 110 is 6.

Therefore $(111110)_2 = (76)_8$

4. Converting Hexadecimal number system into Binary number system :

Method-1 :

- We can convert the Hexadecimal number into decimal and then convert the decimal number into its equivalent binary number.

Example : Converting $(756)_{16}$ to decimal first then into binary.

$$(756)_{16} \Rightarrow (7 \cdot 16^2 + 5 \cdot 16^1 + 6 \cdot 16^0)_{10} \Rightarrow (1878)_{10}$$

OPERATION	OUTPUT	REMAINDER
$1878 \div 2$	939	0
$939 \div 2$	469	1
$469 \div 2$	234	1
$234 \div 2$	117	0
$117 \div 2$	58	1
$58 \div 2$	29	0
$29 \div 2$	14	1
$14 \div 2$	7	0
$7 \div 2$	3	1
$3 \div 2$	1	1
$1 \div 2$	0	1

Therefore $(756)_{16} = (11101010110)_2$

Method-2 :

- We can also use the Hexadecimal number table to convert a number with base 16 to a number with base 2.
- Using this table we can also convert a binary number to a hexadecimal number.



HEXA DECIMAL	BINARY NUMBER
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1
10(A)	1 0 1 0
11(B)	1 0 1 1
12(C)	1 1 0 0
13(D)	1 1 0 1
14(E)	1 1 1 0
15(F)	1 1 1 1

• **Examples :**

1. Convert FDA6 to an equivalent binary number.

Now with the help of the table, we can write as follows :

For **F** Binary number is **1111**

For **D** Binary number is **1101**

For **A** Binary number is **1010**

For **6** Binary number is **0110**

Therefore **(FDA6)₁₆ = (1111110110100110)₂**

2. Convert (101000101011)₂ into hexadecimal number.

Separate the given number with 4 digits each as follows :

|1010|0010|1011

As we know that hexa decimal number for 1010 is A(10) and for 0010 is 2 and for 1011 is B(11).

Therefore **(101000101011)₂ = (A2B)₁₆**

5. Converting Hexa decimal to octal and vice versa

Examples :

1. Convert (1BC)₁₆ into an octal number.

1 → 0001, B → 1011, C → 1100

Now group them from right to left, each having 3 digits.

000, 110, 111, 100

000→0, 110→6, 111→7, 100→4

Hence, **(1BC)₁₆ = (674)₈**

2. Convert (15432)₈ into hexa decimal decimal.

First convert (15432)₈ into binary.

1 → 001, 5 → 101, 4 → 100, 3 → 011, 2 → 010

Now group them from right to left, each having 3 digits.

Now separate 001101100011010 with 4 digits each

001|1011|0001|1010

001→1, 1011→B, 0001→1, 1010→A

Hence, **(15432)₈ = (1B1A)₁₆**



1.3 PROBLEM SOLVING

1. What is the value of $(11)_2 + (22)_3 + (33)_4$ in base-6 format ?

Solution :

First of all, we need to convert $(11)_2 + (22)_3 + (33)_4$ into decimal number system.

$$(11)_2 \Rightarrow (1 \cdot 2^1 + 1 \cdot 2^0)_{10} \Rightarrow (3)_{10}$$

$$(22)_3 \Rightarrow (2 \cdot 3^1 + 2 \cdot 3^0)_{10} \Rightarrow (8)_{10}$$

$$(33)_4 \Rightarrow (3 \cdot 4^1 + 3 \cdot 4^0)_{10} \Rightarrow (15)_{10}$$

$$\text{That implies, } (3)_{10} + (8)_{10} + (15)_{10} \Rightarrow (26)_{10}$$

Now converting $(26)_{10}$ into base-6 format.

OPERATION	OUTPUT	REMAINDER
$26 \div 6$	4	2
$4 \div 6$	0	4

Therefore, $(11)_2 + (22)_3 + (33)_4 = (42)_6$

2. Suppose $(123)_5 = (x8)_y$, what are the possible values of x and y ?

Solution :

Given that, $(123)_5 = (x8)_y$

$$\Rightarrow (1 \cdot 5^2 + 2 \cdot 5^1 + 3 \cdot 5^0) = (x \cdot y^1 + 8 \cdot y^0)$$

$$\Rightarrow (25 + 10 + 3) = (xy + 8)$$

$$\Rightarrow 38 = (xy + 8)$$

$$\Rightarrow 30 = xy$$

Let us see the possible values for x and y that holds the condition $xy=30$.

X	Y	XY
1	30	30
2	15	30
3	10	30
5	6	30
6	5	30
10	3	30
15	2	30
30	1	30

But we know that, In y-number system the values must be less than y in it.

Also that if $x=5, y=6$ then $(58)_6$ is not possible [Since $8 > 6$].

So we can remove the values from the table in which x greater than y as shown below.

X	Y	XY
1	30	30
2	15	30
3	10	30

Therefore the above table shows the possible values for x and y where $(xy=30)$.

Thus the possible conversions for $(123)_5$ in $(x8)_y$ format are as follows :

$$(123)_5 = (18)_{30}$$

$$(123)_5 = (28)_{15}$$

$$(123)_5 = (38)_{10}$$



3. If $38+43 = 80$ then find out the number system in which it is correct ?

Solution :

Let us assume that given is in some b-number system.

Now convert it in to decimal number system.

$$\Rightarrow (3*b^1+8*b^0) + (4*b^1+3*b^0) = (8*b^1+0*b^0)$$

$$\Rightarrow (3b+8)+(4b+3) = 8b$$

$\therefore$ The number system, $b=11$

4. If $\sqrt{22} = 6$ then find out the number system in which it is correct ?

Solution :

Let us assume that given is in some k-number system.

$$\text{Given that } \sqrt{22} = 6 \Rightarrow 22=6^2 \Rightarrow 22=6 \times 6$$

Now convert it in to decimal number system.

$$\Rightarrow (2*k^1+2*k^0) = (6*k^0)(6*k^0)$$

$$\Rightarrow (2k+2) = (6)(6)$$

$$\Rightarrow (2k+2) = 36$$

$$\Rightarrow 2k = 34$$

$\therefore$ The number system, $k=17$

5. Suppose $(42)_9 = (x3)_y$, what are the possible values of x and y ?

Solution :

$$\text{Given that, } (42)_9 = (x3)_y$$

$$\Rightarrow (4*9^1+2*9^0) = (x*y^1+3*y^0)$$

$$\Rightarrow (36+2) = (xy+3)$$

$$\Rightarrow 38 = (xy+3)$$

$$\Rightarrow 35 = xy$$

Let us see the possible values for x and y that holds the condition $xy=35$.

X	Y	XY
1	35	35
5	7	35
7	5	35
35	1	35

But we know that, In y-number system the values must be less than y in it.

So we can remove the values from the table in which x greater than y as shown below.

X	Y	XY
1	35	35
5	7	35

Therefore the above table shows the possible values for x and y where $(xy=35)$.

Thus the possible conversions for $(42)_9$ in $(x3)_y$ format are as follows :

$$(42)_9 = (13)_{35}$$

$$(42)_9 = (53)_7$$



6. Solve $(2.3)_4 + (1.2)_4 = (y)_4$

Solution :

First of all we need to convert $(2.3)_4 + (1.2)_4 = (y)_4$ into decimal number system.

$$\Rightarrow (2 \times 4^0 + 3 \times 4^{-1}) + (1 \times 4^0 + 2 \times 4^{-1}) = (2.75)_{10} + (1.5)_{10} = (4.25)_{10}$$

Now, converting $(4.25)_{10}$ in to base-4 system.

$$\Rightarrow \text{Integer part } (4)_{10} = (10)_4 \text{ and fractional part is } (0.25)_{10} = (10)_4$$

OPERATION	OUTPUT	VALUE
0.25×4	1.00	1
0.00×4	0	0

$$\therefore (2.3)_4 + (1.2)_4 = (y)_4 = (10.10)_4$$

7. If $(11X1Y)_8 = (12C9)_{16}$ then what are the values of X and Y ?

Solution :

Consider RHS and binary numbers for 1 is 0001, for 2 is 0010, for C is 1100 & for 9 is 1001.

$$\Rightarrow (12C9)_{16} = (0001001011001001)_2$$

Now separate $(0001001011001001)_2$ with 3 digits as 0|001|001|011|001|001

Octal values : 001 $\rightarrow$ 1, 011 $\rightarrow$ 3

$$\Rightarrow (12C9)_{16} = (11311)_8$$

$$\Rightarrow (11X1Y)_8 = (11311)_8$$

$$\therefore X=3, Y=1$$

8. Find the minimum number of bits required to represent $(6728)_{10}$ in binary?

NOTE :

In base-x system the values in it are less than x.

$$\begin{aligned} \Rightarrow ((\dots(x-1)(x-1)(x-1)(x-1))_x &= ((x-1)^*x^{n-1} + \dots + (x-1)^*x^2 + (x-1)^*x^1 + (x-1)^*x^0)) \\ &= (x-1)(x^{n-1} + x^{n-2} + \dots + x^1 + x^0) \\ &= (x-1)((x^n - 1)/x - 1) \quad [\text{since it is in GP}] \\ &= x^n - 1 \end{aligned}$$

Solution :

$$\Rightarrow (2^n - 1) \geq 6728$$

$$\Rightarrow (2^n) \geq 6729$$

$$\Rightarrow n \geq \log_2 6729$$

$$\Rightarrow n \geq 13$$

9. Find the minimum decimal equivalent value of $(21C)_x$?

Solution :

The possible x value is $x \geq 13$ [Since $(x^n - 1) \geq 21C$]

$$\Rightarrow (2 \times 13^2 + 1 \times 13^1 + 12 \times 13^0) = (363)_{10}$$

$$\therefore \text{The minimum decimal equivalent value of } (21C)_x \text{ is } (363)_{10}$$

10. Find the minimum of digits are required to represent $(32 \text{ digit})_{10}$ in binary system ?

Solution :

The maximum possible number for $(32 \text{ digit})_{10}$ is $(9 \ 9 \ 9 \dots 9 \ 9)_{10}$ [32 times of 9]

Let us assume 32-digit number as k-digit number and decimal number system as b-number system.



$$\Rightarrow x^n - 1 \geq b^k - 1$$

$$\Rightarrow x^n \geq b^k$$

$$\Rightarrow n \geq k \log_x b$$

Substituting k and b values

$$\Rightarrow n \geq 32 \log_2 10$$

$\therefore$ The minimum of digits required to represent $(32 \text{ digit})_{10}$ in binary system, $n \geq 107$

11. Find the minimum number of bits required to represent $(516)_7$ in base-4 system ?

Solution

Converting $(516)_7$ into decimal number system

$$\Rightarrow (5 \cdot 7^2 + 1 \cdot 7^1 + 6 \cdot 7^0) = (5 \cdot 49) + (7) + 6 = (258)_{10}$$

Now $x^n - 1 \geq 258$

$$\Rightarrow x^n \geq 259$$

$$\Rightarrow n \geq \log_4 259$$

$$\therefore n \geq 5$$

12. In a number system, there is a range of numbers from -3 to 3 and they can be represented as C,B,A,0,1,2,3. Express $(102)_{10}$ in the given number system ?

Solution

In given number system there are 7 values from -3 to 3 which are represented as C,B,A,0,1,2,3.

Now convert $(102)_{10}$ to get the values from range -3 to 3 only as follows :

OPERATION	OUTPUT	REMAINDER
$102 \div 7$	15	-3
$15 \div 7$	2	1
$2 \div 7$	0	2

We get (2 1 -3) i.e., $(21C)_7$ [since $c=-3$]

$\therefore (102)_{10}$ is represented in given number system as $(21C)_7$.

1.4 CLASSIFICATION OF NUMBERS

[-For Video](#)

- The numbers are classified into two :
 - Unsigned Numbers
 - Signed Numbers

1. Unsigned Numbers :

- All Positive numbers and zero are Unsigned numbers because we don't mention sign(+) before positive numbers every time.
- All the unsigned numbers are to be stored directly in the system.

2. Signed Numbers :

- A Signed number has both positive and negative numbers along with the value zero.
Ex : -5, -7, +5, 0
- All the negative signed numbers are to be stored in the form of their complements in the system.
- We have different type of approaches to represent the signed data in the system.



REPRESENTATION OF SIGNED DATA :

- To represent signed data the following approaches and will be used :
 1. Sign Magnitude Representation
 2. Complementary of system.

1. SIGN MAGNITUDE REPRESENTATION

[-For Video](#)

- Signed Magnitude Representation uses the Most Significant Bit (MSB) as the sign bit.
- The MSB is the leftmost bit in a binary number, so in the binary number 101101, 1 is the MSB.
- In Signed Magnitude Representation, the sign of the number is interpreted as follows :
 - 0 means Positive(+)
 - 1 means Negative(-)
- We can determine whether a given number is negative or non negative simply by testing the most significant bit(MSB).

Example : $5 = 101$; $0101 = +5$ and $1101 = -5$

Drawbacks :

- They are two representations of 0

$$00000000 = + (0)_{10}$$

$$10000000 = - (0)_{10}$$
 To test if a number is 0 or not, the CPU will need to see whether it is 00000000 or 10000000.
- It is not convenient for the system to perform arithmetic operations like addition and subtraction.

Example : Adding the numbers (-4) and (+2)

$$\begin{array}{r} 1100 \rightarrow (-4) \text{ First Number} \\ + 0010 \rightarrow (+2) \text{ Second Number} \\ \hline 1110 \rightarrow (-2) \text{ Sum} \end{array}$$

Here, system has given wrong answer of $-6 = 1110$, instead of giving correct answer of $-2 = 1010$.
- Thus, there is a need of designing the different kind of circuits for performing the arithmetic operations.

2. COMPLEMENTARY OF A SYSTEM

[-For Video](#)

- Complements are used in the digital computers in order to simplify the subtraction operation and for the logical manipulations.
- To reduce designing of more circuits to perform arithmetic operations complements are introduced.
- Examples :**
 - We know in probability, the complement of 0.2 is $1 - 0.2 = 0.8$
 - $(84)_{10}$ i.e., the maximum possible number is $10^2 - 1 = 99$.
So complement of $(84)_{10}$ is $99 - 84 = 15$
 - For $(1234)_8$ i.e., the maximum possible number is 7777.
So complement of $(1234)_8$ is $7777 - 1234 = 6543$
- We can able to represent the complements in two forms :
 - i. b-1's complement
 - ii. b's complement
- i. The general form for b-1's complement is $b^n - 1 - x$. That means Subtracting the given number from the maximum possible number of given number system.
 - ii. The general form of b's complement is $b^n - x$. That means Subtracting the given number from the maximum possible number of given number system and adding 1 to it..



Binary system complements :

- As the binary system has **base $r = 2$** . So the two types of complements for the binary system are 2's complement and 1's complement.

1's complement :

- The 1's complement of a number is found by changing all 1's to 0's and all 0's to 1's.
- This is called as taking complement or 1's complement.

Example of 1's Complement is as follows.

$$10101 \Rightarrow 01010$$

2's complement

- The 2's complement of binary number is obtained **by adding 1 to the Least Significant Bit (LSB) of 1's complement of the number**.
- **2's complement = 1's complement + 1**

Example of 2's Complement is as follows.

$$10101 \Rightarrow 01010 + 1 \Rightarrow 01011$$

Examples for complements :

1. $(45)_{10}$:

Max possible number - given number ($99 - 45 = 54$)

9's complement : $(54)_{10}$

10's complement : 9's complement + 1 $\Rightarrow (55)_{10}$

2. $(121)_3$:

Max possible number - given number ($222 - 121 = 101$)

2's complement : $(101)_3$

3's complement : 2's complement + 1 $\Rightarrow (102)_3$

3. $(1234)_8$:

Max possible number - given number ($7777 - 1234 = 6543$)

7's complement : $(6543)_8$

8's complement : 7's complement + 1 $\Rightarrow (6544)_8$

Steps for b-1's complement procedure :

1. Find out the b-1's complement of the number which contains negative sign.
2. Add the complement of second number to the given first number.
3. If the result(x-y) generates a carry then add it to the result directly.
4. If the result(x-y) doesn't generate a carry then find the complement of result and add negative symbol to the result.

Examples :

1. Subtract $(6)_{10}$ from $(84)_{10}$.

Given First number be $(84)_{10}$ and second number be $(6)_{10}$.

As $(84)_{10}$ contains two bits modify $(6)_{10}$ as $(06)_{10}$

Now finding 9's complement of $(06)_{10}$

$$\Rightarrow 99 - 06 = 93$$

Adding the complement of second number to the given first number.

$$\Rightarrow 84 + 93 = 177$$

Here we got an extra bit (carry) as 1 so we need to add it to 77

$$\Rightarrow 77 + 1 = 78$$

$$\therefore (84)_{10} - (6)_{10} = (78)_{10}.$$



2. **Subtract $(84)_{10}$ from $(6)_{10}$.**

Given First number be $(6)_{10}$ and second number be $(84)_{10}$.

As $(84)_{10}$ contains two bits modify $(6)_{10}$ as $(06)_{10}$

Now finding 9's complement of $(84)_{10}$

$$\Rightarrow 99 - 84 = 15$$

Adding the complement of second number to the given first number.

$$\Rightarrow 06 + 15 = 21$$

Here we didn't get an extra bit (carry) so we need to find complement of $(21)_{10}$ and add negative number to the result.

$$\Rightarrow 99 - 21 = 78$$

$$\therefore (84)_{10} - (6)_{10} = -(78)_{10}.$$

3. **Subtract $(76)_{10}$ from $(43)_{10}$.**

Now finding 9's complement of $(76)_{10}$

$$\Rightarrow 99 - 76 = 23$$

$$\Rightarrow 43 + 23 = 66$$

Here we didn't get an extra bit (carry) so we need to find complement of $(66)_{10}$ and add negative number to the result.

$$\Rightarrow 99 - 66 = 33$$

$$\therefore (43)_{10} - (76)_{10} = -(33)_{10}.$$

4. **Subtract $(43)_{10}$ from $(76)_{10}$.**

Now finding 9's complement of $(43)_{10}$

$$\Rightarrow 99 - 43 = 56$$

$$\Rightarrow 76 + 56 = 132$$

Here we got an extra bit (carry) as 1 so we need to add it to 32

$$\therefore (76)_{10} - (43)_{10} = (33)_{10}.$$

5. **Find the value of $(121)_3 - (110)_3$.**

Let us change the given numbers in to decimal numbers.

$$(121)_3 = (16)_{10} \text{ and } (110)_3 = (12)_{10}$$

Now finding 2's complement of $(12)_{10}$

$$\Rightarrow 99 - 12 = 87$$

Adding the complement of second number to the given first number.

$$\Rightarrow 16 + 87 = 103$$

Here we got an extra bit (carry) as 1 so we need to add it to 03

$$\Rightarrow 03 - 1 = 04$$

$$\Rightarrow (4)_{10} = (011)_3.$$

$$\therefore (121)_3 - (110)_3 = (011)_3.$$

Draw Backs of b-1's complement :

- We know that $x + \bar{x} = 0$ and $\bar{x} = b^n - 1 - x$
 $\Rightarrow x + b^n - 1 - x = 0 \Rightarrow b^n - 1 = 0$

For example, $b=2$ and $n=4 \Rightarrow 2^4 - 1 = 0 \Rightarrow 15 = 1111 = 0$

And also 0000 is represented as 0

Thus in b-1's complement we will get many patterns which are redirected to a same value.



b's complement procedure :

1. Find out the b's complement of the number which contains negative sign.
2. Add the complement of second number to the given first number.
3. If the result(x-y) generates a carry then discard it completely.
4. If the result(x-y) doesn't generate a carry then find the complement of result and add negative symbol to the result.

Examples :

1. Find the value of $(84)_{10} - (6)_{10}$.

Given First number be $(84)_{10}$ and second number be $(6)_{10}$.

As $(84)_{10}$ contains two bits modify $(6)_{10}$ as $(06)_{10}$

Now finding 10's complement of $(06)_{10}$

$$\Rightarrow 99 - 06 = 93 + 1 = 94$$

Adding the complement of second number to the given first number.

$$\Rightarrow 84 + 94 = 178$$

Here we got an extra bit (carry) as 1 so discard it.

$$\therefore (84)_{10} - (6)_{10} = (78)_{10}.$$

2. Find the value of $(6)_{10} - (84)_{10}$.

Given First number be $(6)_{10}$ and second number be $(84)_{10}$.

As $(84)_{10}$ contains two bits modify $(6)_{10}$ as $(06)_{10}$

Now finding 10's complement of $(84)_{10}$

$$\Rightarrow 99 - 84 = 15 + 1 = 16$$

Adding the complement of second number to the given first number.

$$\Rightarrow 06 + 16 = 22$$

Here we didn't get an extra bit (carry) so we need to find complement of $(22)_{10}$ and add negative number to the result.

$$\Rightarrow 99 - 22 = 77 + 1 = 78$$

$$\therefore (6)_{10} - (84)_{10} = -(78)_{10}.$$

3. Find the value of $(76)_{10} - (43)_{10}$.

Now finding 10's complement of $(43)_{10}$

$$\Rightarrow 99 - 43 = 56 + 1 = 57$$

$$\Rightarrow 76 + 57 = 133$$

Here we got an extra bit (carry) as 1 so discard it.

$$\therefore (76)_{10} - (43)_{10} = (33)_{10}.$$

4. Find the value of $(43)_{10} - (76)_{10}$.

Now finding 10's complement of $(76)_{10}$

$$\Rightarrow 99 - 76 = 23$$

$$\Rightarrow 23 + 1 = 24$$

$$\Rightarrow 43 + 24 = 67$$

Here we didn't get carry so we need to find 10's complement of $(67)_{10}$

$$\Rightarrow 99 - 67 = 32 + 1 = 33$$

$$\therefore (43)_{10} - (76)_{10} = -(33)_{10}.$$



This Table is Important!

BINARY WITH MSB	UNSIGNED	SIGNED	1's COMPLEMENT	2's COMPLEMENT
000	0	000 (0)	0	0
001	1	001 (1)	1	1
010	2	010 (2)	2	2
011	3	011 (3)	3	3
100	4	100 (-0)	-3	-4
101	5	101 (-1)	-2	-3
110	6	110 (-2)	-1	-2
111	7	111 (-3)	-0	-1

Note :

1. The maximum possible numbers for 3-bit Unsigned number system is $2^n - 1$.
2. The range of 3-bit Unsigned number system is **0 to $2^n - 1$** .
3. The range of 3-bit Signed number system in sign magnitude representation is **0 to $2^{n-1} - 1$ and 0 to $-(2^{n-1} - 1)$** .
4. The range of 3-bit Signed number system in 1's complement is **$-(2^{n-1} - 1)$ to $2^{n-1} - 1$**
5. The range of 3-bit Signed number system in 2's complement is **-2^{n-1} to $2^{n-1} - 1$**

1.5 PROBLEM SOLVING :

1. **How many number of minimum bits are required to represent $(-3)_{10}$ in 2's complement ?**

Solution :

Since $(-3)_{10}$ in 2's complement = 101

$\therefore$ The minimum no. of bits are required to represent $(-3)_{10}$ in 2's complement is 3 bits.

2. **How many number of minimum bits are required to represent $(15)_{10}$ in 2's complement ?**

Solution :

As we know that the range of Signed number system in 2's complement is **-2^{n-1} and $2^{n-1} - 1$**

As $(15)_{10} > 0$ and $< 2^{n-1} - 1$

$$\Rightarrow 2^{n-1} - 1 > 15$$

$$\Rightarrow 2^{n-1} > 16$$

$$\Rightarrow n \geq 5$$

$\therefore$ The minimum no. of bits are required to represent $(15)_{10}$ in 2's complement is greater than or equal to 5 bits.

3. **How many number of minimum bits are required to represent $(-15)_{10}$ in 2's complement.**

Solution :

As we know that the range of Signed number system in 2's complement is **-2^{n-1} and $2^{n-1} - 1$**

As $(-15)_{10} < 0$ and $> -2^{n-1}$

$$\Rightarrow -15 > -2^{n-1}$$

$$\Rightarrow 2^{n-1} \geq 15$$

Let $n = 5$

$$\Rightarrow 2^{5-1} \geq 15$$

$$\Rightarrow 2^4 \geq 15$$

$\therefore$ The minimum no. of bits are required to represent $(-15)_{10}$ in 2's complement is 5 bits.



Note :

[-For Video](#)

1. When we are trying to add two negative numbers, if a carry generates then there exists an overflow [needs space for extra bits to store data].

Example : Adding -62 and -45

2's complement of (-62) is 1000010 and 2's complement of (-45) is 1010011[1010010 + 1]

By adding 1000010(7 bits) and 1010011(7 bits) we will get 10010101 (8 bits) i.e., carry generated.

$$\Rightarrow 10010101 = 1*(-2)^7 + 0*(2)^6 + 0*(2)^5 + 1*(2)^4 + 0*(2)^3 + 1*(2)^2 + 0*(2)^1 + 1*(2)^0$$

$$\Rightarrow -128 + 16 + 4 + 1$$

$$\Rightarrow -(107)$$

2. When we are trying to add one positive and one negative numbers, if a carry generates then when don't consider the overflow [needs space for extra bits to store data].

Example : Adding +62 and -45

2's complement of (+62) is 0111110 and 2's complement of (-45) is 1010011[1010010 + 1]

By adding 1000010(7 bits) and 1010011(7 bits) we will get 10010001 (8 bits) i.e., carry generated.

Here we don't consider left most 1(carry) in 10010001. We will treat it as 0010001.

$$\Rightarrow 0010001 = 0*(-2)^6 + 0*(2)^5 + 1*(2)^4 + 0*(2)^3 + 1*(2)^2 + 0*(2)^1 + 1*(2)^0$$

$$\Rightarrow 16 + 1$$

$$\Rightarrow +(17)$$

3. When we are trying to add two positive numbers, we will consider the overflow whether the carry generates or not.

Example : Adding +62 and +45

2's complement of (+62) is 0111110 and 2's complement of (+45) is 0101101

By adding 1000010(7 bits) and 1010011(7 bits) we will get 1101011 (8 bits) i.e., carry generated.

$$\Rightarrow 1101011 = 1*(-2)^6 + 1*(2)^5 + 0*(2)^4 + 1*(2)^3 + 0*(2)^2 + 1*(2)^1 + 1*(2)^0$$

Since we are getting result as -(21) we need to add a carry (0) before 1101011 to get correct result

$$\Rightarrow 1101011 = 0*(-2)^7 + 1*(2)^6 + 1*(2)^5 + 0*(2)^4 + 1*(2)^3 + 0*(2)^2 + 1*(2)^1 + 1*(2)^0$$

$$\Rightarrow 64 + 32 + 8 + 2 + 1$$

$$\Rightarrow +(107)$$

4. Consider two 2's complement numbers and add them as follows :

$$n1 = a_5a_4a_3a_2a_1$$

$$n1 = b_5b_4b_3b_2b_1$$

$\Rightarrow$ If a carry generates while adding a_4 and b_4 then it is called C_{in}

$\Rightarrow$ If a carry generates while adding a_5 and b_5 then it is called C_{out} .

If $C_{in} = C_{out}$ then there will be **no overflow**.

If $C_{in} \neq C_{out}$ then there will be an **overflow**.

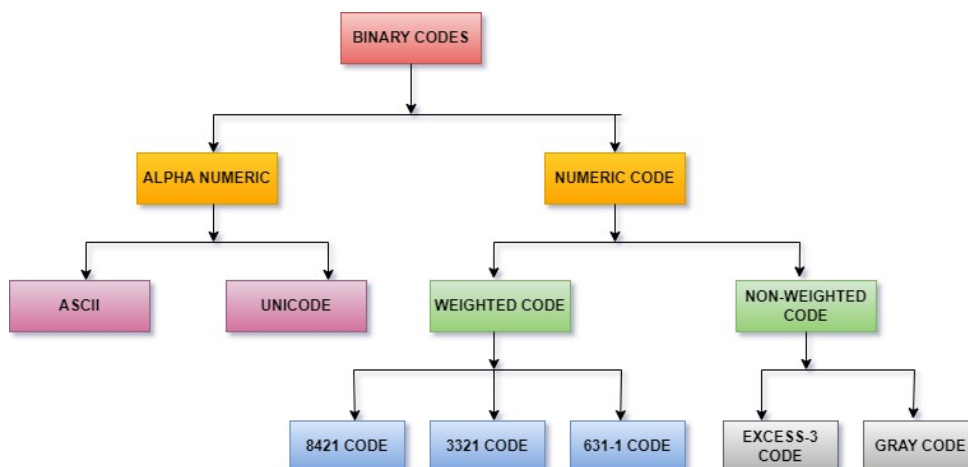
1.6 BINARY CODES

[-For Video](#)

- In the coding, when numbers, letters or words are represented by a specific group of symbols, it is said that the number, letter or word is **being encoded**. The group of symbols is called **as a code**.
- The digital data is represented, stored and transmitted as **group of binary bits**. This group is also called as binary code.
- The binary code is represented by **the number as well as alphanumeric letter**.



CLASSIFICATION OF BINARY CODES :



1. WEIGHTED CODE :

- Weighted binary codes are those binary codes which obey the positional weight principle.
- Each position of the number represents a specific weight. Several systems of the codes are used to express the decimal digits 0 to 9.
- In these codes each decimal digit is represented by a group of four bits.

8421 Code :

[-For Video](#)

- This is also called as Binary Coded Decimal (BCD) code.
- In this code each decimal digit is represented by a 4-bit binary number.
- BCD is a way to express each of the decimal digits with a binary code.
- Generally, In the BCD, with four bits we can represent sixteen numbers (0000 to 1111). But in BCD code only first ten of these are used (0000 to 1001).
- The remaining six code combinations i.e. 1010 to 1111 are invalid in BCD.
- BCD is a sequential code because if we add 1 to existing value then we will get the next value.

DECIMAL	BCD VALUE
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

- We represent 10 in BCD as 0001 0000, 11 as 0001 0001, 12 as 0001 0010 and so on...



3321 code :

[-For Video](#)

- It is a weighted code and also a sequential code.

DECIMAL	3321 Code
0	0000
1	0001
2	0010
3	0011
4	0101
5	1010
6	1100
7	1101
8	1110
9	1111

2. NON WEIGHTED CODE :

[-For Video](#)

- In this type of binary codes, the positional weights are not assigned.
- The examples of non-weighted codes are Excess-3 code and Gray code.

EXCESS-3 Code :

- It is non-weighted code used to express decimal numbers.
- The Excess-3 code words are derived from the 8421 BCD code words by adding $(0011)_2$ or $(3)_{10}$ to each code word in 8421.
- It is also a **sequential code** because if we add 1 to existing value then we will get next value.
- Excess - 3 - code is also known as **self-complementing code** which means 1's complement of an excess - 3 number is the excess - 3 code for the 9's complement of the corresponding decimal number.
- The excess-3 codes are obtained as follows :

$$\text{Decimal Number} \longrightarrow \text{8421 BCD} \xrightarrow[0011]{\text{ADD}} \text{Excess-3}$$

DECIMAL	BCD VALUE	EXCESS-3
0	0000	0011
1	0001	0100
2	0010	0101
3	0011	0110
4	0100	0111
5	0101	1000
6	0110	1001
7	0111	1010
8	1000	1011
9	1001	1100



Gray Code :

[-For Video](#)

- It is the non-weighted code. That means there are no specific weights assigned to the bit position.
- The gray code is **Non sequential code**.
- It has a very special feature that, only one bit will change each time the decimal number is incremented as shown below.
- As only one bit changes at a time, the gray code is called as a **unit distance code**.
- The gray code is a **cyclic code** and it is a **Reflexive code**.

DECIMAL	Gray Code
0	000
1	001
2	011
3	010
4	101
5	111
6	101
7	100

BINARY CODE TO GRAY CODE CONVERSION :

[-For Video](#)

- The Binary to Gray code converter is a logical circuit that is used to convert the binary code into its equivalent Gray code.

Procedure :

1. In the Gray code, the MSB will always be the same as the 1st bit of the given binary number.
2. In order to perform the 2nd bit of the gray code, we perform the exclusive-or (XOR) of the 1st and 2nd bit of the binary number. It means that if both the bits are different, the result will be one else the result will be 0.
3. In order to get the 3rd bit of the gray code, we need to perform the exclusive-or (XOR) of the 2nd and 3rd bit of the binary number. The process remains the same for the 4th bit of the Gray code. Let's take an example to understand these steps.

$$\begin{array}{ccccccc}
 B_4 & \oplus & B_3 & \oplus & B_2 & \oplus & B_1 & \oplus & B_0 \\
 1 & & 1 & & 1 & & 0 & & 1 \\
 \downarrow & & \downarrow & & \downarrow & & \downarrow & & \downarrow \\
 1 & & 0 & & 0 & & 1 & & 1 \\
 G_4 & & G_3 & & G_2 & & G_1 & & G_0
 \end{array}$$

Note : The generalized formula to find n^{th} gray code for given binary code is $b_n = a_{n+1} \oplus a_n$

GRAY CODE TO BINARY CODE CONVERSION :

[-For Video](#)

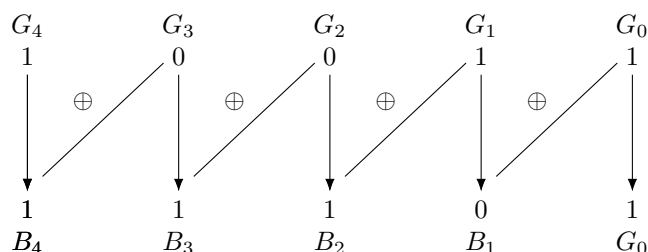
- The Gray to Binary code converter is a logical circuit that is used to convert the gray code into its equivalent binary code. There is the following circuit used to convert the Gray code to binary number.

Procedure :

1. Just like binary to gray, in gray to binary, the 1st bit of the binary number is similar to the MSB of the Gray code.



- The 2nd bit of the binary number is the same as the 1st bit of the binary number when the 2nd bit of the Gray code is 0; otherwise, the 2nd bit is altered bit of the 1st bit of binary number. It means if the 1st bit of the binary is 1, then the 2nd bit is 0, and if it is 0, then the 2nd bit be 1.
- The 2nd step continues for all the bits of the binary number.



1.7 ADDITION & SUBTRACTION ON BCD AND EXCESS-3

BCD ADDITION :

[-For Video](#)

- If the result or sum is a 4-bit binary number which is less than or equal to 9, then the sum is a valid BCD number.
- If the sum is a 4-bit number that is greater than 9 or if a carry is generated, then it is an invalid sum.
- To obtain the corrected result/sum, add 6 (0110) to the 4-bit invalid sum. If a carry is generated when 6 is added, then propagate and add this carry to the next 4-bit group. This step is done to skip the six illegal BCD codes (i.e. 1010, 1011, 1100, 1101, 1110, and 1111).

Examples :

- Perform addition $7 + 4$ in Binary Coded Decimal.

Solution :

Given decimal numbers and their BCD representation is,

$$(7)_{10} = 0111$$

$$(4)_{10} = 0100$$

$$\Rightarrow 0111 + 0100 = 1011 \text{ [Invalid]}$$

Now, adding (6) i.e., 0110 to 1011

$$\Rightarrow 1011 + 0110 = 0001 \ 0001 \text{ [Valid]}$$

$$\therefore 7 + 4 = 0001 \ 0001 \text{ i.e., } 11.$$

- Perform addition $243 + 467$ in Binary Coded Decimal.

Solution :

Given decimal numbers and their BCD representation is,

$$(243)_{10} = 0010 \ 0100 \ 0011$$

$$(467)_{10} = 0110 \ 0110 \ 0111$$

$$\Rightarrow 0010 \ 0100 \ 0011 + 0110 \ 0110 \ 0111 = 0111 \ 1011 \ 1010 \text{ [Invalid]}$$

Now, adding (6) i.e., 0110 to 1011 and 1010

$$\therefore 243 + 467 = 0111 \ 0001 \ 0000 \text{ i.e., } 710.$$



EXCESS-3 ADDITION

[-For Video](#)

1. We have to convert the numbers (which are to be added) into excess 3 forms by adding 0011 with each of the four bit groups them or simply increasing them by 3.
2. Now the two numbers are added using the basic laws of binary addition, there is no exception for this method.
3. Now which of the four groups have produced a carry we have to add 0011 with them and subtract 0011 from the groups which have not produced a carry during the addition.
4. The result which we have obtained after this operation is in Excess 3 form and this is our desired result.

Examples :

1. Perform addition $984 + 599$ in Excess-3.

Solution :

Excess 3 code for 984 : 1100 1011 0111

Excess 3 code for 599 : 1000 1100 1100

Addition : 10100 10111 10011

Remaining bits except carry : 10100 0111 0011

Carry : 1 1

Addition : 10101 1000 0011

If carry generated then add 3 otherwise subtract 3 : +0011 +0011 +0011

Result : 10010 1011 0110

Excess 3 value : 18 11 6

Decimal value : 15 8 3

∴ So final answer of Excess 3 addition is 1583

Problems :

1. Represent $(756)_{10}$ in BCD, 3321, EXCESS-3 and Normal Binary systems ?

Solution :

BCD : 0111 0101 0110

3321 : 1101 1010 1100

EXCESS-3 : 1010 1000 1001

BINARY : 101 111 0100

2. Which of the following is not possible in EXCESS-3 code representation ?
0111, 1011, 1110, 1010

Solution :

Since $0111 = 4$, $1011 = 6$ and $1010 = 7$

∴ 1110 is invalid in EXCESS-3 code.

OPERATIONS ON BINARY NUMBERS :

[-For Video](#)

- We can do operations like Addition, Subtraction, Multiplication and Division on Binary numbers.

a	b	a+b	Carry	a-b	Borrow	ab	a/b
0	0	0	0	0	0	0	Undefined
0	1	1	0	1	1	0	0
1	0	1	0	1	0	0	Undefined
1	1	0	1	0	0	1	1



BCD SUBTRACTION

[-For Video](#)

- We can do BCD subtraction by using both 9's and 10's complements.

By Using 9's Complement :

- In 9's Complement subtraction when 9's Complement of smaller number is added to the larger number carry is generated.
- It is necessary to add this carry to the result. (this is called an end-around carry).
- When larger number is subtracted from smaller one, there is no carry, and the result is in 9's Complement form and negative.
- **Procedure is as follows :**
 1. Find the 9's complement of a negative number.
 2. Add two numbers using BCD addition.
 3. If carry is generated add carry to the result otherwise find the 9's complement of the result and add negative sign to it.

	<u>Regular Subtraction</u>	<u>9's Complement Subtraction</u>
(a)	$\begin{array}{r} 8 \\ - 2 \\ \hline 6 \end{array}$	$\begin{array}{r} 8 \\ + 7 \text{ 9's complement of 2} \\ \hline \textcircled{1} 5 \\ \downarrow + 1 \text{ Add carry to result} \\ \hline 6 \end{array}$
(b)	$\begin{array}{r} 28 \\ - 13 \\ \hline 15 \end{array}$	$\begin{array}{r} 28 \\ + 86 \text{ 9's complement of 13} \\ \hline \textcircled{1} 14 \\ \downarrow + 1 \text{ Add carry to the result} \\ \hline 15 \end{array}$
(c)	$\begin{array}{r} 18 \\ - 24 \\ \hline -6 \end{array}$	$\begin{array}{r} 18 \\ + 75 \text{ 9's complement of 24} \\ \hline 93 \text{ 9's complement of result} \\ \downarrow \text{ (No carry indicates that the} \\ -06 \text{ answer is negative and in} \\ \text{complement form)} \end{array}$

By Using 10's Complement :

- The BCD Subtraction using 10's Complement can be used to perform subtraction by adding the minuend[First number] to the 10's Complement of the subtrahend[Second number] and dropping[discarding] the carry.
- **Procedure is as follows :**
 1. Find the 10's complement of a negative number.
 2. Add two numbers using BCD addition.
 3. If carry is not generated find the 10's Complement of the result.



- Let us see the following examples.

	<u>Regular Subtraction</u>	<u>10's Complement Subtractor</u>
(a)	$\begin{array}{r} 8 \\ - 2 \\ \hline 6 \end{array}$	$\begin{array}{r} 8 \\ + 8 \text{ 10's complement of 2} \\ \hline \cancel{1}6 \text{ Drop carry} \end{array}$
(b)	$\begin{array}{r} 28 \\ - 13 \\ \hline 15 \end{array}$	$\begin{array}{r} 28 \\ + 87 \text{ 10's complement of 13} \\ \hline \cancel{1}15 \text{ Drop carry} \end{array}$
(c)	$\begin{array}{r} 18 \\ - 24 \\ \hline -6 \end{array}$	$\begin{array}{r} 18 \\ + 76 \text{ 10's complement of 24} \\ \hline 94 \text{ 10's complement of result} \\ \downarrow \text{ (No carry indicates that the} \\ -06 \text{ answer is negative and in} \\ \text{the 10's complement form)} \end{array}$

EXCESS-3 SUBTRACTION :

[-For Video](#)

- Follow the below steps to perform excess-3 subtraction.
 - While subtracting A-B, take the complement[either 9's or 10's complement] of B.
 - Add A and complement of B.
 - If carry is generated, add 0011[3] to the sum result.
 - If carry is not generated, subtract 0011 [3] from the sum result.
 - Add the carry with the next higher bit.

- Let us look at some of the examples here.

Example 1 : Subtract $(98)_{10} - (81)_{10}$ in excess-3 code by using 9's complement.

9's complement of 81 is

$$99 - 81 = 18$$

Add 98 and 18 using Excess 3 addition

Excess 3 code for 98 : 1100 1011

Excess 3 code for 18 : 0100 1011

Addition : 10000 10110

Remaining bits except carry : 10000 0110

Carry : 1

Addition : 10001 0110

if carry, add 3 else subtract 3 : +0011 +0011

Result : 0100 1010

Decimal value : 4 9

Excess 3 value : 1 6

The left most bit of the result is 1, called carry and This will be added to 16 i.e., $16+1=17$
So final answer of Excess 3 Subtraction is 17

Example 2 : Subtract $(33)_{10} - (62)_{10}$ in excess-3 code by using 9's complement.



9's complement of 62 is

$$99 - 62 = 37$$

Add 33 and 37 using Excess 3 addition

Excess 3 code for 33 : 0110 0110

Excess 3 code for 37 : 0110 1010

Addition : 1100 10000

Remaining bits except carry : 1100 0000

Carry : 1

Addition : 0111 0000

if carry, add 3 else subtract 3 : -0011 +0011

Result : 1010 0011

Decimal value : 10 3

Excess 3 value : 7 0

9's complement of 70 is $99 - 70 = 29$ and add negative sign to it i.e., -29

So final answer of Excess 3 Subtraction is -29.

Example 3 : Subtract $(98)_{10} - (81)_{10}$ in excess-3 code by using 10's complement.

10's complement of 81 is

$$99 - 81 = 18 + 1 = 19$$

Add 98 and 19 using Excess 3 addition

Excess 3 code for 98 : 1100 1011

Excess 3 code for 19 : 0100 1100

Addition : 10000 10111

Remaining bits except carry : 10000 0111

Carry : 1

Addition : 0001 0111

if carry, add 3 else subtract 3 : +0011 +0011

Result : 0100 1010

Decimal value : 4 10

Excess 3 value : 1 7

So final answer of Excess 3 Subtraction is 17

Example 4 : Subtract $(33)_{10} - (62)_{10}$ in excess-3 code by using 10's complement.

9's complement of 62 is

$$99 - 62 = 37 + 1 = 38$$

Add 33 and 38 using Excess 3 addition

Excess 3 code for 33 : 0110 0110

Excess 3 code for 38 : 0110 1011

Addition : 1100 10001

Remaining bits except carry : 1100 0001

Carry : 1

Addition : 1101 0001

if carry, add 3 else subtract 3 : -0011 +0011

Result : 1010 0100

Decimal value : 10 4

Excess 3 value : 7 1

10's complement of 71 is $99 - 71 = 28$ and add negative sign to it i.e., -29



1.8 ERROR DETECTION AND ERROR CORRECTION

[-For Video](#)

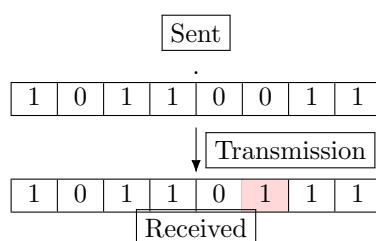
Error :

- Error is a condition when the receiver's information does not match the sender's information. That means a 0 bit may change to 1 or a 1 bit may change to 0.

Types of Errors :

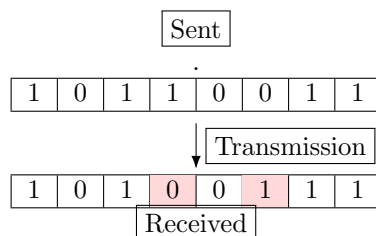
1. Single-Bit Error :

A single-bit error refers to a type of data transmission error that occurs when one bit (i.e., a single binary digit) of a transmitted data unit is altered during transmission, resulting in an incorrect or corrupted data unit.



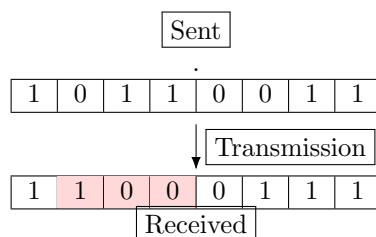
2. Multiple-Bit Error :

A multiple-bit error is an error type that arises when more than one bit in a data transmission is affected. Although multiple-bit errors are relatively rare when compared to single-bit errors, they can still occur, particularly in high-noise or high-interference digital environments.



3. Burst Error :

When several consecutive bits are flipped mistakenly in digital transmission, it creates a burst error. This error causes a sequence of consecutive incorrect values.





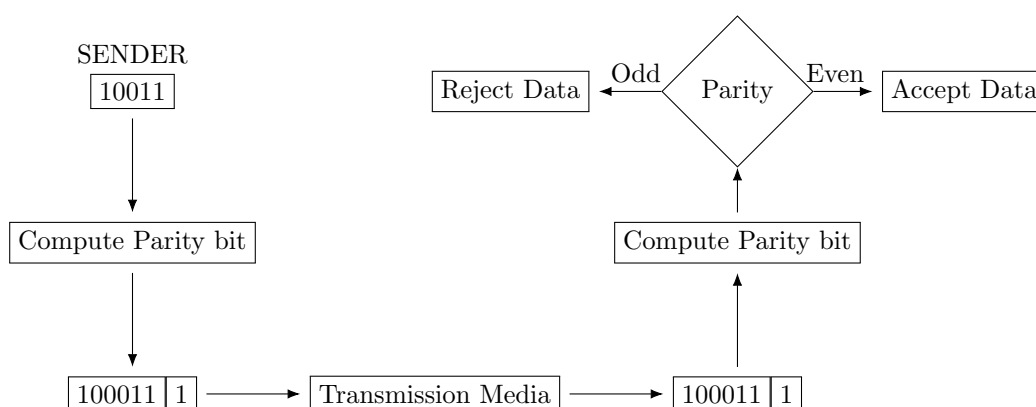
Error Detection Methods :

- To detect errors, a common technique is to introduce redundancy bits that provide additional information. Various techniques for error detection include:
 - Single Parity Check
 - Two-dimensional Parity Check
 - Checksum
 - Cyclic Redundancy Check (CRC)

1. Parity Bit :

[-For Video](#)

- It is a simple error detection method that involves adding an extra bit to a data transmission.
- It works as:
 - 1 is added to the block if it contains an odd number of 1's, and
 - 0 is added if it contains an even number of 1's.
- One extra bit is sent along with the original bits to make number of 1's either even in case of even parity, or odd in case of odd parity.
- The sender while creating a frame counts the number of 1's in it. For example, if even parity is used and number of 1's is even then one bit with value 0 is added. This way number of 1's remains even. If the number of 1's is odd, to make it even a bit with value 1 is added.



DrawBack :

- When more than one bits are erroneous, then it is very hard for the receiver to detect the error.
- So it won't be work for more than 1 bit.

2. Two-dimensional Parity Check :

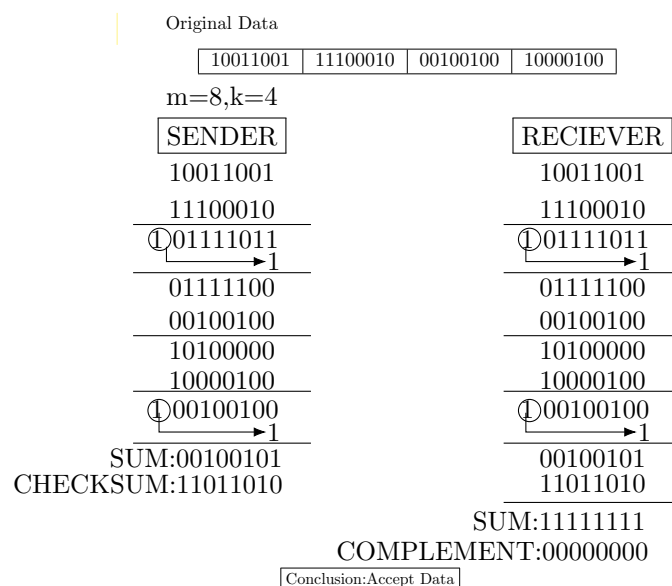
- Two-dimensional Parity check bits are calculated for each row, which is equivalent to a simple parity check bit. Parity check bits are also calculated for all columns, then both are sent along with the data. At the receiving end, these are compared with the parity bits calculated on the received data.

Original Data				
10011001	11100010	00100100	10000100	
	10011001	0	Row Parities	
	11100010	0		
	00100100	0		
	10000100	0		
	11011011	0		
Column Parities				
100110010	111000100	001001000	100001000	110110110
Data to be Sent				



3. Checksum :

- Checksum error detection is a method used to identify errors in transmitted data. The process involves dividing the data into equally sized segments and using a 1's complement to calculate the sum of these segments.
- The calculated sum is then sent along with the data to the receiver. At the receiver's end, the same process is repeated and if all zeroes are obtained in the sum, it means that the data is correct.



DrawBack :

- If one or more bits of a segment are damaged and the corresponding bit or bits of opposite value in a second segment are also damaged.

Error Correction :

- For correcting the errors, one has to know the exact position of the error. For example, If we want to calculate a single-bit error, the error correction code will determine which one of seven bits is in error. To achieve this, we have to add some additional redundant bits.
- For Example, In 4-bits binary number system there will be 16 possible patterns as follows :

Decimal value	Binary value
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111



- If we observe the table properly, Here we need 1 bit of distance between 2 single bit errors i.e., 0000 and 0001. So, we need 2 bits of distance between 2 valid patterns of 1 bit errors i.e., 0000 and 0001.
- As well as we need 3 bits of distance between 2 valid patterns of 2 bit errors i.e., 0000 and 0011.
- As well as we need (t+1) bits of distance between 2 valid patterns of t bit errors.

1.9 HAMMING CODES

[-For Video](#)

- Hamming code is a set of error-correction codes that can be used to detect and correct the errors that can occur when the data is moved or stored from the sender to the receiver.
- It is a technique developed by R.W. Hamming for error correction.
- Redundant bits – Redundant bits are extra binary bits that are generated and added to the information-carrying bits of data transfer to ensure that no bits were lost during the data transfer.
- The number of redundant bits can be calculated using the following formula:

$$2^r \geq m + r + 1$$

or

$$2^p \geq m + p + 1$$

where, p (or) r = redundant bit, m = data bit.

- Suppose the number of data bits is 7, then the number of redundant bits can be calculated using: $2^4 \geq 7 + 4 + 1$ Thus, the number of redundant bits = 4 Parity bits.

Algorithm of Hamming code :

1. Write the bit positions starting from 1 in binary form (1, 10, 11, 100, etc).
2. All the bit positions that are a power of 2 are marked as parity bits (1, 2, 4, 8, etc).
3. All the other bit positions are marked as data bits.
4. **Position 1** - Check 1 bit, then skip 1 bit, check 1 bit and then skip 1 bit and so on..
(Ex - 1,3,5,7,11, etc.)
Position 2 - Check 2 bits, then skip 2 bits, check 2 bits, then skip 2 bits.
(Ex - 2,3,6,7,10,11,14,15, etc.)
Position 4 - Check 4 bits, then skip 4 bits, check 4 bits, then skip 4 bits.
(Ex - 4, 5, 6, 7, 12, 13, 14, 15, etc.)
Position 8 - Check 8 bits, then skip 8 bits, check 8 bits, then skip 8 bits.
(Ex - 8, 9, 10, 11, 12, 13, 14, 15, 24, 25, 26, 27, 28, 29, 30, 31).
5. Since we check for even parity set a parity bit to 1 if the total number of ones in the positions it checks is odd.
6. Set a parity bit to 0 if the total number of ones in the positions it checks is even.

Examples :

1. **Encode a binary word 11001 into the even parity hamming code.**
Given, number of data bits, n =5.
To find the number of redundant bits,
Let us try P=4.
 $2^4 \geq 5 + 4 + 1$
The equation is satisfied and so 4 redundant bits are selected.
So, total code bit = n+P = 9



The redundant bits are placed at bit positions 1, 2, 4 and 8.

Now Construct the bit location table.

Bit Location	9	8	7	6	5	4	3	2	1
Bit designation	D9	P8	D7	D6	D5	P4	D3	P2	P1
Binary representation	1001	1000	0111	0110	0101	0100	0011	0010	0001
Information bits	1		1	0	0		1		
Parity bits		1				1		0	1

Now determine the parity bits

For P1: Bit locations 3, 5, 7 and 9 have three 1s. To have even parity, P1 must be 1.

For P2: Bit locations 3, 6, 7 have two 1s. To have even parity, P2 must be 0.

For P4: Bit locations 5, 6, 7 have one 1s. To have even parity, P3 must be 1.

For P8: Bit locations 8, 9 have one 1s. To have even parity, P2 must be 1.

Thus the encoded 9-bit hamming code is 111001101.

2. Let us assume the even parity hamming code from the above example (111001101) is transmitted and the received code is (110001101). Now from the received code, let us detect and correct the error.

To detect the error, let us construct the bit location table.

Bit Location	9	8	7	6	5	4	3	2	1
Bit designation	D9	P8	D7	D6	D5	P4	D3	P2	P1
Binary representation	1001	1000	0111	0110	0101	0100	0011	0010	0001
Received code	1	1	0	0	0	1	1	0	1

Now Checking the parity bits

For P1 : In locations 1, 3, 5, 7, 9. There is three 1s in this group, which is wrong for even parity.

Hence the bit value for P1 is 1.

For P2 : In locations 2, 3, 6, 7. There is one 1 in this group, which is wrong for even parity. Hence the bit value for P2 is 1.

For P4 : In locations 4, 5, 6, 7. There is one 1 in this group, which is wrong for even parity. Hence the bit value for P3 is 1.

For P8 : In 8, 9. There are two 1s in this group, which is correct for even parity. Hence the bit value for P4 is 0.

The resultant binary word is 0111. It corresponds to the bit location 7 in the above table.

The error is detected in the data bit D7.

The error is 0 and it should be changed to 1.

Thus the corrected code is 111001101.

1.10 LOGIC GATES

[-For Video](#)

- A logic gate is a simple switching circuit that determines whether an input pulse can pass through to the output in digital circuits.
- The building blocks of a digital circuit are logic gates, which execute numerous logical operations that are required by any digital circuit. These can take two or more inputs but only produce one output.
- The mix of inputs applied across a logic gate determines its output. Logic gates use Boolean algebra to execute logical processes.



- Logic gates are found in nearly every digital gadget we use on a regular basis. Logic gates are used in the architecture of our telephones, laptops, tablets, and memory devices.
- The way in which you are arranging the logic gates to get the efficient output is called **Logic Design**.

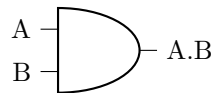
Types of Logic Gates :

- The following types of logic gates are commonly used :
 1. Basic gates [AND, OR, NOT, BUFFER]
 2. Universal gates [NAND, NOR]
 3. Special gates [XOR, XNOR]

1. BASIC GATES :

1. AND :

- An AND gate has a single output and two or more inputs.
- When all of the inputs are 1, the output of this gate is 1.
- The AND gate's Boolean logic is $Y=A.B$ if there are two inputs A and B.
- An AND gate's symbol and truth table are as follows:

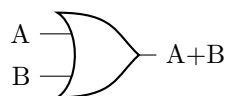


Truth Table

A	B	$Y=A.B$
0	0	0
0	1	0
1	0	0
1	1	1

2. OR :

- Two or more inputs and one output can be used in an OR gate.
- The logic of this gate is that if at least one of the inputs is 1, the output will be 1.
- The OR gate's output will be given by the following mathematical procedure if there are two inputs A and B: $Y=A+B$

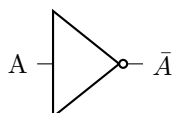


Truth Table

A	B	$Y=A+B$
0	0	0
0	1	1
1	0	1
1	1	1

3. NOT :

- The NOT gate is a basic one-input, one-output gate.
- When the input is 1, the output is 0, and vice versa. A NOT gate is sometimes called an inverter because of its feature.
- If there is only one input A, the output may be calculated using the Boolean equation $Y=\bar{A}$.



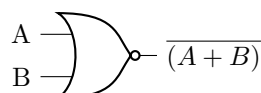
Truth Table

A	$\bar{A}$
0	1
1	0

UNIVERSAL GATES

1. NOR :

- A NOR gate, sometimes known as a “NOT-OR” gate, consists of an OR gate followed by a NOT gate.
- This gate’s output is 1 only when all of its inputs are 0. Alternatively, when all of the inputs are low, the output is high.
- The Boolean statement for the NOR gate is $Y = (A + B)'$ if there are two inputs A and B.

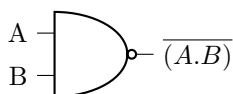


Truth Table

A	B	$Y = (A + B)'$
0	0	1
0	1	0
1	0	0
1	1	0

2. NAND :

- A NAND gate, sometimes known as a ‘NOT-AND’ gate, is essentially a Not gate followed by an AND gate.
- This gate’s output is 0 only if none of the inputs is 0. Alternatively, when all of the inputs are not high and at least one is low, the output is high.
- If there are two inputs A and B, the Boolean expression for the NAND gate is $Y = (A.B)'$



Truth Table

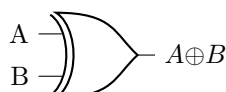
A	B	$Y = A.B$
0	0	1
0	1	1
1	0	1
1	1	0



SPECIAL GATES

1. XOR :

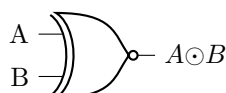
- The Exclusive-OR or 'Ex-OR' gate is a digital logic gate that accepts more than two inputs but gives only one output.
- If any of the inputs is 'High', the output of the XOR Gate is 'High'. If both inputs are 'High', the output is 'Low'. If both inputs are 'Low', the output is 'Low'.
- The Boolean equation for the XOR gate is $Y=A'.B+A.B'$ if there are two inputs A and B.



Truth Table		
A	B	$Y=A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

2. XNOR :

- The Exclusive-NOR or 'EX-NOR' gate is a digital logic gate that accepts more than two inputs but only one output.
- If both inputs are 'High', the output of the XNOR Gate is 'High'. If both inputs are 'Low', the output is 'High'. If one of the inputs is 'Low', the output is 'Low'. If there are two inputs A and B, then the XNOR gate's Boolean equation is: $Y=A.B+A'B'$.



Truth Table		
A	B	$Y=A \odot B$
0	0	1
0	1	0
1	0	0
1	1	1

BOOLEAN ALGEBRA :

[-For Video](#)

- Boolean algebra is a type of logical algebra in which symbols represent logic levels.
- Used to analyze and simplify the circuit diagrams.
- The digits(or symbols) 1 and 0 are related to the logic levels in this algebra.
- At any time, a digital device will be in one of these two binary situations[1 or 0].
- x , $\bar{x}$ are called Boolean Literals.
- $+$, $-$, $.$ are called boolean operators.



1.11 ALGEBRAIC PROPERTIES

-For Video

1. **Annulment law** - When the variable is AND with 0, it will give the result 0, and when the variable is OR with 1, it will give the result 1, i.e.,
 $A.0 = 0$
 $A + 1 = 1$
2. **Identity law** - When the variable is in AND with 1 and OR with 0, the variable remains the same, i.e.,
 $A.1 = A$
 $A + 0 = A$
3. **Idempotent law** - When the variable is AND and OR with itself, the variable remains same or unchanged, i.e.,
 $A.A=A$
 $A+A=A$
4. **Complement Law** - When the variable is AND and OR with its complement, it will give the result 0 and 1 respectively.
 $A.A'=0$
 $A+A'=1$
5. **Double Negation Law** - This law states that, when the variable comes with two negations, the symbol gets removed and the original variable is obtained.
 $((A)')' = A$
6. **Commutative Law** - This law states that no matter in which order we use the variables. It means that the order of variables doesn't matter in this law.
 $A.B = B.A$
 $A+B = B+A$
7. **Associative Law**- This law states that the operation can be performed in any order when the variables priority is of same as '*' and '/'.
 $(A.B).C = A.(B.C)$
 $(A+B)+C = A+(B+C)$
8. **Distributive Law** - This law allows us to open up of brackets. Simply, we can open the brackets in the Boolean expressions.
 $A+(B.C) = (A+B).(A+C)$
 $A.(B+C) = (A.B)+(A.C)$
9. **Absorption Law** - This law allows us for absorbing the similar variables.
 $B+(B.A) = B$
 $B.(B+A) = B$
10. **De Morgan's Law** - The operation of an OR and AND logic circuit will remain same if we invert all the inputs, change operators from AND to OR and OR to AND, and invert the output.
 $(A.B)' = A'+B'$
 $(A+B)' = A'.B'$
11. **Consensus** -

$$xy + \bar{x}z + yz = xy + \bar{x}z$$

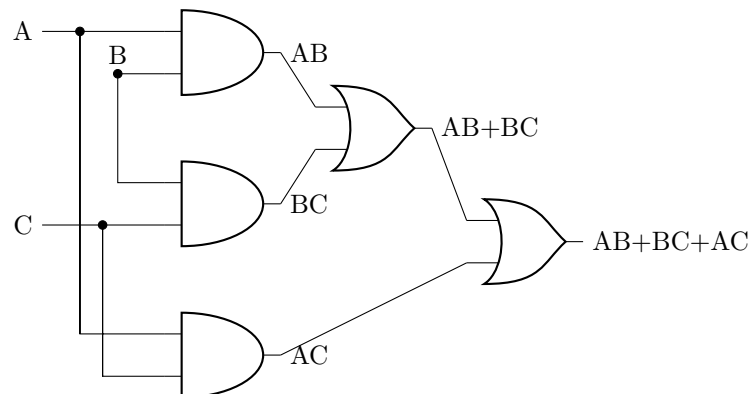
proof :

$$\begin{aligned} &=> xy + \bar{x}z + yz(x+\bar{x}) \\ &=> xy + \bar{x}z + xyz + \bar{x}yz \\ &=> xy(1+z) + \bar{x}z(1+y) \\ &=> xy + \bar{x}z \end{aligned}$$



Designing Logic circuit :

$$f(A,B,C)=AB+BC+CA$$



Switching Expressions :

- **Literal** : variable or its complement
- **Redundant literal** : If value of switching expression is independent of literal x_i then x_i is said to be redundant.

Example : Simplify $T(x,y,z) = x'y'z + yz + xz$

$$\begin{aligned} x'y'z + yz + xz &= z(x'y' + y + x) \\ &= z(x' + y + x) \\ &= z(y + 1) \\ &= z(1) \\ &= z \end{aligned}$$

Thus, literals x and y are redundant in $T(x,y,z)$

- **Examples for simplification :**

$$\begin{aligned} \text{Simplify } T(x,y,z) &= (x + y)[x'(y' + z')] + x'y' + x'z' \\ (x + y)[x'(y' + z')] + x'y' + x'z' &= (x + y)(x + yz) + x'y' + x'z' \\ &= (x + xyz + yx + yz) + x'y' + x'z' \\ &= x + yz + x'y' + x'z' \\ &= x + yz + y' + z' \\ &= x + z + y' + z' \\ &= x + y' + 1 \\ &= 1 \end{aligned}$$

Simplify $T(x,y,z) = x'y + yz' + yz + xy'z'$

$$\begin{aligned} x'y + yz' + yz + xy'z' &= x'y + y(z+z') + z'(y+xy') \\ &= x'y + y.1 + z'[(y+x)(y+y')] \\ &= x'y + y + z'(y+x) \\ &= y(x'+1) + z'(y+x) \\ &= y + z'y + z'x \\ &= y(1+z') + z'x \\ &= y + z'x \end{aligned}$$



1.12 CANONICAL REPRESENTATION FOR SWITCHING OF EXPRESSIONS

[-For Video](#)

- In Boolean algebra, Boolean function can be expressed as Canonical Disjunctive Normal Form known as minterm and some are expressed as Canonical Conjunctive Normal Form known as maxterm.
- In Minterm, we look for the functions where the output results in “1” while in Maxterm we look for function where the output results in “0”.
- We perform Sum of minterm also known as Sum of products (SOP).
- We perform Product of Maxterm also known as Product of sum (POS).
- Boolean functions expressed as a sum of minterms or product of maxterms are said to be in canonical form.

Standard Form :

- A Boolean variable can be expressed in either true form or complemented form. In standard form Boolean function will contain all the variables in either true form or complemented form while in canonical number of variables depends on the output of SOP or POS.
- A Boolean function can be expressed algebraically from a given truth table by forming a :
 - i. minterm for each combination of the variables that produces a 1 in the function and then taking the OR of all those terms.
 - ii. maxterm for each combination of the variables that produces a 0 in the function and then taking the AND of all those terms.

			<i>Minterms</i>	<i>Maxterms</i>
<i>X</i>	<i>Y</i>	<i>Z</i>	<i>Product Terms</i>	<i>Sum Terms</i>
0	0	0	$m_0 = \bar{X} \cdot \bar{Y} \cdot \bar{Z} = \min(\bar{X}, \bar{Y}, \bar{Z})$	$M_0 = X + Y + Z = \max(X, Y, Z)$
0	0	1	$m_1 = \bar{X} \cdot \bar{Y} \cdot Z = \min(\bar{X}, \bar{Y}, Z)$	$M_1 = X + Y + \bar{Z} = \max(X, Y, \bar{Z})$
0	1	0	$m_2 = \bar{X} \cdot Y \cdot \bar{Z} = \min(\bar{X}, Y, \bar{Z})$	$M_2 = X + \bar{Y} + Z = \max(X, \bar{Y}, Z)$
0	1	1	$m_3 = \bar{X} \cdot Y \cdot Z = \min(\bar{X}, Y, Z)$	$M_3 = X + \bar{Y} + \bar{Z} = \max(X, \bar{Y}, \bar{Z})$
1	0	0	$m_4 = X \cdot \bar{Y} \cdot \bar{Z} = \min(X, \bar{Y}, \bar{Z})$	$M_4 = \bar{X} + Y + Z = \max(\bar{X}, Y, Z)$
1	0	1	$m_5 = X \cdot \bar{Y} \cdot Z = \min(X, \bar{Y}, Z)$	$M_5 = \bar{X} + Y + \bar{Z} = \max(\bar{X}, Y, \bar{Z})$
1	1	0	$m_6 = X \cdot Y \cdot \bar{Z} = \min(X, Y, \bar{Z})$	$M_6 = \bar{X} + \bar{Y} + Z = \max(\bar{X}, \bar{Y}, Z)$
1	1	1	$m_7 = X \cdot Y \cdot Z = \min(X, Y, Z)$	$M_7 = \bar{X} + \bar{Y} + \bar{Z} = \max(\bar{X}, \bar{Y}, \bar{Z})$

- From the above table it is clear that minterm is expressed in product format and maxterm is expressed in sum format.
- **Sum of minterms** – The minterms whose sum defines the Boolean function are those which give the 1's of the function in a truth table. Since the function can be either 1 or 0 for each minterm, and since there are 2^n minterms, one can calculate all the functions that can be formed with n variables to be (2^{2^n}) .

Note :

- Complementary for $x' \cdot y' = (\bar{x} \cdot \bar{y}) = \bar{x} + \bar{y} = x + y$
- Duality of $\bar{x} \cdot \bar{y} = \bar{x} + \bar{y}$



Examples :

1. Express the Boolean function $F = xy + x'z$ as a **product of maxterms**.

Solution :

$$\begin{aligned} F &= xy + x'z = (xy + x')(xy + z) = (x + x')(y + x')(x + z)(y + z) = (x' + y)(x + z)(y + z) \\ x' + y &= x' + y + zz' = (x' + y + z)(x' + y + z') \\ x + z &= x + z + yy' = (x + y + z)(x + y' + z) \\ y + z &= y + z + xx' = (x + y + z)(x' + y + z) \\ F &= (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z') \\ &= M_0 * M_2 * M_4 * M_5 \\ &= \pi M(0, 2, 4, 5) \end{aligned}$$

2. $F(A, B, C) = \sum m(1, 4, 5, 6, 7)$

$$F'(A, B, C) = \sum m(0, 2, 3) = m_0 + m_2 + m_3$$

Now, if we take the complement of F' by DeMorgan's theorem, we obtain F in a different form :

$$\begin{aligned} F &= (m_0 + m_2 + m_3)' \\ &= m_0' m_2' m_3' \\ &= M_0 * M_2 * M_3 \\ &= \pi M(0, 2, 3) \end{aligned}$$

3. Convert Boolean expression in standard form $F = y' + xz' + xyz$

$$\begin{aligned} F &= (x + x')y'(z + z') + x(y + y')z' + xyz \\ F &= xy'z + xy'z' + x'y'z + x'y'z' + xyz' + xy'z' + xyz \end{aligned}$$

4. Convert Boolean expression in standard form $F = AB + AC' + BC$

$$\begin{aligned} F &= AB(C + C') + A(B + B')C' + (A + A')BC \\ F &= ABC + ABC' + ABC' + ABC' + ABC + A'BC \\ F &= ABC + ABC' + AB'C' + A'BC \end{aligned}$$

5. Convert Boolean expression in standard form $F = (A + B)(A + C')(B + C')$

$$\begin{aligned} F &= (A + B + C.C')(A + B.B' + C')(A.A' + B + C') \\ F &= (A + B + C)(A + B + C')(A + B' + C')(A' + B + C') \\ F &= \pi M(0, 1, 3, 5) \end{aligned}$$

CONVERSION OF STANDARD SUM OF PRODUCTS INTO STANDARD PRODUCT OF SUMS :

Examples :

1. $f(x, y, z) = \sum m(0, 1, 3, 4, 7)$

$$\begin{aligned} &\Rightarrow (\bar{x}\bar{y}\bar{z}) + (\bar{x}\bar{y}z) + (\bar{x}y\bar{z}) + (x\bar{y}\bar{z}) + (xyz) \\ &\Rightarrow (\bar{x} + \bar{y} + z)(\bar{x} + y + \bar{z})(x + \bar{y} + z) \\ &\Rightarrow \pi M(2, 5, 6) \end{aligned}$$

2. $f(x, y) = x(x + y)$

$$\begin{aligned} &\Rightarrow (x + y.y')(x + y) \\ &\Rightarrow (x + y)(x + y')(x + y) \\ &\Rightarrow (x + y)(x + y') \\ &\Rightarrow \pi M(0, 1) \\ &\Rightarrow \sum m(2, 3) \\ &\Rightarrow xy' + xy \end{aligned}$$



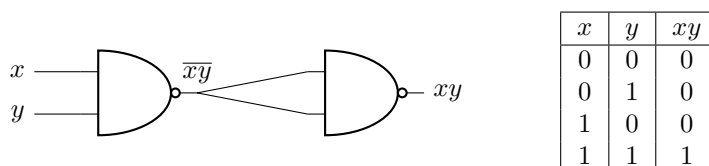
1.13 REALIZATION OF LOGIC GATES USING UNIVERSAL GATES

[-For Video](#)

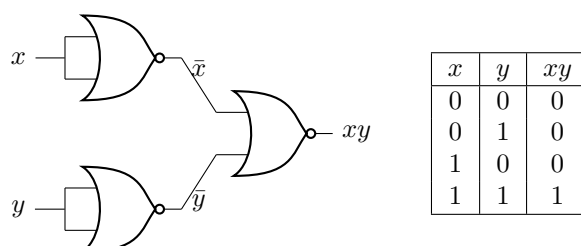
- In Boolean Algebra, the NAND and NOR gates are called universal gates because any digital circuit can be implemented by using any one of these two i.e. any logic gate can be created using NAND or NOR gates only.

1. Implementation of AND Gate using Universal gates.

- Using NAND Gates :** The AND gate can be implemented by using two NAND gates in the below fashion :

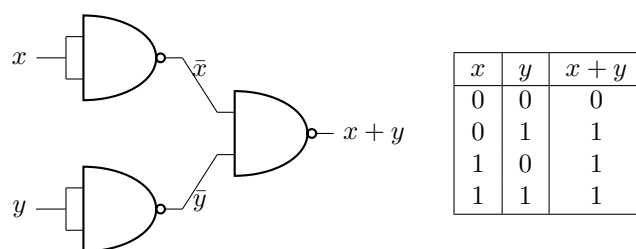


- Using NOR Gates :** The AND gate can be implemented by using two NOR gates in the below fashion :

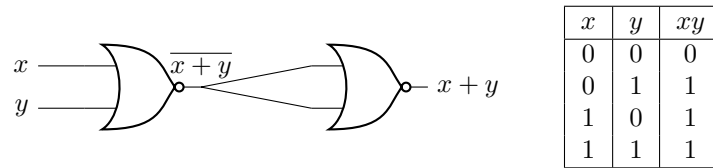


2. Implementation of OR Gate using Universal gates.

- Using NAND Gates :** The OR gate can be implemented using the NAND gate as below:

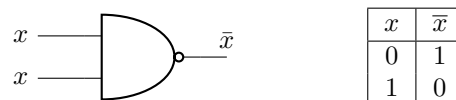


- Using NOR Gates :** The OR gate can be implemented using the NOR gate as below:

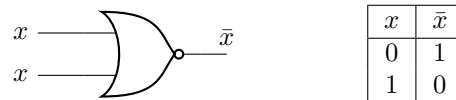


3. Implementation of NOT Gate using Universal gates.

1. **Using NAND Gate :** The NOT gate can be implemented using the NAND gate as below:

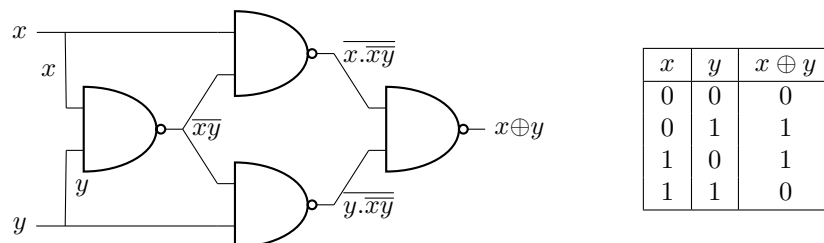


2. **Using NOR Gate :** The NOT gate can be implemented using the NOR gate as below:

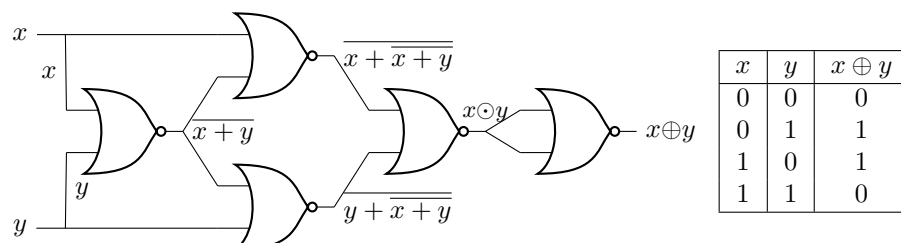


4. Implementation of XOR Gate using Universal gates.

1. **Using NAND Gate :** The XOR gate can be implemented using the NAND gate as below:



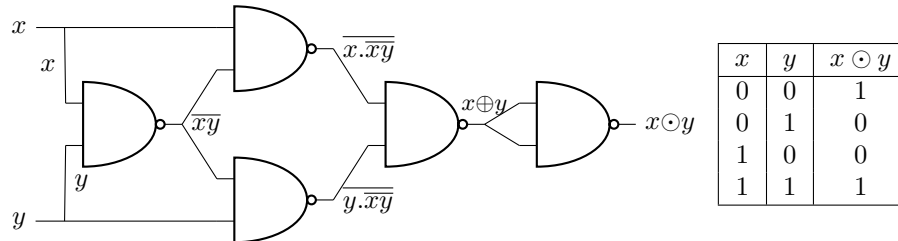
2. **Using NOR Gate :** The XOR gate can be implemented using the NOR gate as below:



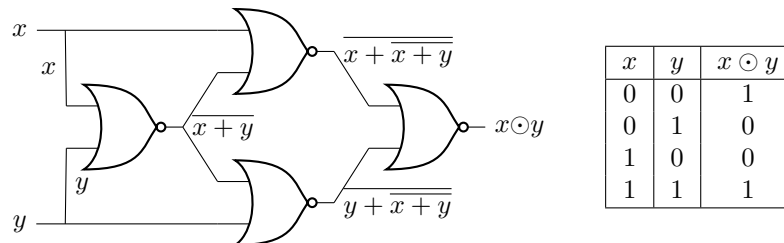


5. Implementation of XNOR Gate using Universal gates.

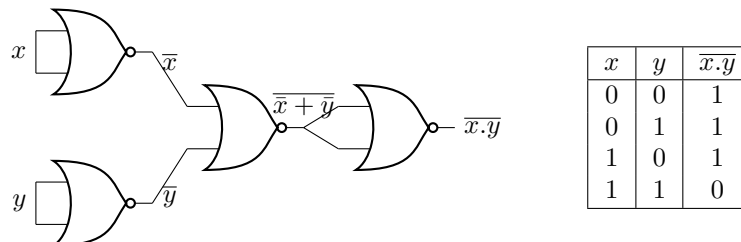
1. **Using NAND Gate :** The XNOR gate can be implemented using the NAND gate as below:



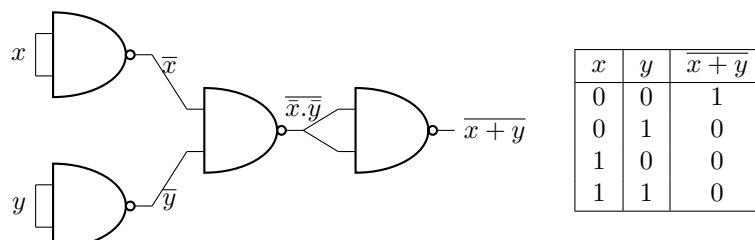
2. **Using NOR Gate :** The XNOR gate can be implemented using the NOR gate as below:



1. **NAND Gate using NOR :** The NAND gate can be implemented using the NOR gate as below:



2. **NOR Gate using NAND :** The NOR gate can be implemented using the NAND gate as below:





Note :

1. For XNOR 5 NAND gates are required to represent through NAND gates.
2. For XNOR 4 NOR gates are required to represent through NOR gates.
3. For XOR 4 NAND gates are required to represent through NAND gates.
4. For XOR 5 NOR gates are required to represent through NOR gates.

NAND-NAND REALIZATION

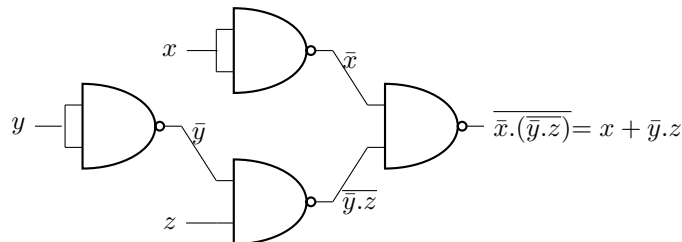
[-For Video](#)

- NAND NAND realization is a concept under which we draw a logic circuit by using only NAND gates.
- The basic logic gates include – NAND gate, NOR gate, OR gate, XOR gate, AND gate and NOT gate.
- Among these NAND gate and NOR gate are universal gates, it means we can make an other logic gate using these two gates.
- In NAND NAND realization the input, output and the truth table of the logical expression remains same, just the circuit gets changed. The changed circuit contains only NAND gate.

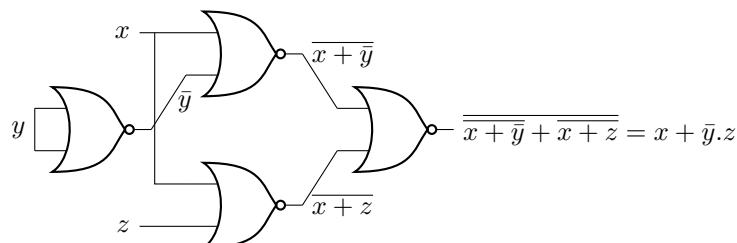
Procedure for NAND NAND Realization :

1. Simplify the given logical expression and then convert it in the SOP form.
2. Draw the circuit for the expression using basic logic gates.
3. Now replace every normal gates by the gates we made using only NAND gate.
4. Draw the final circuit using only NAND gates.

Example for NAND-NAND and NOR-NOR realization : $f(x,y,z) = x + \bar{y}z$
NAND-NAND



NOR-NOR





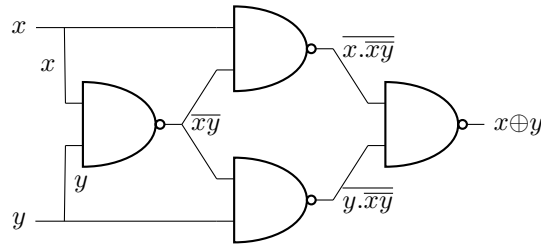
XOR USING UNIVERSAL GATES

XOR using NAND gate :

- An EXOR gate circuit can be made from four NAND gates.
- The output of the XOR gate is the modulo sum of its inputs, i.e.,

$$Y = A \oplus B = A \bar{B} + \bar{A} B$$

Where, A and B are two input variables to the XOR gate, Y is output variable of the XOR gate.



x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0

- The output of the first NAND gate is,

$$Y_1 = \overline{AB}$$

The outputs of the secondary and third NAND gates are,

$$Y_2 = \overline{A \cdot \overline{AB}}$$

$$Y_3 = \overline{B \cdot \overline{AB}}$$

Finally, these two outputs (Y2 and Y3) are connected to the fourth NAND gate. This NAND gate will produce an output which is,

$$Y = \overline{\overline{A \cdot \overline{AB}} \cdot \overline{B \cdot \overline{AB}}}$$

$$Y = \overline{A \cdot \overline{AB} \cdot B \cdot \overline{AB}}$$

$$Y = A(\bar{A} + B) + \bar{B}(\bar{A} + \bar{B})$$

$$Y = A \cdot \bar{A} + A \cdot B + \bar{A} \cdot \bar{B} + B \cdot \bar{B}$$

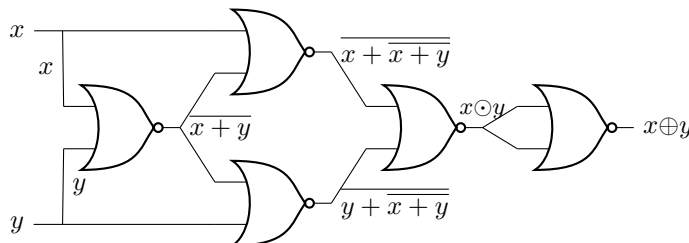
$$Y = A \cdot \bar{B} + \bar{A} \cdot B$$

$$Y = A \oplus B$$

This is the output of the XOR gate. Hence, in this way, we can implement the XOR gate from NAND gates only.

XOR using NOR gate only :

- To design the circuit diagram of an XOR gate using only NOR gates, a minimum of five NOR gates are required.

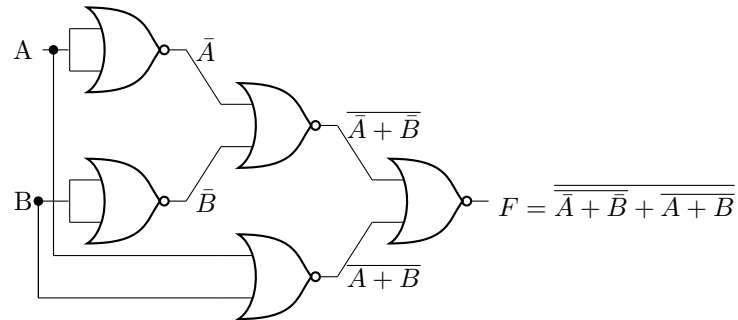


x	y	$x \oplus y$
0	0	0
0	1	1
1	0	1
1	1	0



- Derivation :

$$F = \overline{A}B + A\overline{B}$$



$$F = \overline{A}B + A\overline{B} + A\overline{A} + B\overline{B}$$

$$F = (\overline{A} + \overline{B})(A + B)$$

Taking complement on both sides, we get:

$$\overline{F} = \overline{(\overline{A} + \overline{B})(A + B)}$$

Using de Morgan's Law, we get:

$$\overline{F} = \overline{(\overline{A} + \overline{B})} + \overline{(A + B)}$$

Once again taking complement on both sides, we get:

$$F = \overline{\overline{(\overline{A} + \overline{B})} + \overline{(A + B)}}$$

Hence, this Boolean expression is equivalent to the output of the XOR gate and it can be implemented using NOR gates only.



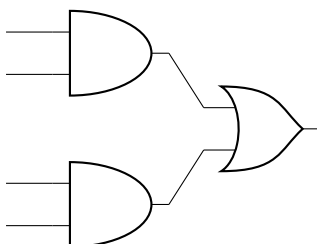
UNIT-2

2.1 SYNTHESIS OF LOGIC GATES

[-For Video](#)

TWO LEVEL SYNTHESIS :

- The term “two-level logic” refers to a logic design that uses no more than two logic gates between input and output. This does not mean that the entire design will only have two logic gates, but it does mean that the single path from input to output will only have two logic gates.



- We explore four logic gates in two-level logic implementation: AND Gate, OR Gate, NAND Gate, and NOR Gate.
- There are a total of 16 two-level logic combinations if we choose one of these four gates at the first level and one at the second level.
- Each two-level combination implements a separate logic function. These 16 combinations are divided into two categories.
 - Degenerative form of logic gate Combination
 - Non-Degenerative form of logic gate Combination.
- The 16 possibilities are shown below:

Degenerative Forms	Non-Degenerative Forms
AND-AND	AND-OR
OR-OR	OR-AND
AND-NAND	NAND-NAND
OR-NOR	NOR-NOR
NAND-NOR	AND-NOR
NOR-NAND	NAND-AND
NAND-OR	OR-NAND
NOR-AND	NOR-OR

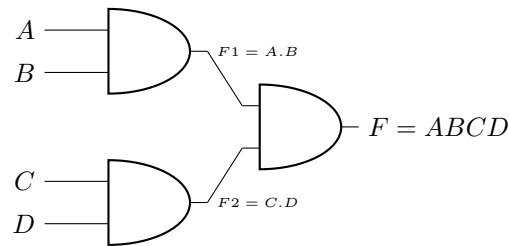
1. Degenerative form of logic gate Combination

[-For Video](#)

- Degenerative form occurs when the output of a two-level logic realization can be achieved with only one logic gate.

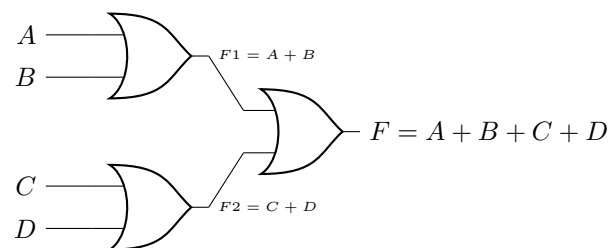
1. AND-AND implementation :

- Because the entire function results in an AND function of all the inputs, this AND-AND gate combination is a degenerate form.
- The outputs of first-level logic gates: $F_1=AB$ and $F_2=CD$. These outputs are applied as inputs of the second level, so the output of the second level is $F=F_1F_2$, which means $F=ABCD$.



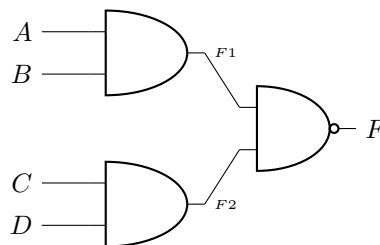
2. OR-OR implementation :

- The outputs of first-level logic gates: $F1=A+B$ and $F2=C+D$. These outputs are applied as inputs of the second level, so the output of the second level is $F=(F1+F2)$ which means $F=(A+B+C+D)$.



3. AND-NAND implementation :

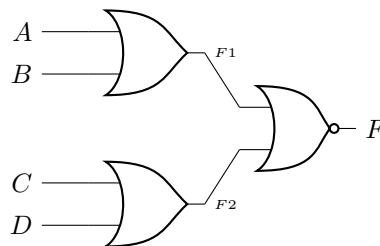
- AND gates are present in the first level of this logic implementation, while NAND gates are present in the second level.



- The outputs of first-level logic gates: $F1=AB$ and $F2=CD$. These outputs are applied as inputs of the second level, so the output of the second level is $F= (F1F2)'$ which means $F=(ABCD)'$.

4. OR-NOR implementation :

- OR-NOR combination of gates results in NOR logic function. And this degenerate form can be used for the NOR function with multiple inputs.

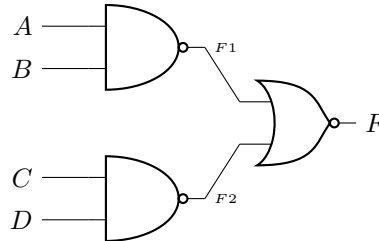




- The outputs of first-level logic gates: $F1=A+B$ and $F2=C+D$. These outputs are applied as inputs of the second level, so the output of the second level is $F=(F1+F2)'$ which means $F=(A+B+C+D)'$.

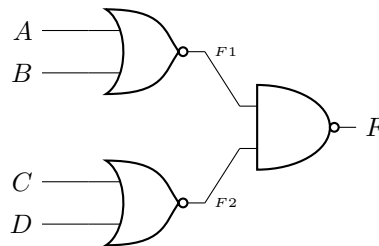
5. **NAND-NOR implementation :**

- The outputs of first level logic gates: $F1=(AB)'$ and $F2=(CD)'$. These outputs are applied as inputs of the second level, so the output of the second level is $F=(F1+F2)'$ which means $F=((AB)'+(CD)')'$.



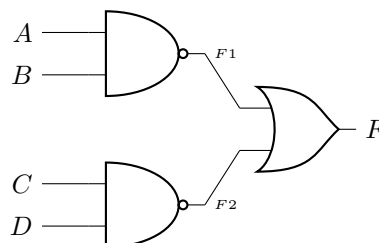
6. **NOR-NAND implementation :**

- The outputs of first level logic gates: $F1=(A+B)'$ and $F2=(C+D)'$. These outputs are applied as inputs of the second level, so the output of the second level is $F=(F1.F2)'$ which means $F=((A+B)')(C+D)')$.



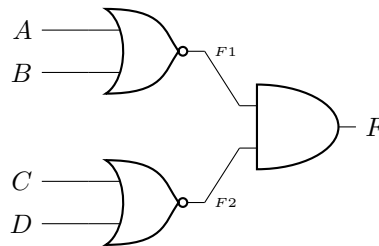
7. **NAND-OR implementation :**

- The outputs of first level logic gates: $F1=(AB)'$ and $F2=(CD)'$. These outputs are applied as inputs of the second level, so the output of the second level is $F=(F1+F2)$ which means $F=((AB)'+(CD)')=(ABCD)'$.



8. **NOR-AND implementation :**

- The outputs of first level logic gates: $F1=(A+B)'$ and $F2=(C+D)'$. These outputs are applied as inputs of the second level, so the output of the second level is $F=(F1.F2)$ which means $F=((A+B)')(C+D)')=(A+B+C+D)'$.



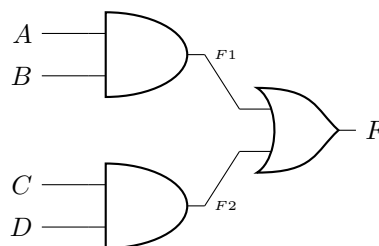
2. Non-Degenerative Forms :

[-For Video](#)

- A non-degenerative form occurs when the output of a two-level logic realization cannot be achieved using a single logic gate.
- Non-degenerate forms are two-level logic combinations that implement the Sum of Product form or the Product of Sum form.

1. AND-OR implementation :

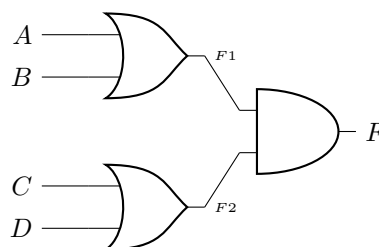
- The first level gate in a While-OR combination is an AND gate, and the second level gate is an OR gate. As shown in the diagram below, this combination implements the Sum of Product (SOP) form.



- The outputs of first-level logic gates: $F1=AB$ and $F2=CD$. These outputs are applied as inputs of the second level, so the output of the second level is $F=(F1+F2)$ which means $F=(AB+CD)$.

2. OR-AND implementation :

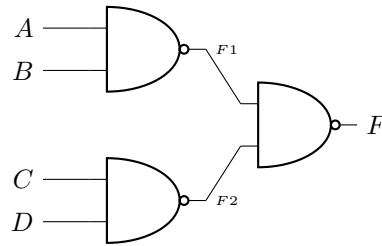
- The first level gate in an OR-AND combination is an OR gate, and the second level gate is an AND gate. The Product of Sum form is implemented with an OR-AND combination.



- The outputs of first-level logic gates: $F1=(A+B)$ and $F2=(C+D)$. These outputs are applied as inputs of the second level, so the output of the second level is $F=(F1.F2)$ which means $F=(A+B)(C+D)$.

3. NAND-NAND implementation :

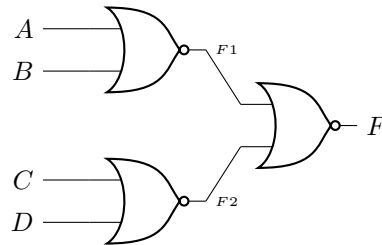
- NAND is a universal gate, and its NAND-NAND combination, like the AND-OR combination, is used to produce the Sum of Product form.



- The outputs of first-level logic gates: $F1=(AB)'$ and $F2=(CD)'$. These outputs are applied as inputs of the second level, so the output of the second level is $F=(F1.F2)'$ which means $F=(AB)+(CD)$.

4. NOR-NOR implementation :

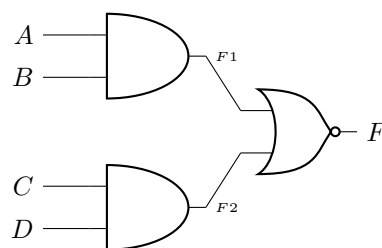
- NOR is also a universal gate and its NOR-NOR combination can be used to implement the Product of Sum form.



- The outputs of first-level logic gates: $F1=(A+B)'$ and $F2=(C+D)'$. These outputs are applied as inputs of the second level, so the output of the second level is $F=(F1+F2)'$ which means $F=((A+B)'+(C+D)')'=(A+B)(C+D)$.

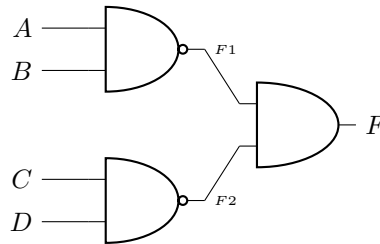
5. AND-NOR implementation :

- The outputs of first-level logic gates: $F1=(AB)$ and $F2=(CD)$. These outputs are applied as inputs of the second level, so the output of the second level is $F=(F1+F2)'$ which means $F=(AB+CD)'$.



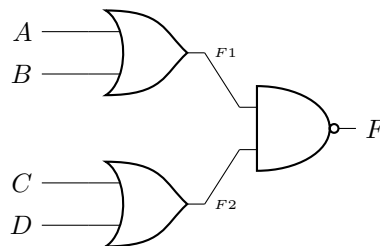
6. NAND-AND implementation :

- The outputs of first-level logic gates: $F1=(AB)'$ and $F2=(CD)'$. These outputs are applied as inputs of the second level, so the output of the second level is $F=(F1F2)$ which means $F=(AB)'(CD)'=(AB+CD)'$.



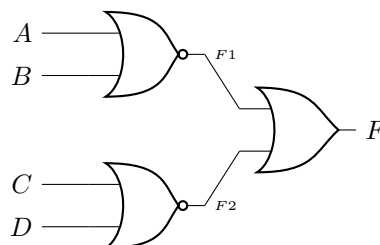
7. OR-NAND implementation :

- The outputs of first-level logic gates: $F1=(A+B)$ and $F2=(C+D)$. These outputs are applied as inputs of the second level, so the output of the second level is $F=(F1F2)'$ which means $F=[(A+B)(C+D)]'$.



8. NOR-OR implementation :

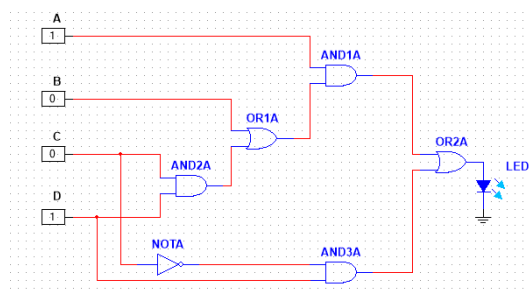
- The outputs of first-level logic gates: $F1=(A+B)'$ and $F2=(C+D)'$. These outputs are applied as inputs of the second level, so the output of the second level is $F=(F1+F2)$, which means $F=(A+B)' + (C+D)' = [(A+B)(C+D)]'$.



MULTILEVEL SYNTHESIS

[-For Video](#)

- It is not in the form of Sum of products and product of sums.
- Example-1 :** $f=A(B+CD) + \bar{C}D$





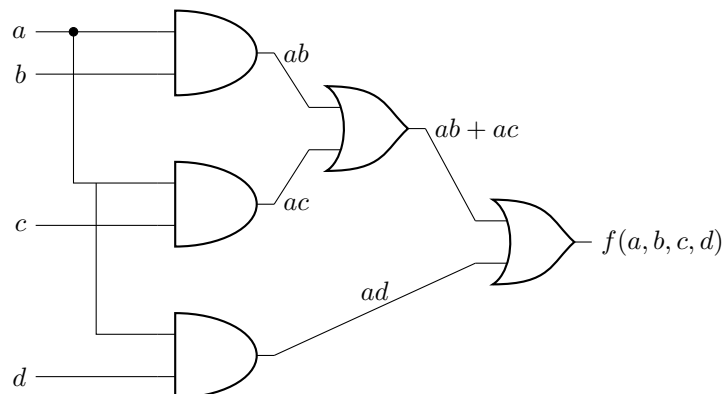
- **Example-2 :** Solve and design the circuit for the function $f(a,b,c,d) = ab\bar{c}\bar{d} + ab\bar{c}d + a\bar{b}\bar{c}d + abcd + a\bar{b}cd + abc\bar{d} + a\bar{b}c\bar{d}$.

Solution :

$$\begin{aligned}
 &\Rightarrow ab\bar{c}(\bar{d}+d) + a\bar{b}\bar{c}d + acd(b+\bar{b}) + ac\bar{d}(b+\bar{b}) \\
 &\Rightarrow ab\bar{c} + a\bar{b}\bar{c}d + acd + ac\bar{d} \\
 &\Rightarrow ab\bar{c} + a\bar{b}\bar{c}d + ac \\
 &\Rightarrow a(b\bar{c} + c) + a\bar{b}\bar{c}d \\
 &\Rightarrow a(b+c)(c+\bar{c}) + a\bar{b}\bar{c}d \\
 &\Rightarrow ac + ab + a\bar{b}\bar{c}d \\
 &\Rightarrow ac + a(b + \bar{b}\bar{c}d) \\
 &\Rightarrow ac + a(b+\bar{b})(b+\bar{c}d) \\
 &\Rightarrow ac + a(b+\bar{b})(b+\bar{c}d) \\
 &\Rightarrow ac + ab + a\bar{c}d \\
 &\Rightarrow ac + a\bar{c}d + ab \\
 &\Rightarrow a(c + \bar{c}d) + ab \\
 &\Rightarrow a(c+d)(c+\bar{c}) + ab \\
 &\Rightarrow a(c+d) + ab \\
 &\Rightarrow \mathbf{ab + ac + ad}
 \end{aligned}$$

Thus the given function requires three 2-input AND gates and two 2-input OR gates.

Circuit Diagram



2.2 K-MAPS(Karnaugh Map)

[-For Video](#)

- We can solve **boolean functions**
 - By using boolean theorems
 - By using K-Maps.
- K-Maps is used to **minimize the boolean expressions without using boolean theorems.**
- It is a **diagrammatic representation of truth tables.**
- K-map can take two forms like **Sum of Product (SOP)** and **Product of Sum (POS)** according to the **need of problem.**
- For a boolean expression consisting of **n-variables, number of cells required in K Map = 2^n cells.**



[-For Video](#)

2- Variable K-Map :

- Two variable K Map is drawn for a boolean expression consisting of two variables.
- The number of cells present in two variable K Map = $2^2 = 4$ cells.
- So, for a boolean function consisting of two variables, we draw a 2 x 2 K Map.

	0	1
0		
1		

3- Variable K-Map :

[-For Video](#)

- Three variable K Map is drawn for a boolean expression consisting of three variables.
- The number of cells present in three variable K Map = $2^3 = 8$ cells.
- So, for a boolean function consisting of three variables, we draw a 2 x 4 K Map.

	00	01	11	10
0				
1				

4- Variable K-Map :

[-For Video](#)

- Four variable K Map is drawn for a boolean expression consisting of four variables.
- The number of cells present in four variable K Map = $2^4 = 16$ cells.
- So, for a boolean function consisting of four variables, we draw a 4 x 4 K Map.



	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

Karnaugh Map Simplification Rules :

- To minimize the given boolean function,
 - Draw a K Map according to the number of variables it contains.
 - Identify **minterms or maxterms** as given in problem and fill the K Map with 0's and 1's according to its function.
 - Then, **minimize the function in accordance with the following rules.**

Rule-1 :

- We can either group 0's with 0's or 1's with 1's but we can not group 0's and 1's together.
- X representing don't care can be grouped with 0's as well as 1's.**

Rule-2 :

- Groups may overlap each other.

Rule-3 :

- We can only **create a group whose number of cells can be represented in the power of 2.**
- In other words, a group **can only contain 2^n** i.e. 1, 2, 4, 8, 16 and so on number of cells.

	00	01	11	10
0	1	1	1	1
1				1

Incorrect

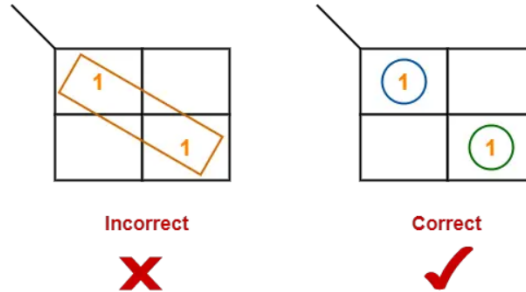
	00	01	11	10
0	1	1	1	1
1				1

correct



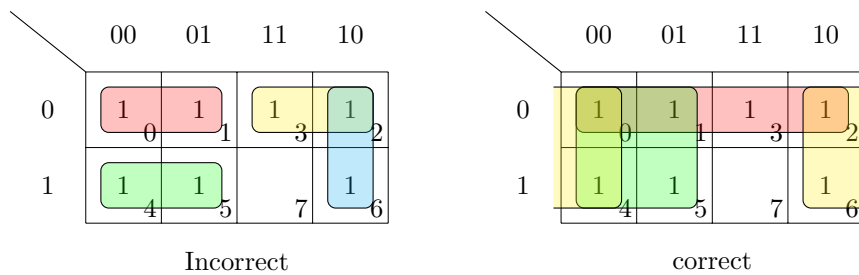
Rule-4 :

- Groups can be only either horizontal or vertical.
- We can not create groups of diagonal or any other shape.



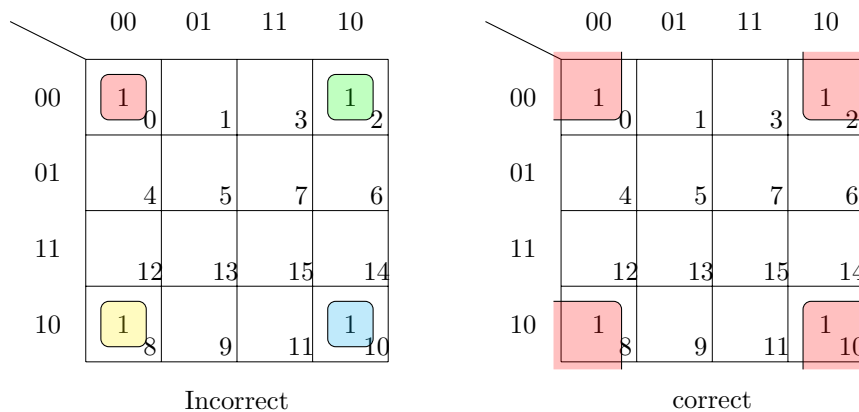
Rule-5 :

- Each group should be as large as possible.
- These groups are called as sub cubes.



Rule-6 :

- Opposite grouping and corner grouping are allowed.
- The example of opposite grouping is shown illustrated in Rule-05.
- The example of corner grouping is shown below.



**Rule-7 :**

- There should be as few groups as possible.

NOTE :

1. The number of minterms is equal to number of sub cubes.
2. Each and every size of minterm is depends on size of sub cube i.e., 2^m cells $\rightarrow$ m-n literals.

PROBLEMS :

1. Minimize the following boolean function

$$F(A, B, C, D) = \sum m(0, 1, 2, 5, 7, 8, 9, 10, 13, 15)$$

Solution :

Since the given boolean expression has 4 variables, so we draw a 4 x 4 K Map.

We fill the cells of K Map in accordance with the given boolean function.

		CD			
		00	01	11	10
AB	00	1	1		1
	01		1	1	
	11		1	1	
	10	1	1		1

$$\text{Now, } F(A, B, C, D) = (A'B + AB)(C'D + CD) + (A'B' + A'B + AB + AB')C'D + (A'B' + AB')(C'D' + CD')$$

$$= BD + C'D + B'D'$$

2. Minimize the following boolean function

$$F(A, B, C) = \sum m(0, 1, 6, 7) + \sum d(3, 5)$$

Solution :

Since the given boolean expression has 3 variables, so we draw a 2 x 4 K Map.

We fill the cells of K Map in accordance with the given boolean function.



		BC			
		00	01	11	10
A	0	1 0	1 1	X 3	0 2
	1	0 4	X 5	1 7	1 6

$$\begin{aligned}\text{Now, } F(A, B, C) &= A'(B'C' + B'C) + A(BC + BC') \\ &= A'B' + AB\end{aligned}$$

Thus, minimized boolean expression is $F(A, B, C) = AB + A'B'$

3. Minimize the following boolean function

$$F(A, B, C) = \sum m(1, 2, 5, 7) + \sum d(0, 4, 6)$$

Solution :

Since the given boolean expression has 3 variables, so we draw a 2 x 4 K Map.

We fill the cells of K Map in accordance with the given boolean function.

		BC			
		00	01	11	10
A	0	X 0	1 1	0 3	1 2
	1	X 4	1 5	1 7	X 6

$$\begin{aligned}F(A, B, C) &= (A + A')(B'C' + B'C) + A(B'C' + B'C + BC + BC') + (A + A')(B'C' + BC') \\ &= B' + A + C'\end{aligned}$$

Thus, minimized boolean expression is $F(A, B, C) = A + B' + C'$

Note :

- The “Don’t Care” conditions allow us to replace the empty cell of a K-Map to form a grouping of the variables which is larger than that of forming groups without don’t care.
- While forming groups of cells, we can consider a “Don’t Care” cell as 1 or 0 or we can also ignore that cell.

4. Minimize the following boolean function

$$F(A, B, C) = \sum m(0, 1, 6, 7) + \sum d(3, 4, 5)$$

Solution :

Since the given boolean expression has 3 variables, so we draw a 2 x 4 K Map.

We fill the cells of K Map in accordance with the given boolean function.



		<i>BC</i>			
		00	01	11	10
<i>A</i>	0	1 0	1 1	X 3	0 2
	1	X 4	X 5	1 7	1 6

$$F(A, B, C) = (A + A')(B'C' + B'C) + A(B'C' + B'C + BC + BC')$$

$$= B' + A$$

Thus, minimized boolean expression is $F(A, B, C) = A + B'$

5. Minimize the following boolean function

$$F(A, B, C, D) = \sum m(0, 1, 3, 5, 7, 8, 9, 11, 13, 15)$$

Solution :

Since the given boolean expression has 4 variables, so we draw a 4 x 4 K Map.

We fill the cells of K Map in accordance with the given boolean function.

Then, we form the groups in accordance with the above rules.

		<i>CD</i>			
		00	01	11	10
<i>AB</i>	00	1 0	1 1	1 3	0 2
	01	0 4	1 5	1 7	0 6
	11	0 12	1 13	1 15	0 14
	10	1 8	1 9	1 11	0 10

$$F(A, B, C, D) = (A'B' + A'B + AB + AB')(C'D + CD) + (A'B' + AB')(C'D' + C'D)$$

$$= D + B'C'$$

Thus, minimized boolean expression is $F(A, B, C, D) = B'C' + D$



6. Minimize the following boolean function

$$F(A, B, C, D) = \sum m(1, 3, 4, 6, 8, 9, 11, 13, 15) + \sum d(0, 2, 14)$$

Solution :

Since the given boolean expression has 4 variables, so we draw a 4 x 4 K Map.

We fill the cells of K Map in accordance with the given boolean function.

		<i>CD</i>			
		00	01	11	10
<i>AB</i>	00	X 0	1 1	1 3	X 2
	01	1 4	0 5	0 7	1 6
	11	0 12	1 13	1 15	X 14
	10	1 8	1 9	1 11	0 10

$$\begin{aligned}
 F(A, B, C, D) &= (AB + AB')(C'D + CD) + (A'B' + AB')(C'D + CD) + (A'B' + AB')(C'D' + C'D) \\
 &\quad + (A'B' + A'B)(C'D' + CD') \\
 &= AD + B'D + B'C' + A'D'
 \end{aligned}$$

Thus, minimized boolean expression is $F(A, B, C, D) = AD + B'D + B'C' + A'D'$

7. Minimize the following boolean function

$$F(A, B, C, D) = \sum m(0, 2, 8, 10, 14) + \sum d(5, 15)$$

Solution :

Since the given boolean expression has 4 variables, so we draw a 4 x 4 K Map.

We fill the cells of K Map in accordance with the given boolean function.

		<i>CD</i>			
		00	01	11	10
<i>AB</i>	00	1 0	0 1	0 3	1 2
	01	0 4	X 5	0 7	0 6
	11	0 12	0 13	X 15	1 14
	10	1 8	0 9	0 11	1 10

$$F(A, B, C, D) = (AB + AB')CD' + (A'B' + AB')(C'D' + CD')$$



$$= ACD' + B'D'$$

Thus, minimized boolean expression is $F(A, B, C, D) = ACD' + B'D'$

8. Minimize the following boolean function

$$F(A, B, C, D) = \sum m(3, 4, 5, 7, 9, 13, 14, 15)$$

Solution :

Since the given boolean expression has 4 variables, so we draw a 4 x 4 K Map.

We fill the cells of K Map in accordance with the given boolean function.

		<i>CD</i>			
		00	01	11	10
<i>AB</i>	00	0	1	1	2
	01	1	1	1	6
	11		1	1	1
	10		1		10

$$\begin{aligned} F(A, B, C) &= A'B(C'D' + C'D) + (A'B' + A'B)(CD) + (AB + AB')(C'D) + AB(CD + CD') \\ &= A'BC' + A'CD + AC'D + ABC \end{aligned}$$

Thus, minimized boolean expression is $F(A, B, C, D) = A'BC' + A'CD + AC'D + ABC$

9. Minimize the following boolean function

$$F(W, X, Y, Z) = \sum m(1, 3, 4, 6, 9, 11, 12, 14)$$

Solution :

Since the given boolean expression has 4 variables, so we draw a 4 x 4 K Map.

We fill the cells of K Map in accordance with the given boolean function.

$$\begin{aligned} F(W, X, Y, Z) &= (W'X + WX)(Y'Z' + YZ') + (W'X' + WX')(Y'Z + YZ) \\ &= XZ' + X'Z \\ &= X \oplus Z \end{aligned}$$

Thus, minimized boolean expression is $F(W, X, Y, Z) = X \oplus Z$



		YZ			
		00	01	11	10
WX	00	0	1	1	2
	01	1			1
	11	1			1
	10		1	1	

2.3 VARIOUS IMPLICANTS IN K-MAPS

[-For Video](#)

Implicant :

- Implicant is a product/minterm term in Sum of Products (SOP) or sum/maxterm term in Product of Sums (POS) of a Boolean function.

Ex : consider a boolean function, $F = AB + ABC + BC$.

Implicants are AB, ABC, and BC.

Prime Implicants :

- A group of squares or rectangles made up of a bunch of adjacent minterms which is allowed by the definition of K-Map are called prime implicants(PI) i.e. all possible groups formed in K-Map.
- Simply, it is an implicant which is not a sub part of other implicant.

		00	01	11	10
0	0	1	1		
	1				
1	4		1	1	
	5				

No.of Prime Implicants=3

Essential Prime Implicants :

- These are those subcubes(groups) that cover at least one minterm that can't be covered by any other prime implicant. Essential prime implicants(EPI) are those prime implicants that always appear in the final solution.



	00	01	11	10
0	1 0	1 1		3 2
1		4 1	5 1	7 6

No.of Essential Prime Implicants=2

Redundant Prime Implicants :

- The prime implicants for which each of its minterm is covered by some essential prime implicant are redundant prime implicants(RPI). This prime implicant never appears in the final solution.

	00	01	11	10
0	1 0	1 1		3 2
1		4 1	5 1	7 6

No.of Redundant Prime Implicants=1

Selective Prime Implicants :

- The prime implicants for which are neither essential nor redundant prime implicants are called selective prime implicants(SPI). These are also known as non-essential prime implicants. They may appear in some solution or may not appear in some solution.

	00	01	11	10
0	1 0	1 1		3 2
1		4 1	5 1	7 6

No.of Selective Prime Implicants=2



Examples :

1. $F = \sum(1, 5, 6, 7, 11, 12, 13, 15)$, find number of implicant, PI, EPI, RPI and SPI.

	00	01	11	10
00	0	1	3	2
01	4	1	1	6
11	1	1	1	14
10	8	9	1	10

Expression : $BD + A'C'D + A'BC + ACD + ABC'$

No. of Implicants = 8

No. of Prime Implicants(PI) = 5

No. of Essential Prime Implicants(EPI) = 4

No. of Redundant Prime Implicants(RPI) = 1

No. of Selective Prime Implicants(SPI) = 0

2. $F = \sum m(0, 1, 5, 8, 12, 13)$, find number of implicant, PI, EPI, RPI and SPI.

Expression : $A'B'C' + C'DB + C'D'A$

	00	01	11	10
00	1	1	3	2
01	4	1	7	6
11	1	1	15	14
10	1	9	11	10

No. of Implicants = 6

No. of Prime Implicants(PI) = 6

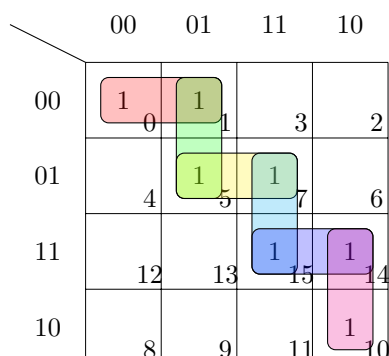
No. of Essential Prime Implicants(EPI) = 3

No. of Redundant Prime Implicants(RPI) = 3

No. of Selective Prime Implicants(SPI) = 6



3. $F = \sum m(0, 1, 5, 7, 15, 14, 10)$, find number of implicant, PI, EPI, RPI and SPI.



No. of Implicants = 7
 No. of Prime Implicants(PI) = 6
 No. of Essential Prime Implicants(EPI) = 2
 No. of Redundant Prime Implicants(RPI) = 2
 No. of Selective Prime Implicants(SPI) = 4

2.4 VARIABLE ENTRANT MAP

[-For Video](#)

- K-map is the best manual technique to solve Boolean equations, but it becomes difficult to manage when number of variables exceed 5 or 6. So, a technique called Variable Entrant Map (VEM) is used to increase the effective size of k-map.
- It allows a smaller map to handle large number of variables. This is done by writing output in terms of input.
- A VEM can be used to plot more than 'n' variables using an 'n' variable K-map.
- Consider a function $F(A,B,C) = (0,1,2,5)$

A	B	C	F
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0

If we define F in terms of 'C', then this function can be written as:



A	B	C	F	F in terms of c
0	0	0	1	1
0	0	1	1	
0	1	0	1	C'
0	1	1	0	
1	0	0	0	C
1	0	1	1	
1	1	0	0	0
1	1	1	0	

A	B	F
0	0	1
0	1	C'
1	0	C
1	1	0

The Variable entrant map for this is as follows :

		B	
		0	1
A	0	1 0	$\overline{C}$ 1
	1	C 2	0 3

Minimization procedure for VEM :

- Write all the variables(original and complimented forms are treated as two different variables) in the map as 0, leave 0's, minterms and don't cares as it is and obtain the SOP expression.
 (a) Select one variable and make all occurrences of that variable as 1, write minterms (1's) as don't cares, leave 0's and don't cares as it is. Now, obtain the SOP expression.
 (b) Multiply the obtained SOP expression with the concerned variable.
- Repeat step 2 for all the variables in the k-map. SOP of VEM is obtained by ORing all the obtained SOP expressions.

Example : Let's apply the above procedure on a sample VEM (X is used to represent don't care)

		AB			
		00	01	11	10
C	0	0 0	0 1	D 3	D 2
	1	1 4	1 5	$\overline{D}$ 7	$\overline{D}$ 6



- **Step 1 :** Write all the variables as 0 (D and D' are considered as two different variables), leave minterms, 0's and don't cares as it is and obtain the SOP expression.

		AB			
		00	01	11	10
C	0	0 0	0 1	0 3	0 2
	1	1 4	1 5	0 7	0 6

- **Step-2 :** (a) Replace all occurrences of D with 1, all occurrences of D' with 0 and all 1's with don't care. Leave 0's and don't cares as it is.

		AB			
		00	01	11	10
C	0	0 0	0 1	1 3	1 2
	1	X 4	X 5	0 7	0 6

- (b) Multiply the obtained SOP with the concerned variable.
SOP obtained: $AC'D$

- **Step-3 :** Repeat step 2 for D'
- (a) Replace all occurrences of D' with 1, all occurrences of D with 0 and all 1's with don't care. Leave 0's and don't cares as it is.

		AB			
		00	01	11	10
C	0	0 0	0 1	0 3	0 2
	1	X 4	X 5	1 7	1 6

- (b) Multiply the obtained SOP with the concerned variable.
SOP obtained: CD'



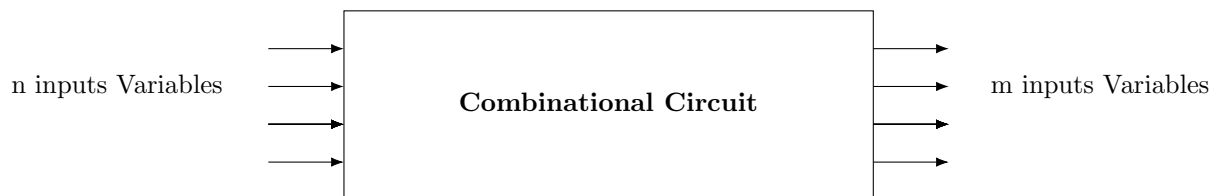
- **Step 4 :** SOP of VEM is obtained by ORing all the obtained SOP expressions. Therefore, the SOP expression for the given VEM is: $A'C + AC'D + CD'$

2.5 COMBINATIONAL AND SEQUENTIAL CIRCUITS [-For Video](#)

- There are two types of Digital circuits depending on their output and memory used:
 - (i) Combinational circuit
 - (ii) Sequential circuit

1. Combinational Circuits :

- A combinational circuit consists of input variables and output variables. Since these circuits are not dependent upon previous input to generate any output, so they are combinational logic circuits. A combinational circuit can have an n number of inputs and m number of outputs.
- In combinational circuits, the output at any time is a direct function of the applied external inputs. These circuits are developed using AND, OR, NOT, NAND and NOR logic gates. These logic gates are building blocks of combinational circuits.



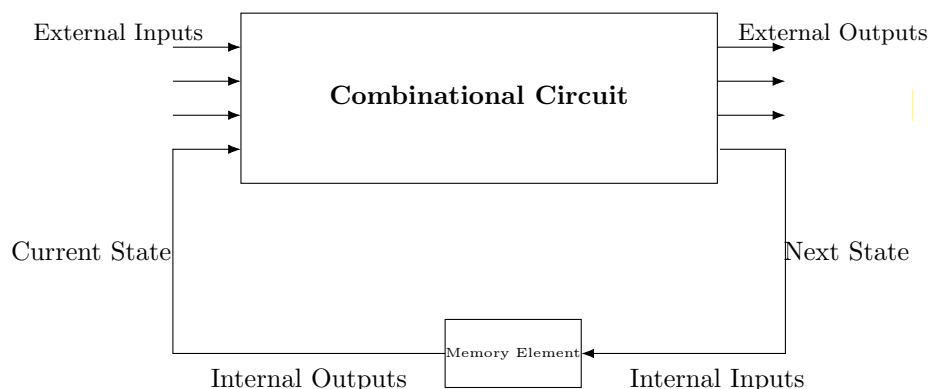
Block Diagram of Combinational Circuit

• Examples :

1. Arithmetic circuit
2. Converting one code to another code
3. Encoders & Decoders
4. Multiplexers and Demultiplexers

2. Sequential circuits

- A sequential circuit is specified by a time sequence of inputs, outputs and internal states. The output of a sequential circuit depends not only on the combination of present inputs but also on the previous outputs. Unlike combinational circuits, sequential circuits include memory elements with combinational circuits.



Block Diagram of Sequential Circuit

- **Examples :** 1. Shift registers 2. Flip-Flops 3. Counters



2.6 ARITHMETIC CIRCUITS

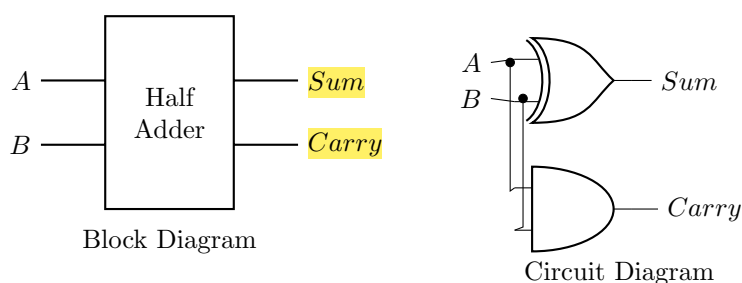
ADDERS :

- Addition is one of the most basic operations performed by different electronic devices like computers, calculators, etc.
- The electronic circuit that performs the addition of two or more numbers, more specifically binary numbers, is called as adder.

1. Half Adder :

[-For Video](#)

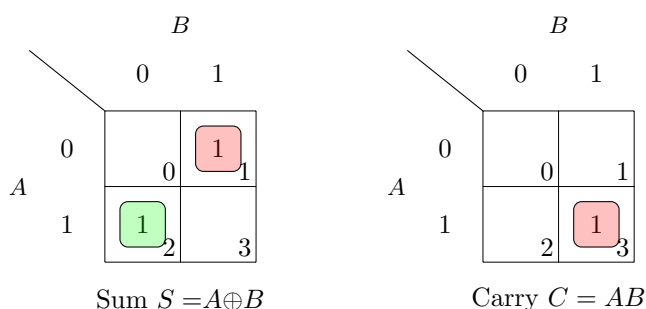
- A combinational logic circuit which is designed to add two binary digits is called as a half adder.
- The half adder provides the output along with a carry value. The half adder circuit is designed by connecting an EX-OR gate and one AND gate. It has two input terminals and two output terminals for sum and carry.
- The block diagram and circuit diagram of a half adder are shown below :



- From the logic circuit diagram of half adder, it is clear that A and B are the two input bits, S is the output i.e., sum and C is the output i.e., carry bit.
- In the case of a half adder, the output of the EX-OR gate is the sum of two bits and the output of the AND gate is the carry. Although, the carry obtained in one addition will not be forwarded in the next addition because of this it is known as half adder.
- Truth Table :

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

- K-Map :





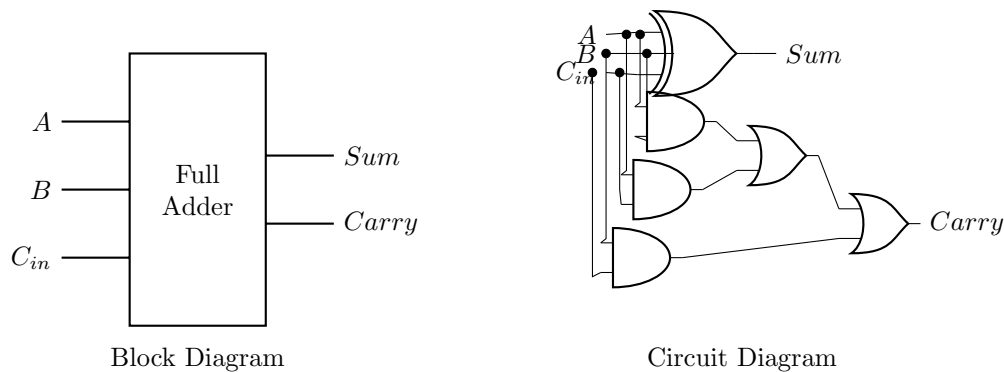
Characteristic Equations of Half-Adder :

- The characteristic equations of half adder, i.e., equations of sum (S) and carry (C) are obtained according to the rules of binary addition.
- The sum (S) of the half-adder is the XOR of A and B. Thus, Sum, $S = A \oplus B = AB' + A'B$
- The carry (C) of the half-adder is the AND of A and B. Therefore, Carry, $C = AB$.

2. Full Adder :

[-For Video](#)

- A Combinational circuit which is designed to add three binary digits and produces two outputs (sum and carry) is known as a full adder.
- Thus, a full adder circuit adds three binary digits, where two are the inputs and one is the carry forwarded from the previous addition.
- The block diagram and circuit diagram of the full adder are shown in Figure :

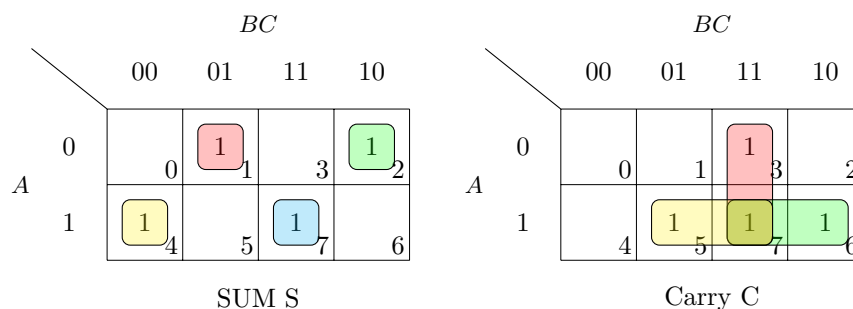


- Hence, the circuit of the full adder consists of one EX-OR gate, three AND gates and one OR gate
- Full adder takes three inputs namely A, B, and C_{in} . Where, A and B are the two binary digits, and C_{in} is the carry bit from the previous stage of binary addition.
- The sum output of the full adder is obtained by XOR-ing the bits A, B, and C_{in} . While the carry output bit (C_{out}) is obtained using AND and OR operations.
- **TruthTable :**

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



• **K-Map :**



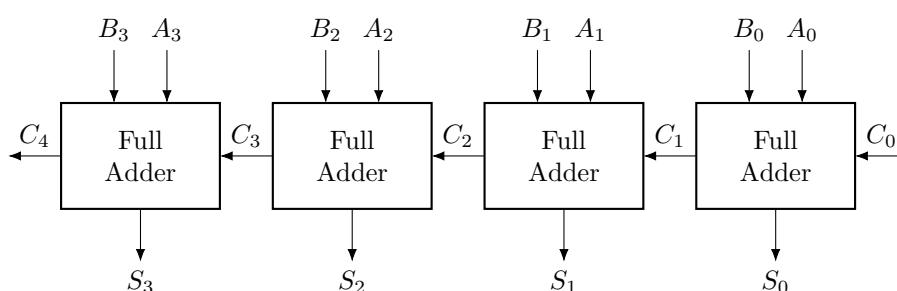
Characteristic Equations of Full-Adder :

- The characteristic equations of the full adder, i.e. equations of sum (S) and carry output (C_{out}) are obtained according to the rules of binary addition.
- The sum (S) of the full-adder is the XOR of A, B and C_{in}.
Thus, Sum, $S = A \oplus B \oplus C = A'B'C_{in} + A'BC'_{in} + AB'C_{in} + ABC_{in}$
- The carry (C) of the full-adder is $C = AB + AC_{in} + BC_{in}$.

3. Binary Adder or Ripple Carry Adder or Parallel Adder :

[-For Video](#)

- A Binary Adder is a digital circuit that implements the arithmetic sum of two binary numbers supported with any length is known as a binary adder.
- It is generated using full-adder circuits connected in sequence.
- The output carries from one full-adder linked to the input carry of the next full-adder.
- The following block diagram demonstrates the interconnections of four full-adder circuits to support a 4-bit binary adder.



- The input carry to the binary adder is C₀ and the output carry is C₄. The S outputs of the full-adders create the needed sum bits.
- An n-bit binary adder is needed n full-adders.
- The output carries from each full-adder is linked to the input carry of the next-high-order full-adder.



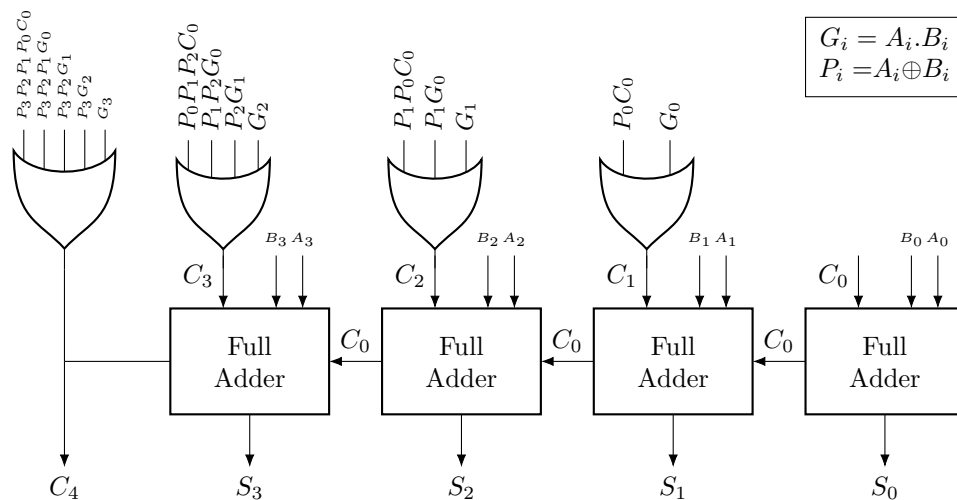
Draw Backs :

- In Ripple Carry Adder, Each full adder has to wait for its carry-in from its previous stage full adder.
- Thus, nth full adder has to wait until all (n-1) full adders have completed their operations.
- This causes a delay and makes ripple carry adder extremely slow.
- The situation becomes worst when the value of n becomes very large.
- To overcome this disadvantage, Carry Look Ahead Adder comes into play.

4. Carry Look Ahead Adder :

[-For Video](#)

- Carry Look Ahead Adder is an improved version of the ripple carry adder.
- It generates the carry-in of each full adder simultaneously without causing any delay.
- The time complexity of carry look ahead adder = $\theta(\log n)$.
- The working of carry look ahead adder is based on the principle that the carry-in of any stage full adder is independent of the carry bits generated during intermediate stages.
- **Advantages :**
 1. It generates the carry-in for each full adder simultaneously.
 2. It reduces the propagation delay.
- **Circuit diagram :**



- The carry-in of any stage full adder depends only on the following two parameters :
 1. Bits being added in the previous stages
 2. Carry-in provided in the beginning

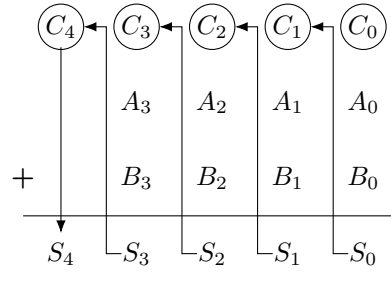
Disadvantages :

1. It involves complex hardware.
2. It is costlier since it involves complex hardware.
3. It gets more complicated as the number of bits increases.



• **Example :**

Consider two 4-bit binary numbers A3A2A1A0 and B3B2B1B0 are to be added.
Mathematically, the two numbers will be added as-



• From here, we have

$$C_1 = C_0 (A_0 \oplus B_0) + A_0 B_0$$

$$C_2 = C_1 (A_1 \oplus B_1) + A_1 B_1$$

$$C_3 = C_2 (A_2 \oplus B_2) + A_2 B_2$$

$$C_4 = C_3 (A_3 \oplus B_3) + A_3 B_3$$

• For simplicity, Let-

$$G_i = A_i B_i \text{ where } G \text{ is called carry generator}$$

$$P_i = A_i \oplus B_i \text{ where } P \text{ is called carry propagator}$$

• Then, re-writing the above equations, we have-

$$C_1 = C_0 P_0 + G_0 \dots\dots\dots (1)$$

$$C_2 = C_1 P_1 + G_1 \dots\dots\dots (2)$$

$$C_3 = C_2 P_2 + G_2 \dots\dots\dots (3)$$

$$C_4 = C_3 P_3 + G_3 \dots\dots\dots (4)$$

• Now, Clearly, C_1 , C_2 and C_3 are intermediate carry bits.

So, let's remove C_1 , C_2 and C_3 from RHS of every equation.

Substituting (1) in (2), we get C_2 in terms of C_0 .

Then, substituting (2) in (3), we get C_3 in terms of C_0 and so on.

• Finally, we have the following equations-

$$C_1 = C_0 P_0 + G_0$$

$$C_2 = C_0 P_0 P_1 + G_0 P_1 + G_1$$

$$C_3 = C_0 P_0 P_1 P_2 + G_0 P_1 P_2 + G_1 P_2 + G_2$$

$$C_4 = C_0 P_0 P_1 P_2 P_3 + G_0 P_1 P_2 P_3 + G_1 P_2 P_3 + G_2 P_3 + G_3$$

• These equations show that the carry-in of any stage full adder depends only on

1. Bits being added in the previous stages
2. Carry bit which was provided in the beginning

Note :

1. The following formula is used to calculate number of gates required for evaluating all carry bits :

For a n-bit carry look ahead adder to evaluate all the carry bits, it requires,

$$\text{Number of AND gates} = n(n+1)/2$$

$$\text{Number of OR gates} = n$$



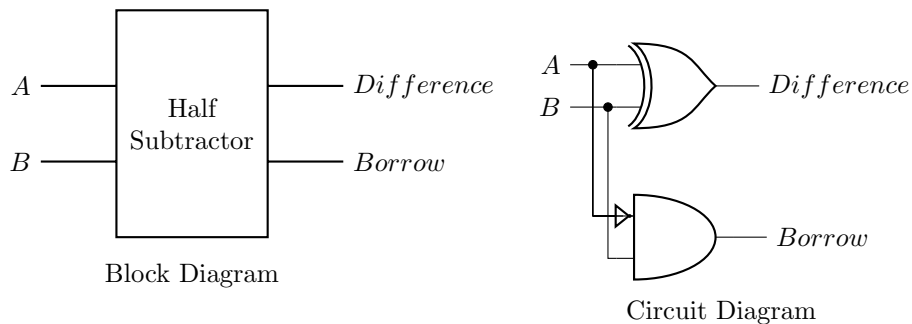
SUBTRACTORS :

[-For Video](#)

- A Subtractor is a combinational logic circuit that can perform the subtraction of two number (binary numbers) and produce the difference between them.

1. Half Subtractor :

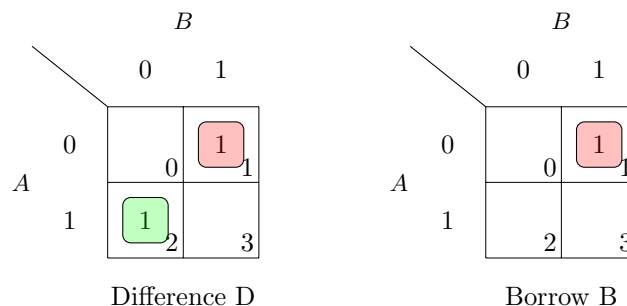
- It is a combinational logic circuit that have two inputs and two outputs (i.e. difference and borrow).
- The half subtractor produces the difference between the two binary bits at the input and also produces a borrow output.
- In the subtraction (A-B), A is called as Minuend bit and B is called as Subtrahend bit.
- The block diagram and logic circuit diagram of the half subtractor is shown in Figure:



- Hence, from the logic circuit diagram, it is clear that a half subtractor can be realized using an XOR gate together with a NOT gate and an AND gate.
- In the half subtractor as shown in figure-1, A and B are the inputs, d and b are the outputs. Where, d indicates the difference and b indicates the borrow output.
- The borrow output(b) is the signal that tells the next stage that a 1 has been borrowed.
- **Truth Table :**

A	B	difference(d)	Borrow(b)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

- **K-Map :**



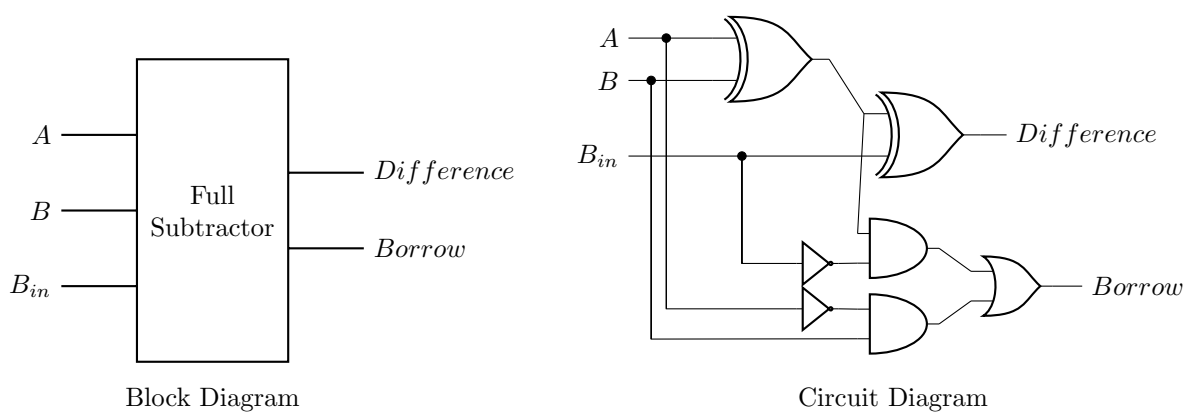


Characteristic Equations of Half-Subtractor :

- The characteristic equations of the half subtractor, i.e. equations of the difference bit (d) and the output borrow bit(b) are obtained by following the rules of binary subtraction.
- The difference bit(d) of the half subtractor is given by XORing the two inputs A and B. Therefore, Difference, $D = A \oplus B = A'B + AB'$
- The borrow (b) of the half subtractor is the AND of A' (complement of A) and B. Therefore, Borrow $b = A'B$.

2. Full Subtractor :

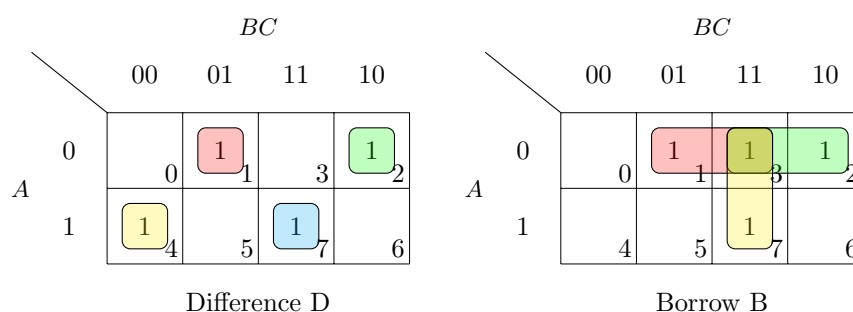
- A full-subtractor is a combinational circuit that has three inputs A, B, b_{in} and two outputs d and b. Where, A is the minuend, B is subtrahend, b_{in} is borrow produced by the previous stage, d is the difference output and b is the borrow output.



• Truth Table :

A	B	b_{in}	Difference	borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

• K-Map :



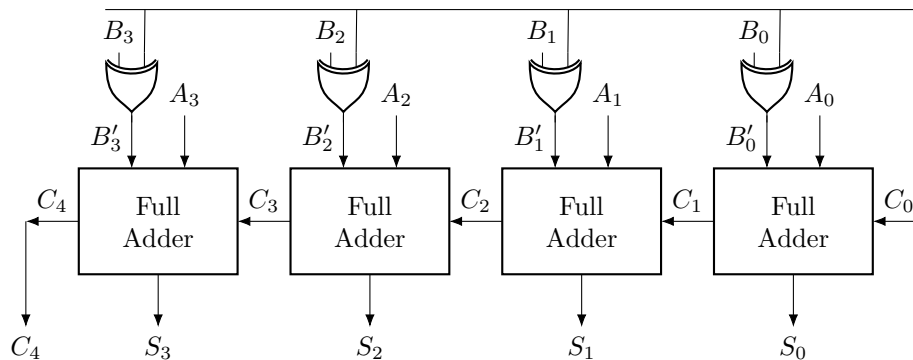


Characteristic Equations of Half-Subtractor :

- The characteristic equations of the full subtractor, i.e. equations of the difference (d) and borrow output (b) are obtained by following the rules of binary subtraction.
- The difference(d) of the full subtractor is the XOR of A, B, and b_{in} .
Thus, Difference, $S = A \oplus B \oplus b_{in} = A'B'b_{in} + A'Bb'_{in} + AB'b_{in} + ABb_{in}$
- The borrow(b) of the full subtractor is given by,
From the logic circuit diagram and k-map Borrow, $b = A'B + (A \oplus B)b_{in}$

4-Bit Binary Adder-Subtractor :

- A Binary Adder-Subtractor is capable of both the addition and subtraction of binary numbers in one circuit itself.
- Let's consider two 4-bit binary numbers A and B as inputs to the Digital Circuit for the operation with digits
A0 A1 A2 A3 for A
B0 B1 B2 B3 for B



- The circuit consists of 4 full adders since we are performing operations on 4-bit numbers.
- There is a control line K that holds a binary value of either 0 or 1 which determines that the operation is carried out is addition or subtraction.
- Assume that we have two 3-bit numbers, i.e., $X=100$ and $Y=011$, and feed them in Full-Adder as an input.

$$X_2 = 1 \quad X_1 = 0 \quad X_0 = 0$$

$$Y_2 = 0 \quad Y_1 = 1 \quad Y_0 = 1$$

For $K=0$:

$$Y_0 \oplus K = Y_0 \text{ and } C_{in} = K = 0$$

So, from first Full-Adder

$$S_0 = X_0 + Y_0 + C_{in}$$

$$S_0 = 0 + 1 + 0$$

$$S_0 = 1$$

$$C_0 = 0$$

Similarly,

$$S_1 = X_1 + Y_1 + C_0$$

$$S_1 = 0 + 1 + 0$$



$$S_1=1 \text{ and } C_1=0$$

Similarly,

$$S_2 = X_2 + Y_2 + C_1$$

$$S_2 = 1 + 0 + 0$$

$$S_2=1 \text{ and } C_2=0$$

Thus,

$$X = 100 = 4$$

$$Y = 011 = 3$$

$$\text{Sum} = 0111 = 7$$

For $K=1$

$$Y_0 \oplus K = Y_0' \text{ and } C_{in} = k = 1$$

So,

$$S_0 = X_0 + Y_0' + C_{in}$$

$$S_0 = 0 + 0 + 1$$

$$S_0=1 \text{ and } C_0=0$$

Similarly,

$$S_1 = X_1 + Y_1' + C_0$$

$$S_1 = 0 + 0 + 0$$

$$S_1=0 \text{ and } C_1=0$$

Similarly,

$$S_2 = X_2 + Y_2' + C_1$$

$$S_2 = 1 + 1 + 0$$

$$S_2=0 \text{ and } C_2=0$$

Thus,

$$X = 010 = 4$$

$$Y = 011 = 3$$

$$\text{Difference} = 001 = 1$$

MULTIPLIER

[-For Video](#)

- A binary multiplier is used to multiply two binary numbers.
- The binary multiplier is also called an add-shift adder.
- Consider two binary numbers num1 and num2
 num1=12 -> which equivalent to binary value as 1100 -> multiplier
 num2=13 -> which equivalent to binary value as 1101 -> multiplicand

$$\begin{array}{r}
 1100 \\
 1101 \\
 \hline
 1100 \\
 0000 \\
 1100 \\
 1100 \\
 \hline
 10011100
 \end{array}$$



REGISTER SHIFT METHOD :

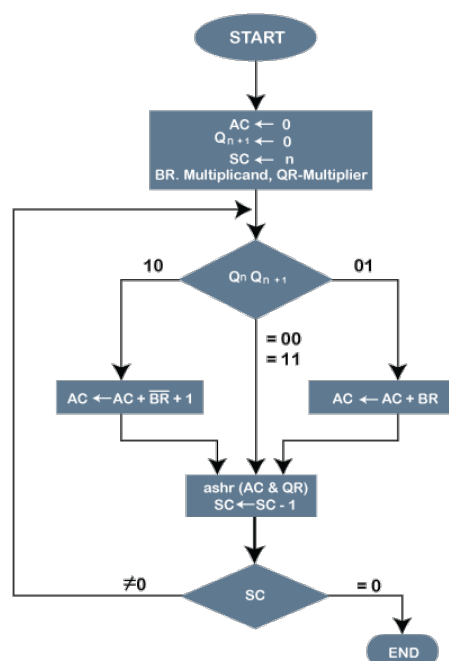
- Consider two binary numbers 1010 and 1011
Here 1010 is Multiplicand(M)
Here 1011 is Multiplier(Q)
Now, Consider a 8-bit register

0	0	0	0	0	0	0	0
1010							
10100000							
01010000							
1010							
11110000							
01111000							
0000							
01111000							
00111100							
1010							
11011100							
01101110							

2.7 BOOTH'S ALGORITHM

[-For Video](#)

- The booth algorithm is a multiplication algorithm that allows us to multiply the two signed binary integers in 2's complement, respectively.
- It is also used to speed up the performance of the multiplication process.
- It works on the string bits 0's in the multiplier that requires no additional bit only shift the right-most string bits and a string of 1's in a multiplier bit weight 2^k to weight 2^m that can be considered as $2^{k+1} - 2^m$.
- In the flowchart, initially, AC and $Q_n + 1$ bits are set to 0, and the SC is a sequence counter that represents the total bits set n, which is equal to the number of bits in the multiplier. There are BR that represent the multiplicand bits, and QR represents the multiplier bits.
- After that, we encountered two bits of the multiplier as Q_n and $Q_n + 1$, where Q_n represents the last bit of QR, and $Q_n + 1$ represents the incremented bit of Q_n by 1.
- Suppose two bits of the multiplier is equal to 10; it means that we have to subtract the multiplier from the partial product in the accumulator AC and then perform the arithmetic shift operation (ashr).



- If the two of the multipliers equal to 01, it means we need to perform the addition of the multiplicand to the partial product in accumulator AC and then perform the arithmetic shift operation (ashr), including $Q_n + 1$.
- The arithmetic shift operation is used in Booth's algorithm to shift AC and QR bits to the right by one and remains the sign bit in AC unchanged. And the sequence counter is continuously decremented till the computational loop is repeated, equal to the number of bits (n).

Example :

- A numerical example of booth's algorithm is shown below for $n = 4$. It shows the step by step multiplication of -5 and -7.

BR = -5 = 1011,

BR' = 0100, $\leftarrow$ 1's Complement (change the values 0 to 1 and 1 to 0)

BR'+1 = 0101 $\leftarrow$ 2's Complement (add 1 to the Binary value obtained after 1's complement)

QR = -7 = 1001 $\leftarrow$ 2's Complement of 0111 (7 = 0111 in Binary)

The explanation of first step is as follows: Q_{n+1}

AC = 0000, QR = 1001, $Q_{n+1} = 0$, SC = 4

$Q_n Q_{n+1} = 10$

So, we do AC + (BR)'+1, which gives AC = 0101

On right shifting AC and QR, we get

AC = 0010, QR = 1100 and $Q_{n+1} = 1$

Operation	AC	QR	Q_{n+1}	SC
	0000	1001	0	4
AC + BR' + 1	0101	1001	0	
ASHR	0010	1100	1	3
AC + BR	1101	1100	1	
ASHR	1110	1110	0	2
ASHR	1111	0111	0	1
AC + BR' + 1	0100	0111	0	
ASHR	0010	0011	1	0



Product is calculated as follows:

$$\text{Product} = AC \text{ QR}$$

$$\text{Product} = 0010 \ 0011 = 35$$

BCD Adder :

[-For Video](#)

- A BCD adder is a circuit that adds two BCD digits in parallel and makes a sum digit also in BCD.
- A BCD adder should contain the correction logic in its internal construction.
- To implement BCD Adder Circuit we require :
 1. 4-bit binary adder for initial addition
 2. Logic circuit to detect sum greater than 9 and
 3. One more 4-bit adder to add 0110 in the sum if sum is greater than 9 or carry is 1.
- The logic circuit to detect sum greater than 9 can be determined by simplifying the boolean expression of given BCD Adder Truth Table.

Inputs				Output
S_3	S_2	S_1	S_0	Y
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	1
1	0	1	1	1
1	1	0	0	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

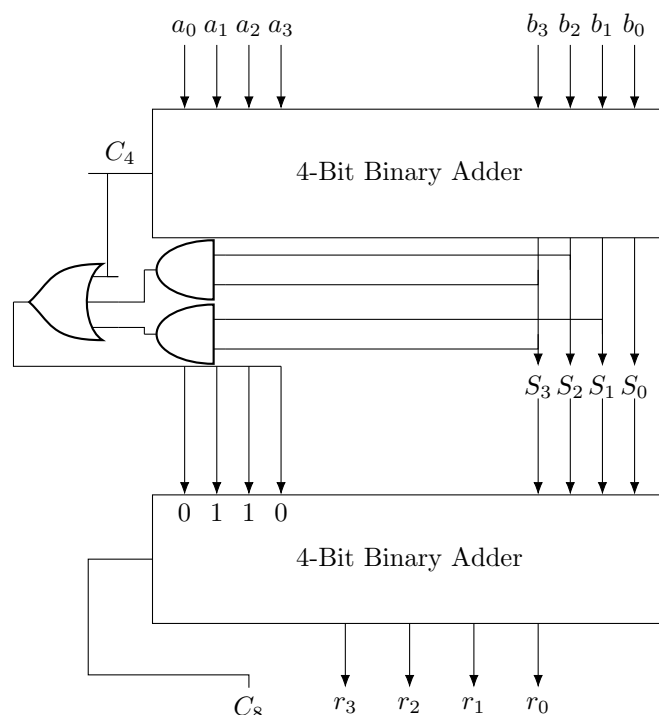
Table 3.10

		$S_1 S_0$			
		00	01	11	10
$S_3 S_2$	00	0	1	3	2
	01	4	5	7	6
	11	12	13	15	14
	10	8	9	11	10

K-Map for BCD Adder



- The block diagram of BCD adder is as follows :



- As shown in the Figure , the two BCD numbers, together with input carry, are first added in the top 4-bit binary adder to produce a binary sum.
- When the output carry is equal to zero (i.e. when sum ≤ 9 and Cout = 0) nothing (zero) is added to the binary sum. When it is equal to one (i.e. when sum > 9 or Cout = 1), binary 0110 is added to the binary sum through the bottom 4-bit binary adder.
- The output carry generated from the bottom binary adder can be ignored, since it supplies information already available at the output-carry terminal.

Converting BCD(8421) to Excess-3 :

[-For Video](#)

- A BCD digit can be converted to its corresponding Excess-3 code by simply adding 3 to it. Since we have only 10 digits(0 to 9) in decimal, we don't care about the rest & marked them with a cross(X).
- Let A,B,C,and D be the bits representing the binary numbers, where D is the LSB and A is the MSB.
- Let w, x, y, and z be the bits representing the excess-3 code of the binary numbers, where z is the LSB and w is the MSB.
- The truth table for the conversion is given below. The X's mark is don't care condition(term).



BCD(8421)				Excess-3			
A	B	C	D	w	x	y	z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	0	0	X	X	X	X
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X

- To find the corresponding digital circuit, we will use the K-Map technique for each of the Excess-3 code bits as output with all of the bits of the BCD number as input.

CD

	00	01	11	10
00	0 0	0 1	0 3	0 2
01	0 4	1 5	1 7	1 6
11	X 12	X 13	X 15	X 14
10	1 8	1 9	X 11	X 10

K-Map for *w*

CD

	00	01	11	10
00	0 0	1 1	1 3	1 2
01	1 4	0 5	0 7	0 6
11	X 12	X 13	X 15	X 14
10	0 8	1 9	X 11	X 10

K-Map for *x*

CD

	00	01	11	10
00	1 0	0 1	1 3	0 2
01	1 4	0 5	1 7	0 6
11	X 12	X 13	X 15	X 14
10	1 8	0 9	X 11	X 10

K-Map for *y*

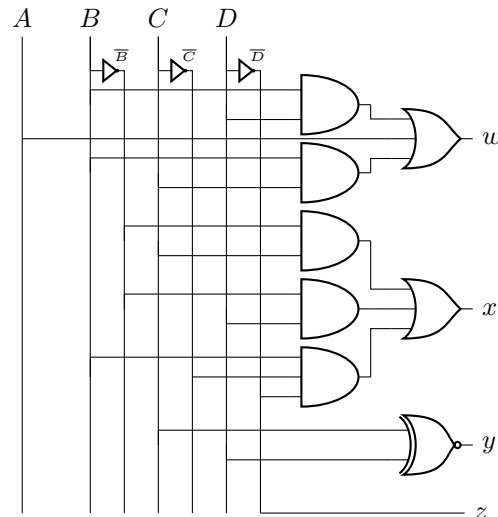
CD

	00	01	11	10
00	1 0	0 1	0 3	1 2
01	1 4	0 5	0 7	1 6
11	X 12	X 13	X 15	X 14
10	1 8	0 9	X 11	X 10

K-Map for *z*



- Corresponding minimized Boolean expressions for Excess-3 code bits are
 $w = A + BC + BD$
 $x = B'C + B'D + BC'D'$
 $y = CD + C'D'$
 $z = D'$
- The corresponding digital circuit is as follows :



Converting Excess-3 to BCD(8421) :

[-For Video](#)

- Let A, B, C, and D be the bits representing the binary numbers, where D is the LSB and A is the MSB.
- Let w, x, y, and z be the bits representing the excess code of the binary numbers, where z is the LSB and w is the MSB.
- The truth table for the conversion is given below. The X's mark is don't care condition.

Excess-3				BCD			
w	x	y	z	A	B	C	D
0	0	0	0	X	X	X	X
0	0	0	1	X	X	X	X
0	0	1	0	X	X	X	X
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	1
0	1	0	1	0	0	1	0
0	1	1	0	0	0	1	1
0	1	1	1	0	1	0	0
1	0	0	0	0	1	0	1
1	0	0	1	0	1	1	0
1	0	1	0	0	1	1	1
1	0	1	1	1	0	0	0
1	1	0	0	1	0	0	1
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X



- K-Map for D, C, B, A are as follows respectively :

		<i>yz</i>			
		00	01	11	10
<i>wx</i>	00	X ₀	X ₁	0 ₃	X ₂
	01	0 ₄	0 ₅	0 ₇	0 ₆
	11	1 ₁₂	X ₁₃	X ₁₅	X ₁₄
	10	0 ₈	0 ₉	1 ₁₁	0 ₁₀

K-Map for A

		<i>yz</i>			
		00	01	11	10
<i>wx</i>	00	X ₀	X ₁	0 ₃	X ₂
	01	0 ₄	0 ₅	1 ₇	0 ₆
	11	0 ₁₂	X ₁₃	X ₁₅	X ₁₄
	10	1 ₈	1 ₉	0 ₁₁	1 ₁₀

K-Map for B

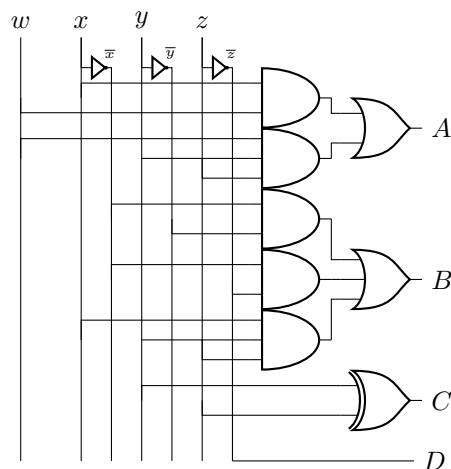
		<i>yz</i>			
		00	01	11	10
<i>wx</i>	00	X ₀	X ₁	0 ₃	X ₂
	01	0 ₄	1 ₅	0 ₇	1 ₆
	11	0 ₁₂	X ₁₃	X ₁₅	X ₁₄
	10	0 ₈	1 ₉	0 ₁₁	1 ₁₀

K-Map for C

		<i>yz</i>			
		00	01	11	10
<i>wx</i>	00	X ₀	X ₁	0 ₃	X ₂
	01	1 ₄	0 ₅	0 ₇	1 ₆
	11	1 ₁₂	X ₁₃	X ₁₅	X ₁₄
	10	1 ₈	0 ₉	0 ₁₁	1 ₁₀

K-Map for D

- Corresponding minimized boolean expressions for Excess-3 code bits are
 $A = wx + wyz$ $B = x'y' + x'z' + xyz$ $C = y'z + yz'$ $D = z'$
- The corresponding digital circuit –

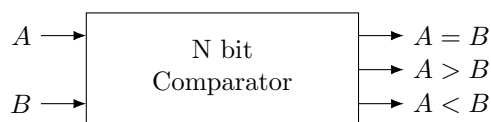




2.8 COMPARATOR

[-For Video](#)

- Comparator is a combinational circuit that compares two digital or binary numbers in order to find out whether one binary number is equal, less than, or greater than the other binary number.
- We logically design a circuit for which we will have two inputs one for A and the other for B and have three output terminals, one for $A > B$ condition, one for $A = B$ condition, and one for $A < B$ condition.



Block Diagram of Comparator Circuit

- There are different ways to implement a magnitude comparator, such as using a combination of XOR, AND and OR gates, or by using a cascaded arrangement of full adders.

1-Bit Comparator :

- A comparator used to compare two bits is called a single-bit comparator.
- It consists of two inputs each for two single-bit numbers and three outputs to generate less than, equal to, and greater than between two binary numbers.
- The truth table for a 1-bit comparator is given below.

A	B	$A < B$	$A = B$	$A > B$
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

- From the above truth table logical expressions for each output can be expressed as follows.

$$A > B : AB'$$

$$A < B : A'B$$

$$A = B : A'B' + AB$$

- From the above expressions, we can derive the following formula.

$$\Rightarrow A < B + A > B = A'B + AB'$$

Taking complements on both sides

$$\Rightarrow (A < B + A > B)' = (A'B + AB')'$$

$$\Rightarrow (A < B + A > B)' = (A'B)'(AB')'$$

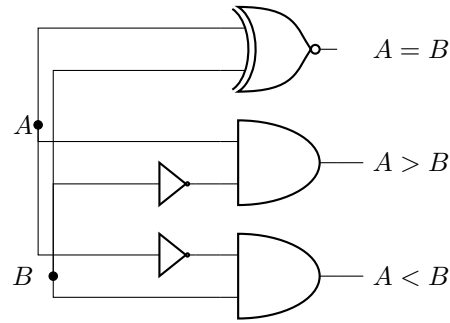
$$\Rightarrow (A < B + A > B)' = (A+B')(A'+B)$$

$$\Rightarrow (A < B + A > B)' = (AA' + AB + A'B' + BB')$$

$$\Rightarrow (A < B + A > B)' = (AB + A'B')$$

$$\Rightarrow (A < B + A > B)' = (A=B)$$

- By using these Boolean expressions, we can implement a logic circuit for this comparator as given below.



2-Bit Comparator :

- A comparator used to compare two binary numbers each of two bits is called a 2-bit Magnitude comparator.
- It consists of four inputs and three outputs to generate less than, equal to, and greater than between two binary numbers.
- The truth table for a 2-bit comparator is given below.

A ₁	A ₀	B ₁	B ₀	A<B	A=B	A>B
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

- From the K-maps logical expressions for each output can be expressed as follows.

$$A > B : A_1 B_1' + A_0 B_1' B_0' + A_1 A_0 B_0'$$

$$A = B : A_1' A_0' B_1' B_0' + A_1' A_0 B_1' B_0 + A_1 A_0 B_1 B_0 + A_1 A_0' B_1 B_0'$$

$$: A_1' B_1' (A_0' B_0' + A_0 B_0) + A_1 B_1 (A_0 B_0 + A_0' B_0')$$

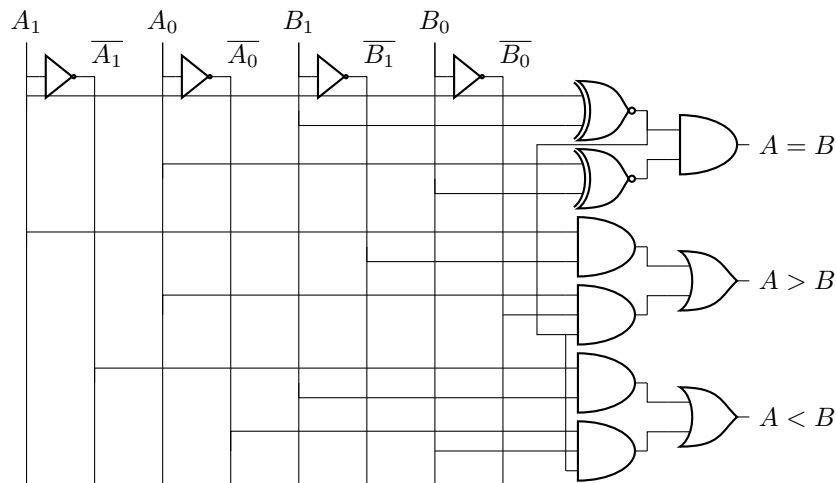
$$: (A_0 B_0 + A_0' B_0') (A_1 B_1 + A_1' B_1')$$

$$: (A_0 \oplus B_0) (A_1 \oplus B_1)$$

$$A < B : A_1' B_1 + A_0' B_1 B_0 + A_1' A_0' B_0$$



- By using these Boolean expressions, we can implement a logic circuit for this comparator as given below.

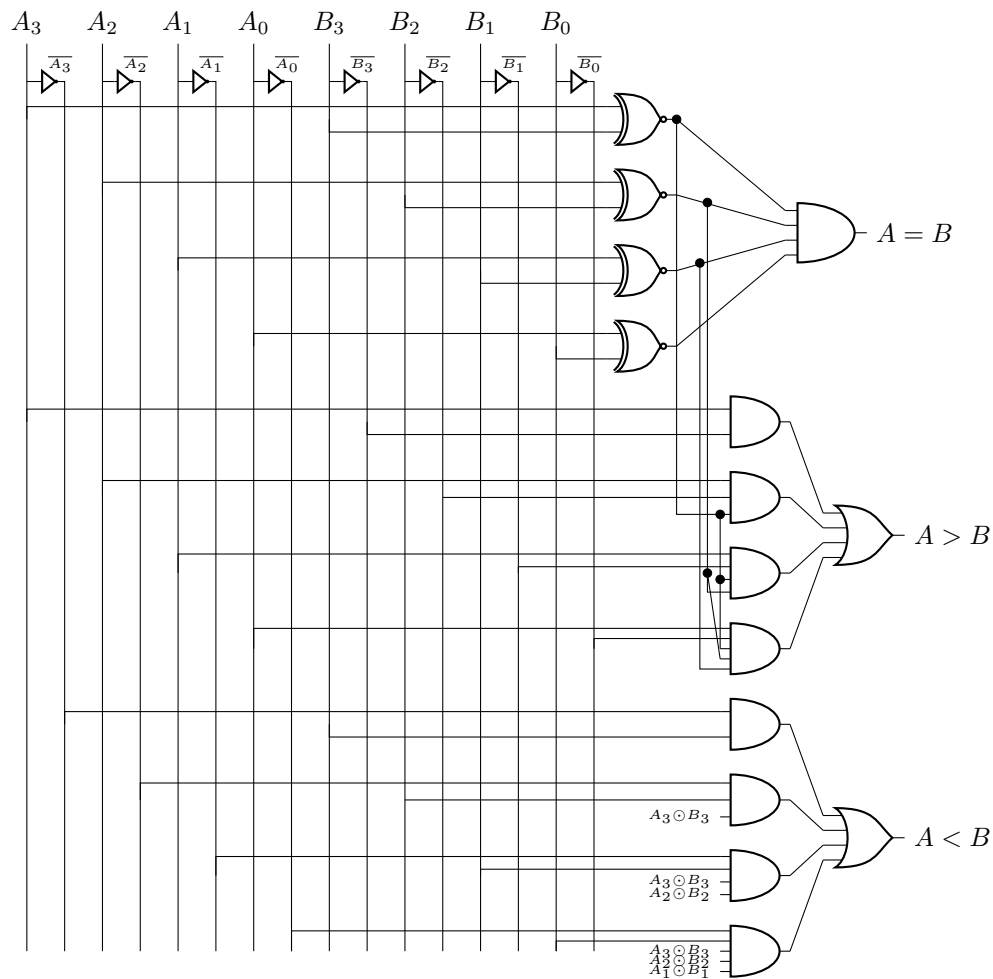


4-Bit Comparator :

- A comparator used to compare two binary numbers each of four bits is called a 4-bit magnitude comparator.
- It consists of eight inputs each for two four-bit numbers and three outputs to generate less than, equal to, and greater than between two binary numbers.
- In a 4-bit comparator, the condition of $A > B$ can be possible in the following four cases.
 1. If $A_3 = 1$ and $B_3 = 0$
 2. If $A_3 = B_3$ and $A_2 = 1$ and $B_2 = 0$
 3. If $A_3 = B_3$, $A_2 = B_2$ and $A_1 = 1$ and $B_1 = 0$
 4. If $A_3 = B_3$, $A_2 = B_2$, $A_1 = B_1$ and $A_0 = 1$ and $B_0 = 0$
- Similarly, the condition for $A < B$ can be possible in the following four cases.
 1. If $A_3 = 0$ and $B_3 = 1$
 2. If $A_3 = B_3$ and $A_2 = 0$ and $B_2 = 1$
 3. If $A_3 = B_3$, $A_2 = B_2$ and $A_1 = 0$ and $B_1 = 1$
 4. If $A_3 = B_3$, $A_2 = B_2$, $A_1 = B_1$ and $A_0 = 0$ and $B_0 = 1$
- The condition of $A = B$ is possible only when all the individual bits of one number exactly coincide with the corresponding bits of another number.

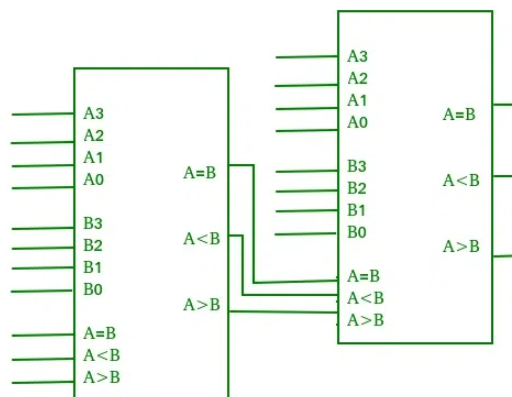


- The circuit diagram is as follows :



Cascading Comparator :

- Comparator performing the comparison operation to more than four bits by cascading two or more 4-bit comparators is called a cascading comparator.
- When two comparators are to be cascaded, the outputs of the lower-order comparator are connected to the corresponding inputs of the higher-order comparator.



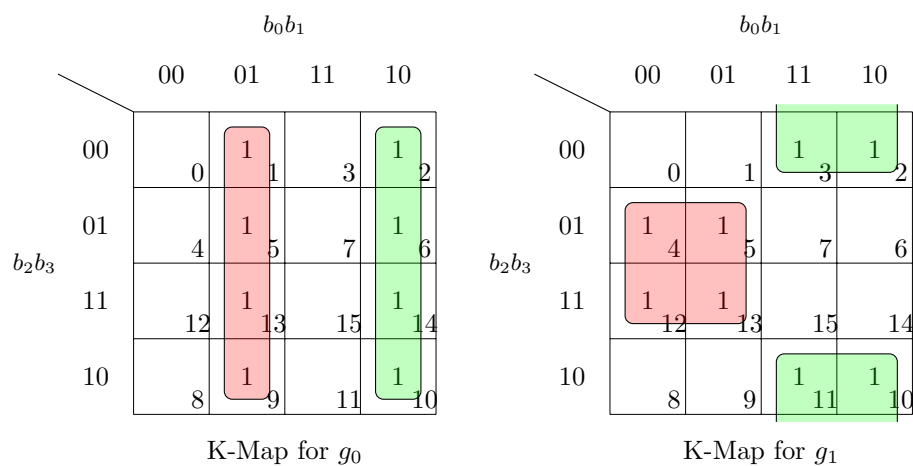


Conversion of Binary to Gray code :

- Let b_0, b_1, b_2, b_3 be the bits representing the binary numbers, where b_0 is the LSB and b_3 is the MSB and Let g_0, g_1, g_2, g_3 be the bits representing the gray code of the binary numbers, where g_0 is the LSB and g_3 is the MSB.
- The truth table for the conversion is as follows :

Binary				Graycode			
b_3	b_2	b_1	b_0	g_3	g_2	g_1	g_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

- To find the corresponding digital circuit, we will use the K-Map technique for each of the gray code bits as output with all of the binary bits as input.
- K-map for g_0, g_1, g_2 and g_3 are as follows respectively :





		b_0b_1			
		00	01	11	10
b_2b_3	00	0	1	3	2
	01	1	1	1	1
	11				
	10	1	1	1	1
		4	5	7	6
		12	13	15	14
		8	9	11	10

K-Map for g_2

		b_0b_1			
		00	01	11	10
b_2b_3	00	0	1	3	2
	01				
	11	1	1	1	1
	10	1	1	1	1
		4	5	7	6
		12	13	15	14
		8	9	11	10

K-Map for g_3

- Corresponding minimized boolean expressions for gray code bits are as follows :

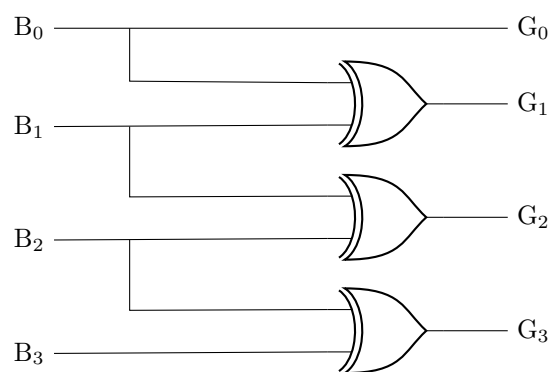
$$g_0 = b_0b_1' + b_0'b_1 = b_0 \oplus b_1$$

$$g_1 = b_2b_1' + b_1'b_2 = b_1 \oplus b_2$$

$$g_2 = b_2b_3' + b_3'b_2 = b_3 \oplus b_2$$

$$g_3 = b_3$$

- The corresponding digital circuit is



Conversion of Gray code to Binary code :

- Let g_0, g_1, g_2, g_3 be the bits representing the gray code of the binary numbers, where g_0 is the LSB and g_3 is the MSB and Let b_0, b_1, b_2, b_3 be the bits representing the binary numbers, where b_0 is the LSB and b_3 is the MSB.

- The truth table for the conversion is as follows :



Gray Code				Binary			
g_3	g_2	g_1	g_0	b_3	b_2	b_1	b_0
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	1	0	0	1	0
0	0	1	0	0	0	1	1
0	1	1	0	0	1	0	0
0	1	1	1	0	1	0	1
0	1	0	1	0	1	1	0
0	1	0	0	0	1	1	1
1	1	0	0	1	0	0	0
1	1	0	1	1	0	0	1
1	1	1	1	1	0	1	0
1	1	1	0	1	0	1	1
1	0	1	0	1	1	0	0
1	0	1	1	1	1	0	1
1	0	0	1	1	1	1	0
1	0	0	0	1	1	1	1

- To find the corresponding digital circuit, we will use the K-Map technique for each of the binary code bits as output with all of the gray bits as input.
- K-map for b_0 , b_1 , b_2 and b_3 are as follows respectively :

$g_0 g_1$

	00	01	11	10
00	0	1	3	2
01	4		7	6
11	12	13	15	14
10	8	9	11	10

K-Map for b_0

$g_0 g_1$

	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

K-Map for b_1

$g_0 g_1$

	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

K-Map for b_2

$g_0 g_1$

	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

K-Map for b_3



- Corresponding minimized boolean expressions for gray code bits are as follows :

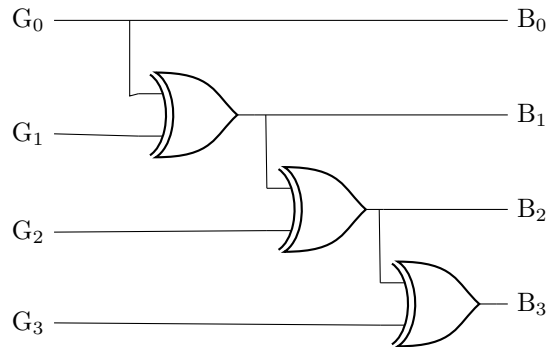
$$b_0 = g_0 \oplus g_1 \oplus g_2 \oplus g_3$$

$$b_1 = g_0 \oplus g_1 \oplus g_2$$

$$b_2 = g_0 \oplus g_1$$

$$b_3 = g_3$$

- The corresponding digital circuit is

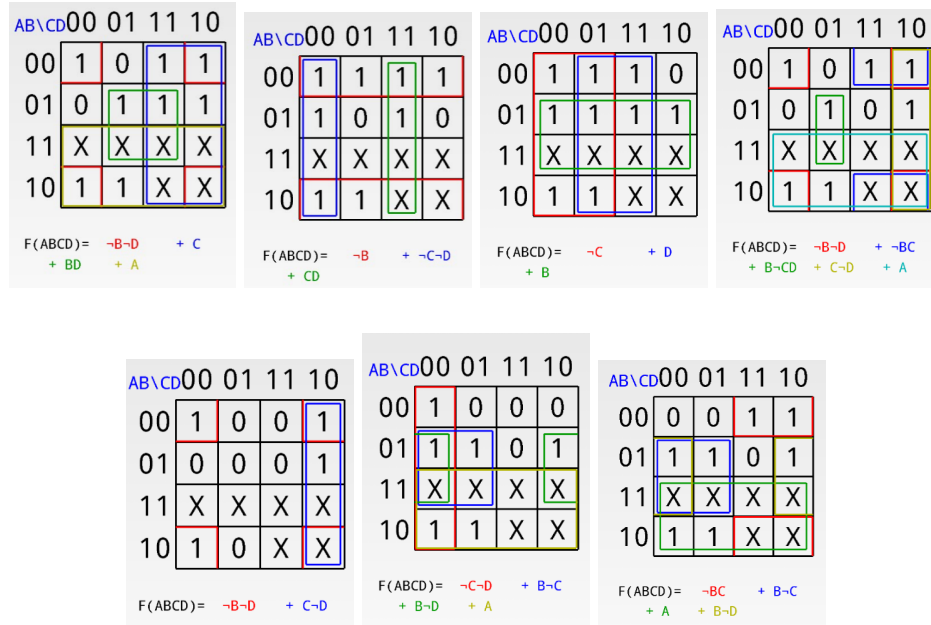


Conversion of BCD to 7 Segment code :

- In Binary Coded Decimal (BCD) encoding scheme each of the decimal numbers(0-9) is represented by its equivalent binary pattern(which is generally of 4-bits).
- Whereas, Seven segment display is an electronic device which consists of seven Light Emitting Diodes (LED's) arranged in a some definite pattern (common cathode or common anode type), which is used to display Hexadecimal numerals(in this case decimal numbers,as input is BCD i.e., 0-9).
- The truth table for the conversion is as follows :

BCD				7-Segment						
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	0	1	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1

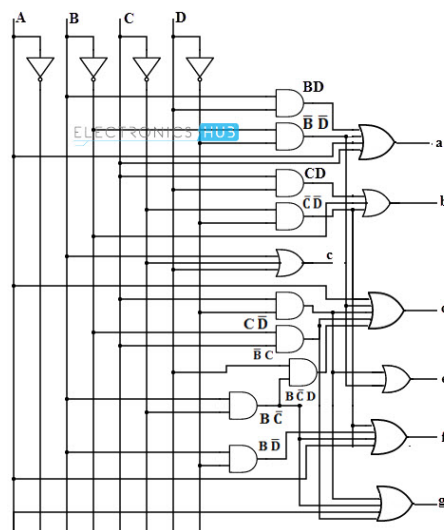
- To find the corresponding digital circuit, we will use the K-Map technique for each of the 7-segment's bits as output with all of the bcd bits as input.
- K-map for a, b, c, d, e, f and g are as follows respectively :



- Corresponding minimized boolean expressions for 7-segment code bits are as follows :

$$\begin{aligned}
 a &= A + C + BD + \bar{B}\bar{D} \\
 b &= \bar{B} + \bar{C}\bar{D} + CD \\
 c &= B + \bar{C} + D \\
 d &= \bar{B}\bar{D} + C\bar{D} + B\bar{C}D + \bar{B}C + A \\
 e &= \bar{B}\bar{D} + C\bar{D} \\
 f &= A + \bar{C}\bar{D} + B\bar{C} + B\bar{D} \\
 g &= A + B\bar{C} + \bar{B}C + C\bar{D}
 \end{aligned}$$

- The corresponding digital circuit is

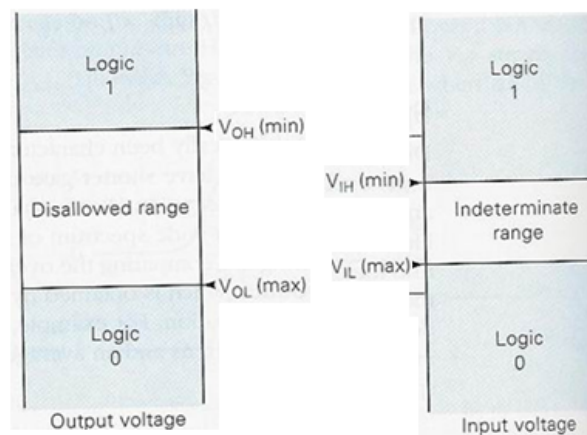




2.9 NOISE MARGINS

[-For Video](#)

- The circuit's ability to tolerate noise signals is referred to as the noise immunity, a quantitative measure of which is called noise margin.
- The noise margins defined above are referred to as dc noise margins. Strictly speaking, the noise is generally thought of as an a.c. signal with amplitude and pulse width.
- V_{NH} = HIGH-state noise margin
 V_{NL} = LOW-state noise margin
 V_{IL} = LOW-state input voltage
 V_{IH} = HIGH-state input voltage
 V_{OL} = LOW-state output voltage
 V_{OH} = HIGH-state output voltage
Where,
 $V_{NH} = V_{OH} - V_{IH}$
 $V_{NL} = V_{IL} - V_{OL}$
- See below Figure to calculate noise margin



- Manufacturers specify voltage limits to represent the logical 0 or 1. These limits are not the same at the input and output sides.
- For example, a particular gate A may output a voltage of 4.8V when it is supposed to output a HIGH but, at its input side, it can take a voltage of 3V as HIGH.
- In this way, if any noise should corrupt the signal, there is some margin for error.



UNIT-3

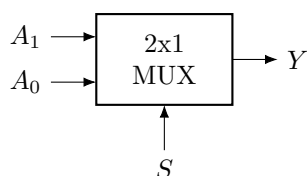
3.1 MULTIPLEXER

[-For Video](#)

- Multiplexer is a combinational circuit that has maximum of 2^n data inputs, 'n' selection lines and single output line.
- One of these data inputs will be connected to the output based on the values of selection lines.
- Since there are 'n' selection lines, there will be 2^n possible combinations of zeros and ones. So, each combination will select only one data input.
- Multiplexer is also called as MUX.

2x1 Multiplexer :

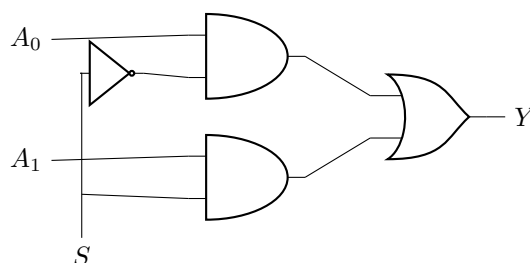
- In 2×1 multiplexer, there are only two inputs, i.e., A_0 and A_1 , 1 selection line, i.e., S and single outputs, i.e., Y.
- On the basis of the combination of inputs which are present at the selection line S, one of these 2 inputs will be connected to the output.
- The block diagram of 2×1 multiplexer is given below.



- Truth table of 2×1 Multiplexer is shown below.

Input	Output
S	Y
0	I_0
1	I_1

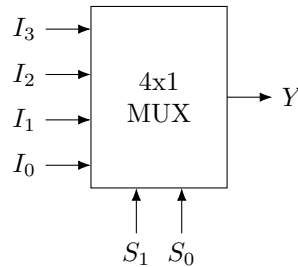
- From Truth table, The logical expression of the term Y is as follows: $Y = \bar{S}A_0 + SA_1$
- The circuit diagram of 2×1 multiplexer is shown in the following figure.





4x1 Multiplexer :

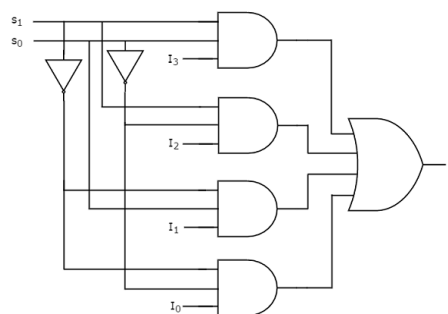
- 4x1 Multiplexer has four data inputs I_3, I_2, I_1 & I_0 , two selection lines S_1 & S_0 and one output Y .
- The block diagram of 4x1 Multiplexer is shown in the following figure.



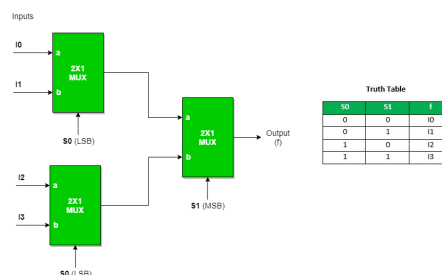
- One of these 4 inputs will be connected to the output based on the combination of inputs present at these two selection lines.
- Truth table of 4x1 Multiplexer is shown below.

Select lines		Output
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

- From Truth table, we can directly write the Boolean function for output, Y as $Y = \bar{S}_1 \bar{S}_0 I_0 + \bar{S}_1 S_0 I_1 + S_1 \bar{S}_0 I_2 + S_1 S_0 I_3$
- The circuit diagram of 4x1 multiplexer is shown in the following figure.



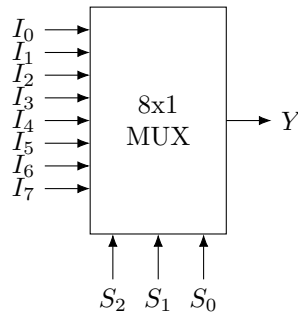
- Three 2 x 1 MUX are required to implement 4 x 1 MUX.





8x1 Multiplexer :

- In the 8 to 1 multiplexer, there are total eight inputs, i.e., $I_0, I_1, I_2, I_3, I_4, I_5, I_6,$ and I_7 , 3 selection lines, i.e., S_0, S_1 and S_2 and single output, i.e., Y .
- On the basis of the combination of inputs that are present at the selection lines S_0, S_1 and S_2 one of these 8 inputs are connected to the output.
- The block diagram of the 8×1 multiplexer is given below.

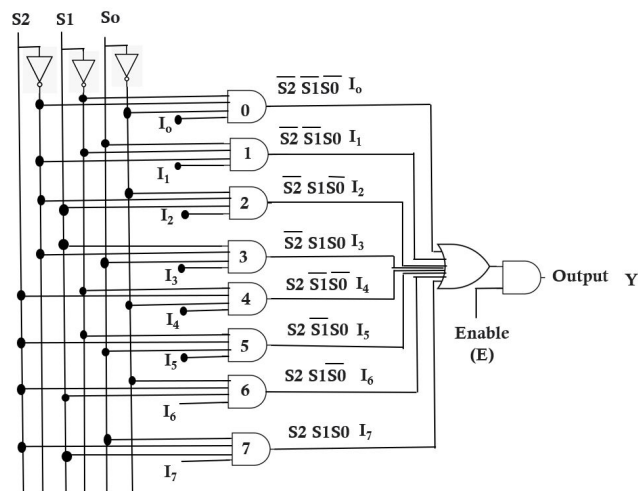


- Truth table of 8x1 Multiplexer is shown below.

Inputs			Output
S_2	S_1	S_0	Y
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

- The logical expression of the term Y is as follows :

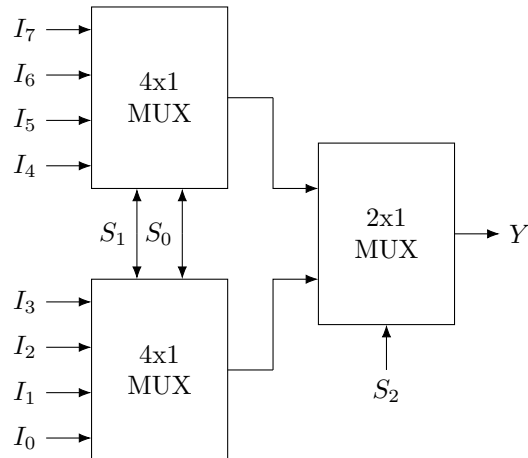
$$Y = \bar{S}_0 \bar{S}_1 \bar{S}_2 I_0 + S_0 \bar{S}_1 \bar{S}_2 I_1 + \bar{S}_0 S_1 \bar{S}_2 I_2 + S_0 S_1 \bar{S}_2 I_3 + \bar{S}_0 \bar{S}_1 S_2 I_4 + S_0 \bar{S}_1 S_2 I_5 + \bar{S}_0 S_1 S_2 I_6 + S_0 S_1 S_2 I_7$$
- Logical circuit of the above expression is as follows :





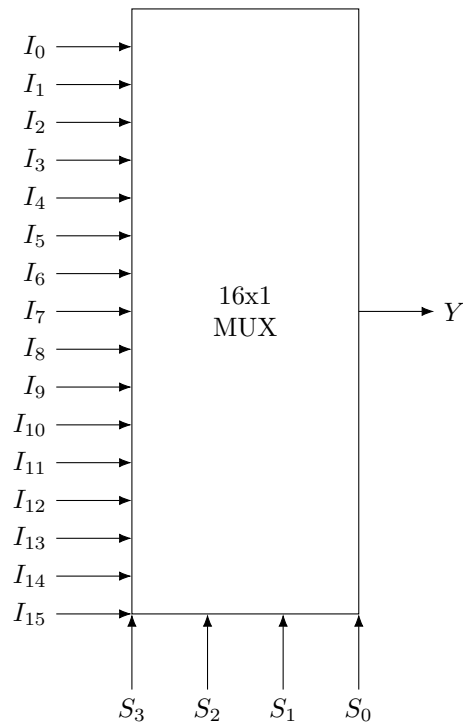
8 × 1 multiplexer using 4 × 1 and 2 × 1 multiplexer :

- For getting 8 data inputs, we need two 4 × 1 multiplexers. The 4 × 1 multiplexer produces one output. So, in order to get the final output, we need a 2 × 1 multiplexer.
- The block diagram of 8 × 1 multiplexer using 4 × 1 and 2 × 1 multiplexer is given below.



16x1 Multiplexer :

- In 16 x 1 multiplexer, Input lines = 16 and Selection lines = 4.
- Input lines will be I_0 to I_{15} and Selection lines will be S_0 to S_3
- The block diagram 16 × 1 multiplexer is as follows.

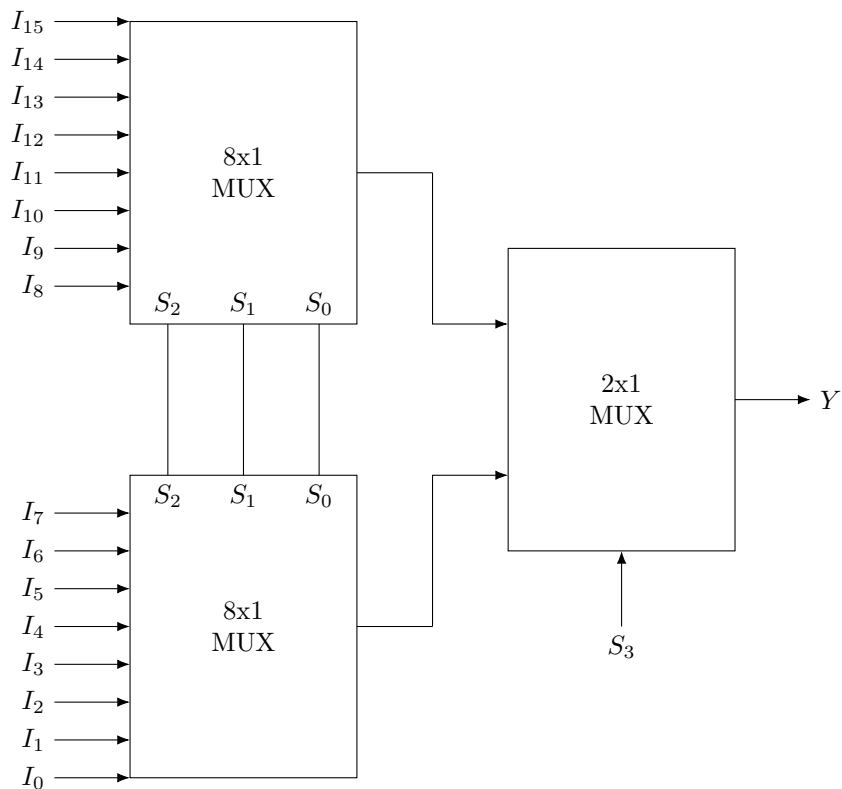




- Truth table of 16x1 Multiplexer is shown below.

Inputs				Output
S_2	S_2	S_1	S_0	Y
0	0	0	0	I_0
0	0	0	1	I_1
0	0	1	0	I_2
0	0	1	1	I_3
0	1	0	0	I_4
0	1	0	1	I_5
0	1	1	0	I_6
0	1	1	1	I_7
1	0	0	0	I_8
1	0	0	1	I_9
1	0	1	0	I_{10}
1	0	1	1	I_{11}
1	1	0	0	I_{12}
1	1	0	1	I_{13}
1	1	1	0	I_{14}
1	1	1	1	I_{15}

- We can implement 16x1 Multiplexer using 8x1 Multiplexers and 2x1 Multiplexer.
- We require two 8x1 Multiplexers in first stage in order to get the 16 data inputs. Since, each 8x1 Multiplexer produces one output, we require a 2x1 Multiplexer in second stage by considering the outputs of first stage as inputs and to produce the final output.

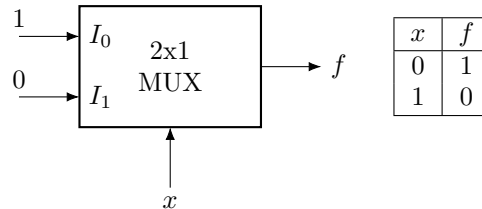




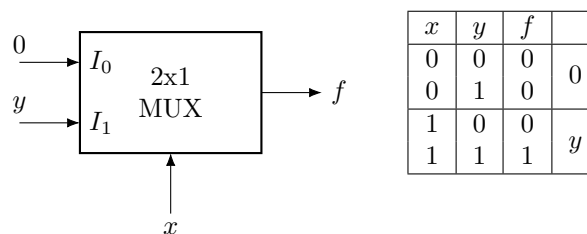
Functionally Completeness :

- Multiplexer can **act as universal combinational circuit**. All the standard logic gates can be implemented with multiplexers.

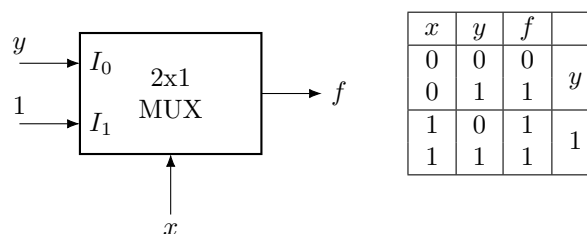
1. Implementation of NOT gate using 2x1 Mux :



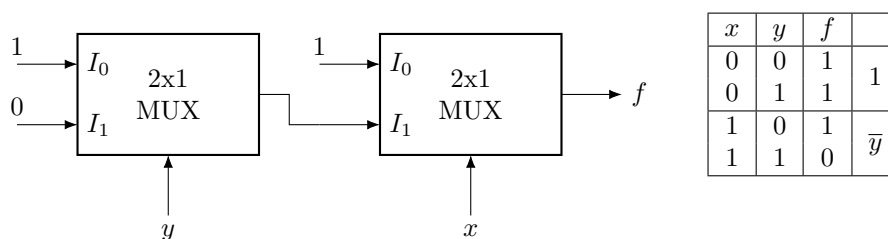
2. Implementation of AND gate using 2x1 Mux :



3. Implementation of OR gate using 2x1 Mux :

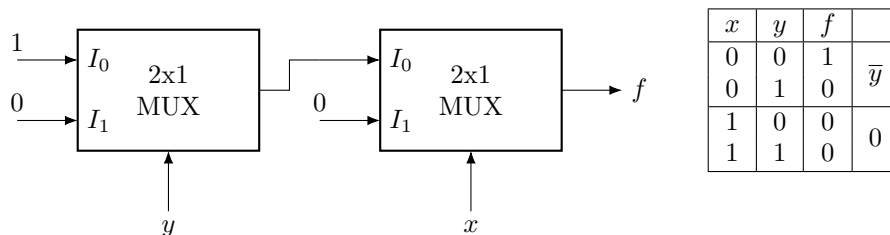


4. Implementation of NAND gate using 2x1 Mux :

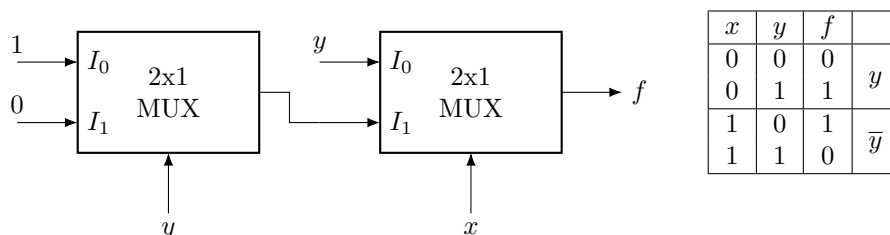




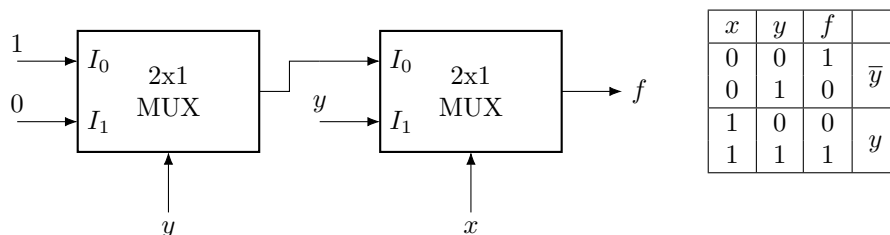
5. Implementation of NOR gate using 2x1 Mux :



6. Implementation of EX-OR gate using 2x1 Mux :



7. Implementation of EX-NOR gate using 2x1 Mux :



HALF ADDER :

[-For Video](#)

Using 4 x 1 Multiplexer :

- The Boolean functions of sum and carry outputs are

$$S(X, Y) = \sum m(1, 2)$$

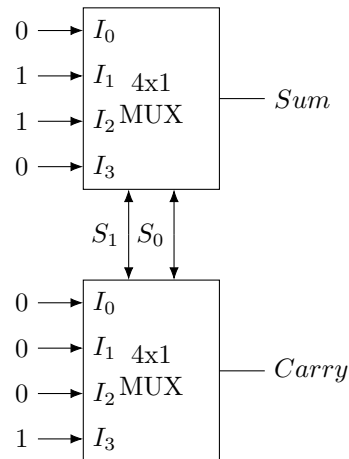
$$C(X, Y) = \sum m(3)$$
- Here, X and Y represent the two inputs and S and C represents the two outputs of the Half Adder.
- Truth table for sum & carry as follows :

S ₁	S ₀	Z
0	0	0
0	1	1
1	0	1
1	1	0

S ₁	S ₀	Z
0	0	0
0	1	0
1	0	0
1	1	1



- Logic diagram of a Half Adder :



- To implement the logic diagram of a Half Adder we need two multiplexers the first multiplexer produces sum output and the second MUX generates the carry output of the Half Adder. X and Y are the two selection inputs to both multiplexers to select a particular input.

Using 2 x 1 Multiplexer :

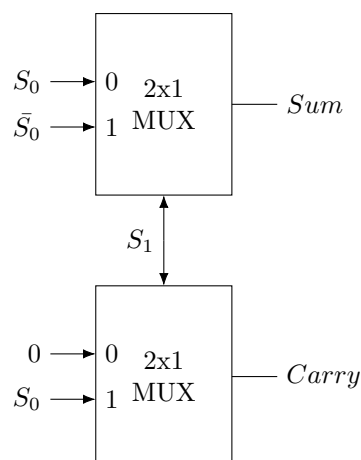
- Here we will implement the logic diagram of a Half Adder using two 2 x 1 multiplexers. The first multiplexer will be used to find the S output while the second multiplexer will be used to generate the C output of Half Adder.
- The general output equation of a 2 x 1 multiplexer is as follows:

$$Z = I_0 S_0' + I_1 S_0$$
- Truth table for sum & carry as follows :

S_1	S_0	Z	
0	0	0	S_0
0	1	1	
1	0	1	S_0'
1	1	0	

S_1	S_0	Z	
0	0	0	0
0	1	0	
1	0	0	S_0
1	1	1	

- Logic diagram of a Half Adder :





FULL ADDER :

Using 8 x 1 Multiplexer :

- The Boolean functions of sum and carry outputs are

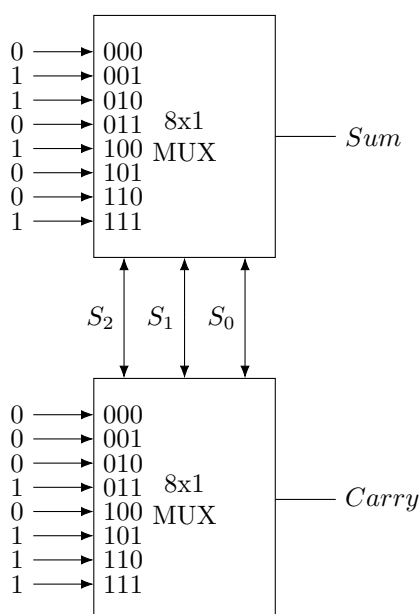
$$S(X, Y, Z) = \sum m(1, 2, 4, 7)$$

$$C(X, Y, Z) = \sum m(3, 5, 6, 7)$$

- Truth table for sum & carry as follows :

S_2	S_1	S_0	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

- Logic diagram of a Full Adder :



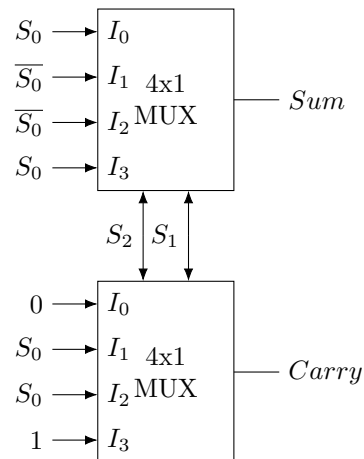
Using 4 x 1 Multiplexer :

- Truth table for sum & carry as follows :

S_2	S_1	S_0	Sum		Carry	
0	0	0	0	S_0	0	0
0	0	1	1	S_0	0	
0	1	0	1	S_0'	0	S_0
0	1	1	0	S_0'	1	
1	0	0	1	S_0'	0	S_0
1	0	1	0	S_0'	1	
1	1	0	0	S_0	1	1
1	1	1	1	S_0	1	



- Logic diagram of a Full Adder :



SUBTRACTORS :

[-For Video](#)

HALF SUBTRACTOR USING 4x1 MUX :

- The Boolean functions of sub and borrow outputs are

$$S(X, Y) = \sum m(1, 2)$$

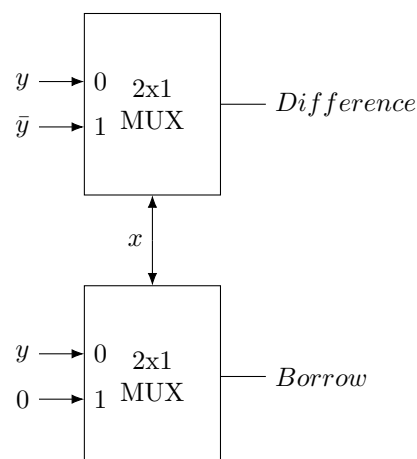
$$B(X, Y) = \sum m(1)$$

- Truth table for Sub & Borrow as follows :

S ₁	S ₀	Z
0	0	0
0	1	1
1	0	1
1	1	0

S ₁	S ₀	Z
0	0	0
0	1	1
1	0	0
1	1	0

- Logic diagram :





Using 2 x 1 Multiplexer :

- Here we will implement the logic diagram of a Half Subtractor using two 2 X 1 multiplexers.

- The Boolean functions of sub and borrow outputs are

$$S(X, Y) = \sum m(1, 2)$$

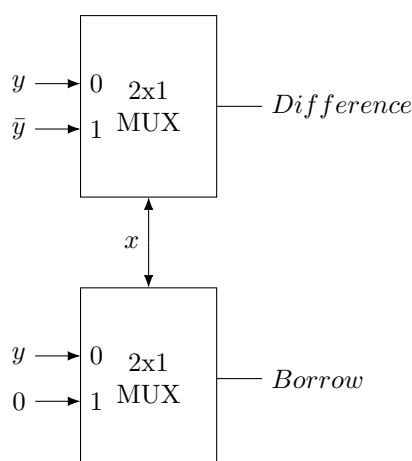
$$B(X, Y) = \sum m(1)$$

- Truth table for sub & borrow as follows :

x	y	Z	
0	0	0	
0	1	1	y
1	0	1	
1	1	0	y'

x	y	Z	
0	0	0	
0	1	1	y
1	0	0	
1	1	0	0

- Logic diagram:



FULL SUBTRACTOR :

Using 8 x 1 Multiplexer :

- The Boolean functions of sum and carry outputs are

$$S(X, Y, Z) = \sum m(1, 2, 4, 7)$$

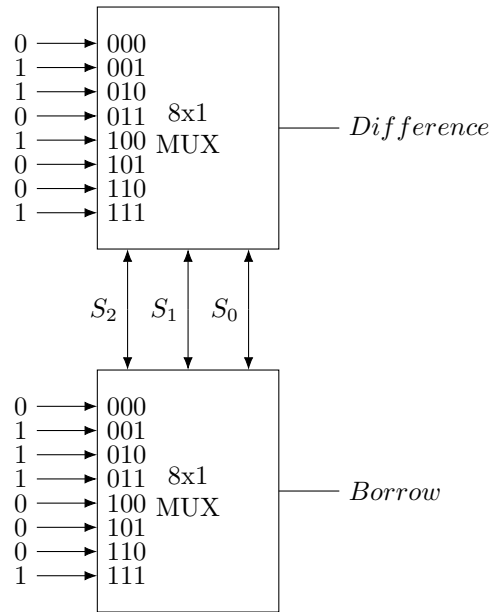
$$B(X, Y, Z) = \sum m(1, 2, 3, 7)$$

- Truth table for sub & borrow as follows :

S ₂	S ₁	S ₀	Sum	Carry
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



- Logic diagram :

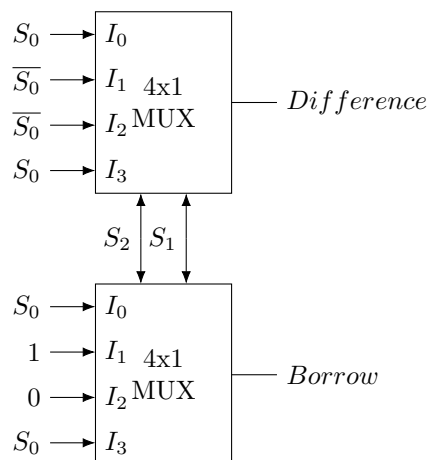


Using 4 x 1 Multiplexer :

- Truth table for sub & borrow as follows :

S_2	S_1	S_0	Sub		Borrow	
0	0	0	0		0	
0	0	1	1	S_0	1	S_0
0	1	0	1		1	
0	1	1	0	S_0'	1	1
1	0	0	1		0	
1	0	1	0	S_0'	0	0
1	1	0	0		0	
1	1	1	1	S_0	1	S_0

- Logic diagram :





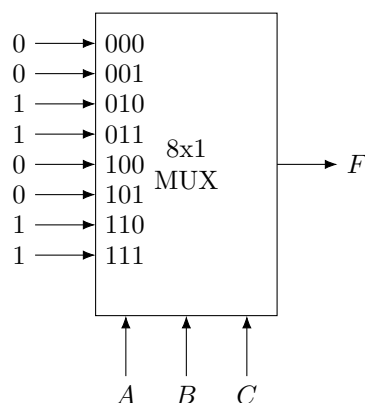
Problems :

1. Implement the boolean expression $F(A, B, C) = \sum m(2, 3, 6, 7)$ using a multiplexer.

Solution :

There are 3 variables in the given expression, hence $2^n = 2^3 = 8 : 1$ multiplexer. So, the mux has 8 input lines, 3 selection lines and one output.

The inputs, corresponding to the minterms (2, 3, 6, 7) are connected to logic 1 and the remaining terms to logic 0(grounded). The given input variables are connected as three selection lines.

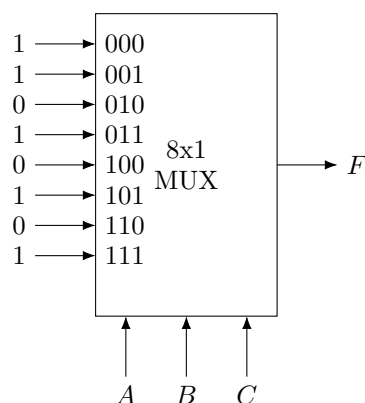


2. Implement the boolean expression $F(A, B, C) = \sum m(0, 1, 3, 5, 7)$ using a multiplexer.

Solution :

There are 3 variables and hence 8 : 1 multiplexr is used to solve the expression. The three input variables(A, B, C) are connected as three selection lines.

The inputs, corresponding to the minterms (0, 1, 3, 5, 7) are connected to logic 1 and the remaining terms to the logic 0(grounded).



3. Implement the boolean expression $F(A, B, C) = \sum m(0, 2, 5, 6)$ using 4 : 1 multiplexer.

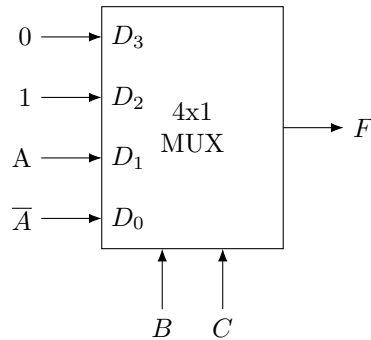
Solution :

In the given boolean expression, there are 3 variables. We should use $2^3 : 1 = 8 : 1$ multiplexer. But as per the question, it is to be implemented with 4 : 1 mux.

For 4 : 1 multiplexer, there should be 2 selection lines. So from the given 3 variables, the 2 least significant variables(B, C) are used as selection line inputs.



The minterms given in the boolean expression is circled and analyzed. After analyzing, the input values of 4 : 1 mux is obtained as $\bar{A}$, A, 1, 0. Thus the circuit can be drawn as follows.



4. Implement $F(A, B, C, D) = \sum m(0, 1, 5, 6, 8, 10, 12, 15)$ using 8 : 1 multiplexer.

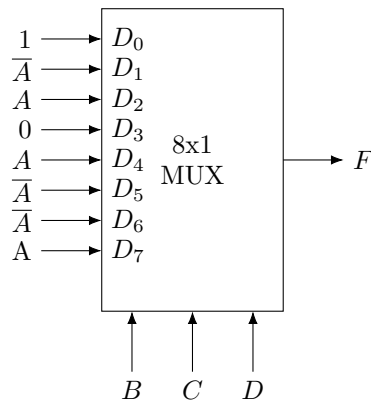
Solution :

In the given boolean expression, there are 4 variables. We should use $2^4 : 1 = 16 : 1$ multiplexer. But as per the question, it is to be implemented with 8 : 1 mux.

For 8 : 1 multiplexer, there should be 3 selection lines. So from the given 4 variables, the 3 least significant variables(B, C, D) are used as selection line inputs.

The 8 inputs are derived using the implementation table shown below

From the derived input, 8 : 1 multiplexer can be drawn as below





Note :

[-For Video](#)

1. The no. of $N \times 1$ mux required to convert $M \times 1$ mux is $\sum_{k=1}^z \lceil M/N^k \rceil$ where $z = \lceil \log_N M \rceil$

Example : If $M=32$ and $N=4$ then

$$\Rightarrow z = \lceil \log_4 32 \rceil = 3$$

$$\begin{aligned} \Rightarrow \sum_{k=1}^3 \lceil 32/4^k \rceil &= 32/4 + 32/4^2 + 32/4^3 \\ &= 8 + 2 + 1 \\ &= 11 \text{ [4x1 Mux are required]} \end{aligned}$$

2. The no. of levels required to convert $N \times 1$ mux into $M \times 1$ mux is $k \geq \lceil \log_N M \rceil$

Example : 8×1 Mux to 2×1 Mux

$$\Rightarrow k \geq \lceil \log_2 8 \rceil$$

$$\Rightarrow k \geq \lceil \log_2 8 \rceil$$

$$\Rightarrow 3 \lceil \log_2 2 \rceil = 3 \text{ levels}$$

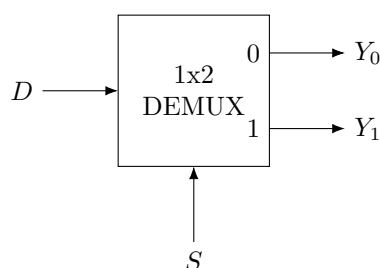
3.2 DE-MULTIPLEXERS :

[-For Video](#)

- De-Multiplexer is a combinational circuit that performs the reverse operation of Multiplexer.
- It has single input, 'n' selection lines and maximum of 2^n outputs.
- The input will be connected to one of these outputs based on the values of selection lines.
- Since there are 'n' selection lines, there will be 2^n possible combinations of zeros and ones. So, each combination can select only one output.
- De-Multiplexer is also called as De-Mux.

1-to-2 Demultiplexer :

- A 1-to-2 demultiplexer consists of one input line, two output lines and one select line.



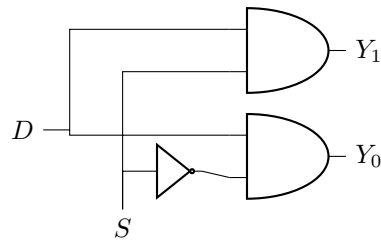
- The truth table of a 1x2 demultiplexer is shown below :

S	Y_1	Y_0
0	0	D
1	D	0

- Therefore, the expression for output Y_0 is $Y_0 = S' D$
- The expression for output Y_1 is $Y_1 = S D$



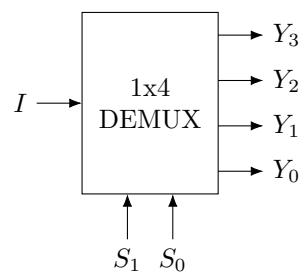
- Circuit Diagram :



1x4 De-Multiplexer :

[-For Video](#)

- 1x4 De-Multiplexer has one input I, two selection lines, s_1 & s_0 and four outputs Y_3 , Y_2 , Y_1 & Y_0 .

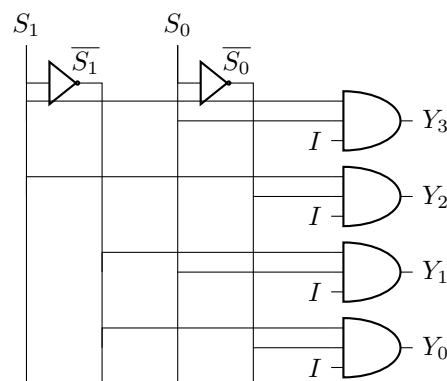


- The Truth table of 1x4 De-Multiplexer is shown below.

S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

- From the above Truth table, we can directly write the Boolean functions for each output as
 $Y_3 = S_1 S_0 I$
 $Y_2 = S_1 S_0' I$
 $Y_1 = S_1' S_0 I$
 $Y_0 = S_1' S_0' I$

- The circuit diagram of 1x4 De-Multiplexer is as follows :



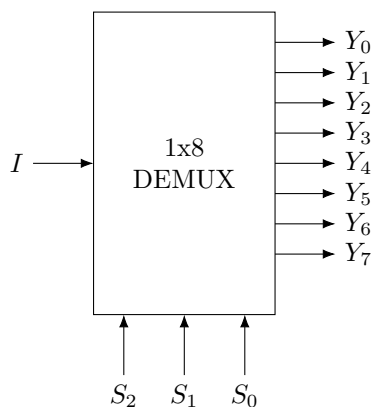


1x8 De-Multiplexer :

- we require two 1x4 De-Multiplexers in second stage in order to get the final eight outputs.
- Since, the number of inputs in second stage is two, we require 1x2 DeMultiplexer in first stage so that the outputs of first stage will be the inputs of second stage.
- The Truth table of 1x8 De-Multiplexer is shown below.

s ₂	s ₁	s ₀	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀
0	0	0	0	0	0	0	0	0	0	I
0	0	1	0	0	0	0	0	0	I	0
0	1	0	0	0	0	0	0	I	0	0
0	1	1	0	0	0	0	I	0	0	0
1	0	0	0	0	0	I	0	0	0	0
1	0	1	0	0	I	0	0	0	0	0
1	1	0	0	I	0	0	0	0	0	0
1	1	1	I	0	0	0	0	0	0	0

- The block diagram of 1x8 De-Multiplexer is as follows :



ADDERS

Half Adder :

[-For Video](#)

- The Boolean functions of sum and carry outputs are

$$S(X, Y) = \sum m(1, 2)$$

$$C(X, Y) = \sum m(3)$$

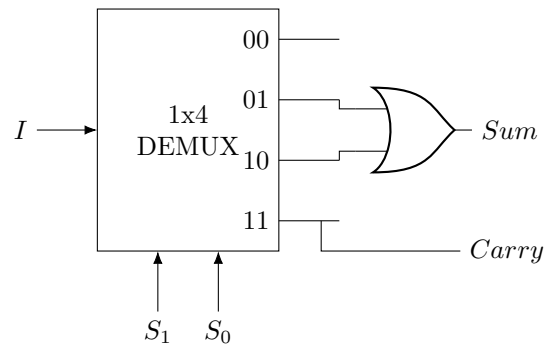
- Truth table for sum & carry as follows :

S ₁	S ₀	Z
0	0	0
0	1	1
1	0	1
1	1	0

S ₁	S ₀	Z
0	0	0
0	1	0
1	0	0
1	1	1



- Block Diagram :



FULL ADDER :

- The Boolean functions of sum and carry outputs are

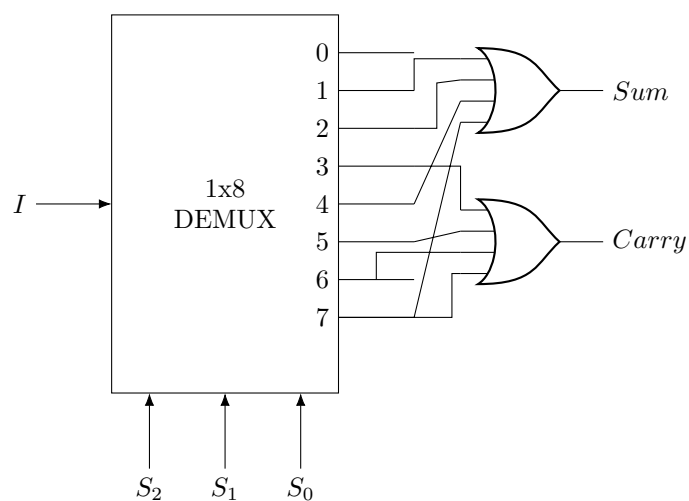
$$S(X, Y, Z) = \sum m(1, 2, 4, 7)$$

$$C(X, Y, Z) = \sum m(3, 5, 6, 7)$$

- Truth table for sum & carry as follows :

S ₂	S ₁	S ₀	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

- Logic diagram of a Full Adder :





SUBTRACTORS

Half Subtractor :

- The Boolean functions of sub and borrow outputs are

$$S(X, Y) = \sum m(1, 2)$$

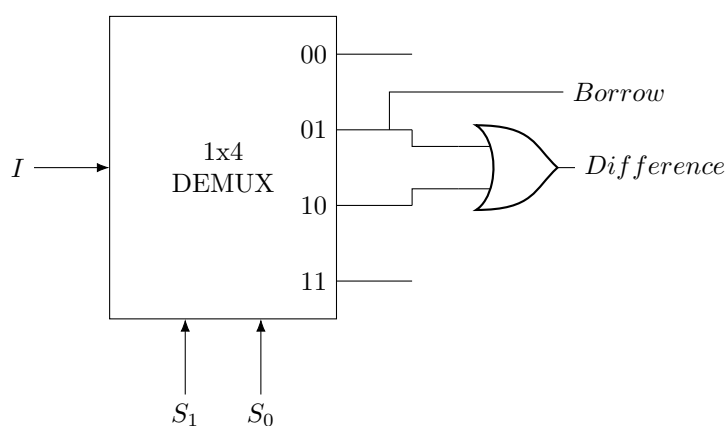
$$B(X, Y) = \sum m(1)$$

- Truth table for sub & borrow as follows :

S ₁	S ₀	Z
0	0	0
0	1	1
1	0	1
1	1	0

S ₁	S ₀	Z
0	0	0
0	1	1
1	0	0
1	1	0

- Block Diagram :



FULL SUBTRACTOR

- The Boolean functions of sub and Borrow outputs are

$$S(X, Y, Z) = \sum m(1, 2, 4, 7)$$

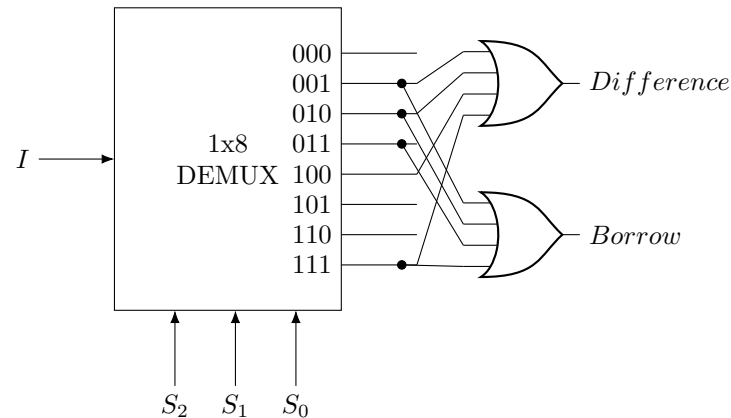
$$B(X, Y, Z) = \sum m(1, 2, 3, 7)$$

- Truth table for sub & borrow as follows :

S ₂	S ₁	S ₀	Sub	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1



- Logic diagram :



3.3 DECODERS

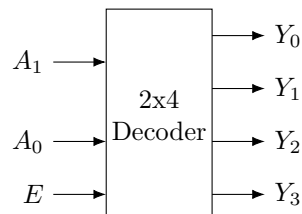
[-For Video](#)

- Decoder is a combinational circuit that has 'n' input lines and maximum of 2^n output lines.
- One of these outputs will be active High based on the combination of inputs present, when the decoder is enabled.
- That means decoder detects a particular code. The outputs of the decoder are nothing but the min terms of 'n' input variables lines, when it is enabled.

2 to 4 DECODER :

[-For Video](#)

- 2 to 4 Decoder has two inputs A_1 & A_0 and four outputs Y_3 , Y_2 , Y_1 & Y_0 .
- The block diagram of 2 to 4 decoder is as follows :



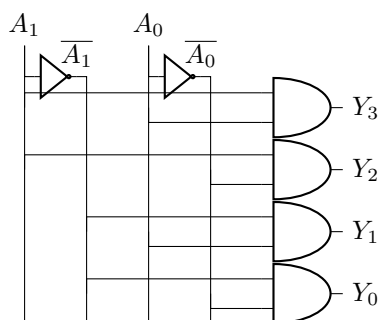
- The truth table is as follows :

Enable	Inputs		Outputs			
E	A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

- From Truth table, we can write the Boolean functions for each output as
 $Y_3 = E \cdot A_1 \cdot A_0$
 $Y_2 = E \cdot A_1 \cdot A_0'$
 $Y_1 = E \cdot A_1' \cdot A_0$
 $Y_0 = E \cdot A_1' \cdot A_0'$



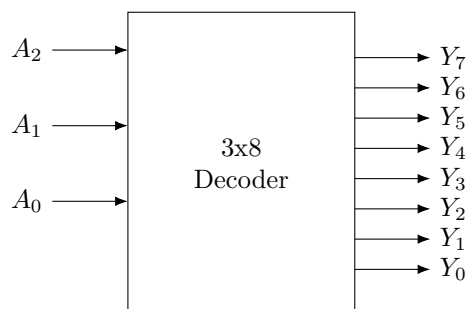
- The circuit diagram of 2 to 4 decoder is shown below :



3 to 8 Decoder :

[-For Video](#)

- The 3 to 8 line decoder is also known as **Binary to Octal Decoder**.
- In a 3 to 8 line decoder, there is a total of eight outputs, i.e., $Y_0, Y_1, Y_2, Y_3, Y_4, Y_5, Y_6,$ and Y_7 and three inputs, i.e., $A_0, A_1,$ and A_2 . This circuit has an enable input 'E'.
- Just like 2 to 4 line decoder, when enable 'E' is set to 1, one of these four outputs will be 1.
- The block diagram and the truth table of the 3 to 8 line encoder are given below. —



- The truth table is as follows :

Enable	Inputs			Outputs							
E	A ₂	A ₁	A ₀	Y ₀	Y ₁	Y ₂	Y ₃	Y ₄	Y ₅	Y ₆	Y ₇
0	x	x	x	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	0	0	0	0
1	0	0	1	0	1	0	0	0	0	0	0
1	0	1	0	0	0	1	0	0	0	0	0
1	0	1	1	0	0	0	1	0	0	0	0
1	1	0	0	0	0	0	0	1	0	0	0
1	1	0	1	0	0	0	0	0	1	0	0
1	1	1	0	0	0	0	0	0	0	1	0
1	1	1	1	0	0	0	0	0	0	0	1

- The logical expressions of the terms $Y_0, Y_1, Y_2, Y_3, Y_4, Y_5, Y_6,$ and Y_7 are as follows :

$$Y_0 = A_0' \cdot A_1' \cdot A_2'$$

$$Y_1 = A_0 \cdot A_1' \cdot A_2'$$

$$Y_2 = A_0' \cdot A_1 \cdot A_2'$$

$$Y_3 = A_0 \cdot A_1 \cdot A_2'$$

$$Y_4 = A_0 \cdot A_1' \cdot A_2$$

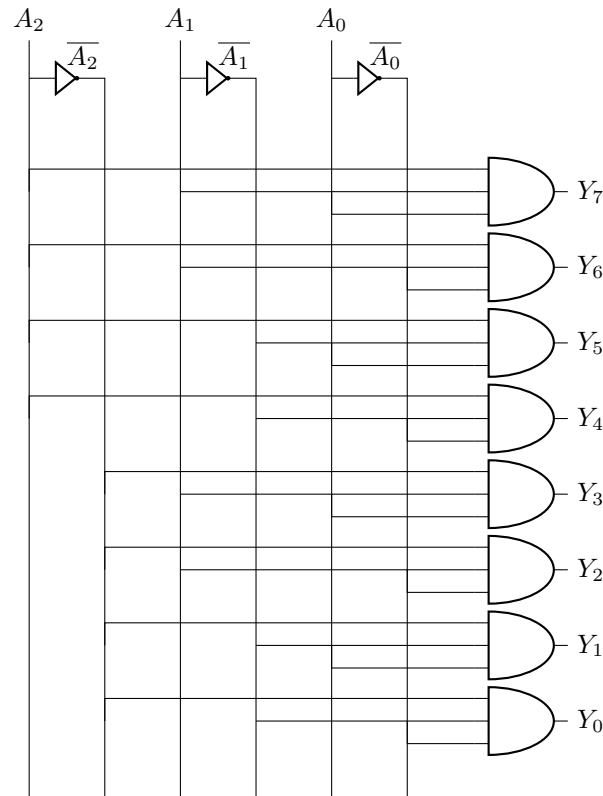


$$Y_5 = A_0 \cdot A_1' \cdot A_2$$

$$Y_6 = A_0' \cdot A_1 \cdot A_2$$

$$Y_7 = A_0 \cdot A_1 \cdot A_2$$

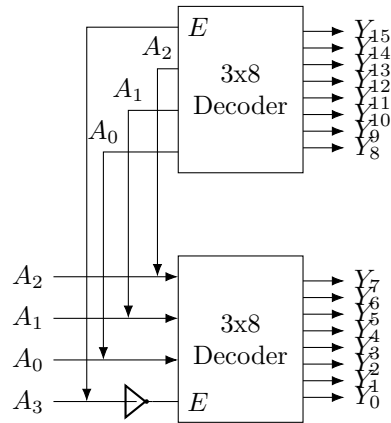
- Logical circuit of the above expressions is as follows :



4 to 16 Decoder :

[-For Video](#)

- In the 4 to 16 line decoder, there is a total of 16 outputs, i.e., $Y_0, Y_1, Y_2, \dots, Y_{15}$ and four inputs, i.e., A_0, A_1, A_2 , and A_3 .
- The 4 to 16 line decoder can be constructed using either 2 to 4 decoder or 3 to 8 decoder.
- Required number of lower order decoders = m_2/m_1
 $m_1 = 8$
 $m_2 = 16$
- Required number of 3 to 8 decoders = $16/8 = 2$
- The block diagram and the truth table of the 3 to 8 line decoder are given below.



- The truth table is as follows :

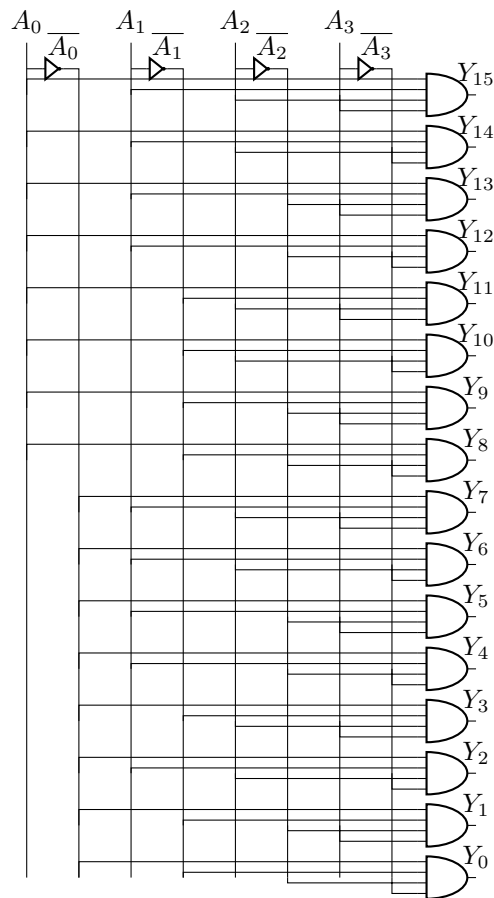
INPUTS				OUTPUTS															
A ₃	A ₂	A ₁	A ₀	Y ₁₅	Y ₁₄	Y ₁₃	Y ₁₂	Y ₁₁	Y ₁₀	Y ₉	Y ₈	Y ₇	Y ₆	Y ₅	Y ₄	Y ₃	Y ₂	Y ₁	Y ₀
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	1	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
1	0	1	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

- The logical expression of the term A₀, A₁, A₂, ..., A₁₅ are as follows:

$$\begin{aligned}
 Y_0 &= A_0' . A_1' . A_2' . A_3' \\
 Y_1 &= A_0' . A_1' . A_2' . A_3 \\
 Y_2 &= A_0' . A_1' . A_2 . A_3' \\
 Y_3 &= A_0' . A_1' . A_2 . A_3 \\
 Y_4 &= A_0' . A_1 . A_2' . A_3' \\
 Y_5 &= A_0' . A_1 . A_2' . A_3 \\
 Y_6 &= A_0' . A_1 . A_2 . A_3' \\
 Y_7 &= A_0' . A_1 . A_2 . A_3 \\
 Y_8 &= A_0 . A_1' . A_2' . A_3' \\
 Y_9 &= A_0 . A_1' . A_2' . A_3 \\
 Y_{10} &= A_0 . A_1' . A_2 . A_3' \\
 Y_{11} &= A_0 . A_1' . A_2 . A_3 \\
 Y_{12} &= A_0 . A_1 . A_2' . A_3' \\
 Y_{13} &= A_0 . A_1 . A_2' . A_3 \\
 Y_{14} &= A_0 . A_1 . A_2 . A_3' \\
 Y_{15} &= A_0 . A_1 . A_2 . A_3
 \end{aligned}$$



- Logical circuit of the above expressions is as follows :



ADDERS USING DECODERS

[-For Video](#)

Half Adder :

- The Boolean functions of sum and carry outputs are

$$S(X, Y) = \sum m(1, 2)$$

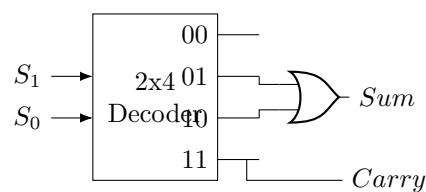
$$C(X, Y) = \sum m(3)$$

- Truth table for sum & carry as follows :

S ₁	S ₀	Z
0	0	0
0	1	1
1	0	1
1	1	0

S ₁	S ₀	Z
0	0	0
0	1	0
1	0	0
1	1	1

- Block Diagram :





Full Adder :

- The Boolean functions of sum and carry outputs are

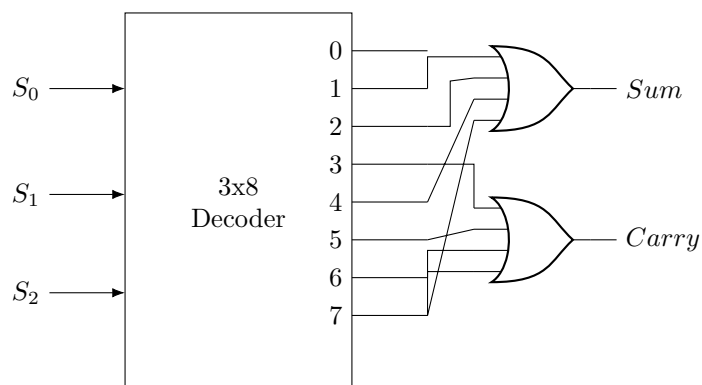
$$S(X, Y, Z) = \sum m(1, 2, 4, 7)$$

$$C(X, Y, Z) = \sum m(3, 5, 6, 7)$$

- Truth table for sum & carry as follows :

S_2	S_1	S_0	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

- Logic diagram of a Full Adder :



SUBTRACTORS

Half Subtractor :

- The Boolean functions of sub and borrow outputs are

$$S(X, Y) = \sum m(1, 2)$$

$$B(X, Y) = \sum m(1)$$

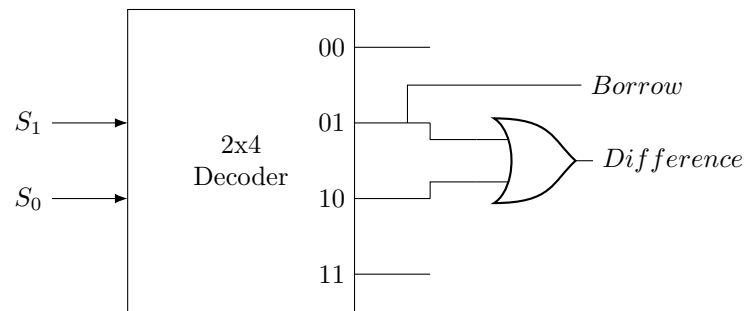
- Truth table for sub & borrow as follows :

S_1	S_0	Z
0	0	0
0	1	1
1	0	1
1	1	0

S_1	S_0	Z
0	0	0
0	1	1
1	0	0
1	1	0



- Block Diagram :



Full Subtractor :

- The Boolean functions of sub and Borrow outputs are

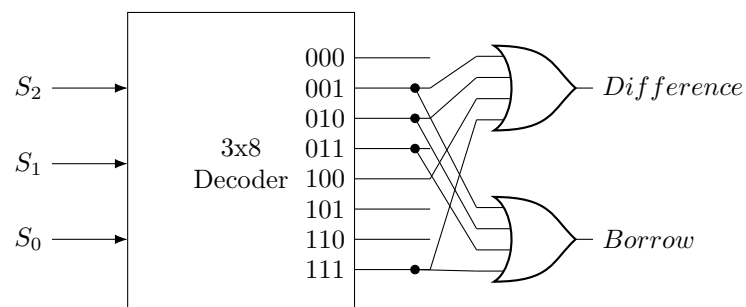
$$S(X, Y, Z) = \sum m(1, 2, 4, 7) \text{ and,}$$

$$C(X, Y, Z) = \sum m(1, 2, 3, 7)$$

- Truth table for sub & borrow as follows :

S_2	S_1	S_0	Sub	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

- Logic diagram :



Advantages of Decoders :

[-For Video](#)

1. Logic circuit implementation.
2. Conversions
3. Parity Checker
4. ROM matrix
5. Address Decoding



Parity Checker :

- A parity decoder or parity generator is used to add an extra bit at the end of the signal or single byte.
- For example, you are transferring 3 bits of data so one more bit is added to the data making it 4 bits. The extra bit that is added is used for error checking. When data is received at the receiver end then the parity bit is checked for error detection.
- Two main types of parity decoder are explained below:-

1. Even parity decoder

3-bit message			Even parity bit generator (P)
A	B	C	Y
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

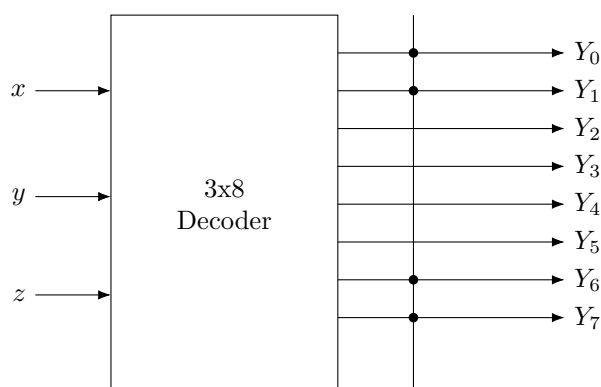
2. Odd parity decoder

3-bit message			Odd parity bit generator (P)
A	B	C	Y
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

ROM Matrix :

[-For Video](#)

1. Implement $F(x,y,z) = \sum m(0,1,6,7)$ by using ROM matrix ?





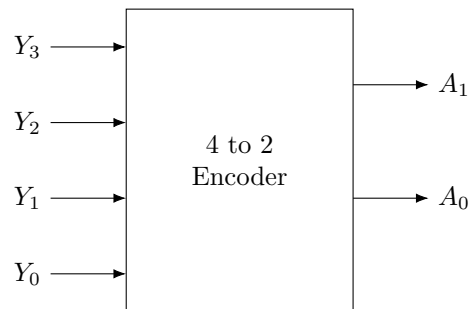
3.4 ENCODERS

[-For Video](#)

- An Encoder is a combinational circuit that performs the reverse operation of Decoder. It has maximum of 2^n input lines and 'n' output lines.
- It will produce a binary code equivalent to the input, which is active High. Therefore, the encoder encodes 2^n input lines with 'n' bits.
- It is optional to represent the enable signal in encoders.

4 to 2 Encoder :

- 4 to 2 Encoder has four inputs Y_3 , Y_2 , Y_1 & Y_0 and two outputs A_1 & A_0 .
- The block diagram of 4 to 2 Encoder is as follows :



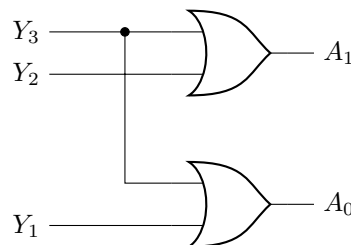
- The Truth table of 4 to 2 encoder is shown below.

Inputs				Outputs	
Y_3	Y_2	Y_1	Y_0	A_1	A_0
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

- From Truth table, we can write the Boolean functions for each output as

$$A_1 = Y_3 + Y_2$$

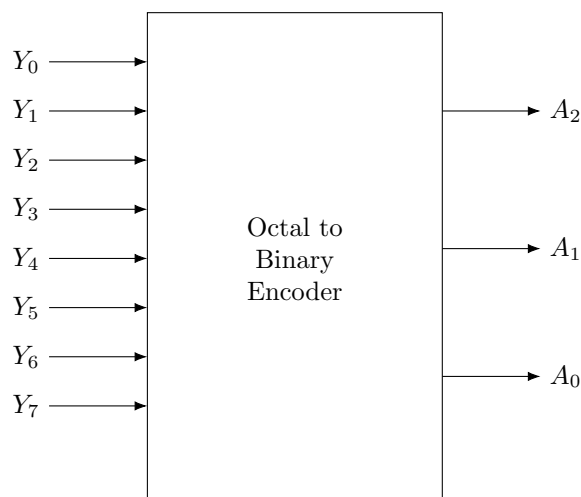
$$A_0 = Y_3 + Y_1$$
- The circuit diagram of 4 to 2 encoder is shown below.





Octal to Binary Encoder :

- Octal to binary Encoder has eight inputs, Y_7 to Y_0 and three outputs A_2 , A_1 & A_0 . Octal to binary encoder is nothing but 8 to 3 encoder.
- The block diagram of octal to binary Encoder is as follows :



- The Truth table is shown below.

Inputs								Outputs		
Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

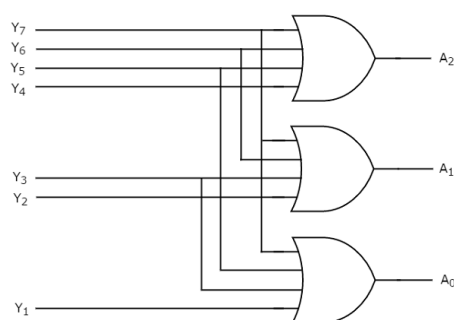
- From Truth table, we can write the Boolean functions for each output as

$$A_2 = Y_7 + Y_6 + Y_5 + Y_4$$

$$A_1 = Y_7 + Y_6 + Y_3 + Y_2$$

$$A_0 = Y_7 + Y_5 + Y_3 + Y_1$$

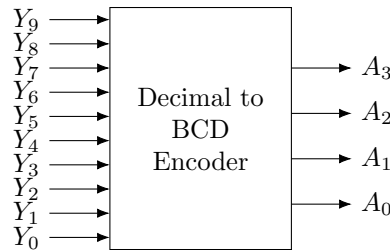
- The circuit diagram of octal to binary encoder is as follows :





Decimal to BCD Encoder :

- The Octal to Binary Encoder is also known as 10 to 4 line Encoder.
- In 10 to 4 line encoder, there are total of ten inputs, i.e., $Y_0, Y_1, Y_2, Y_3, Y_4, Y_5, Y_6, Y_7, Y_8$, and Y_9 and four outputs, i.e., A_0, A_1, A_2 , and A_3 .
- In 10-input lines, one input-line is set to true at a time to get the respective BCD code in the output side.
- The block diagram decimal to BCD encoder is given below.



- The Truth table of decimal to bcd to 2 encoder is shown below.

Inputs										Outputs			
Y_9	Y_8	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0	A_3	A_2	A_1	A_0
0	0	0	0	0	0	0	0	0	1	0	0	0	0
0	0	0	0	0	0	0	0	1	0	0	0	0	1
0	0	0	0	0	0	0	1	0	0	0	0	1	0
0	0	0	0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	0	1	0	0	0	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	0	0	1	1	0
0	0	1	0	0	0	0	0	0	0	0	1	1	1
0	1	0	0	0	0	0	0	0	0	1	1	1	1
1	0	0	0	0	0	0	0	0	0	1	1	1	1

- The logical expression of the term A_0, A_1, A_2 , and A_3 is as follows :

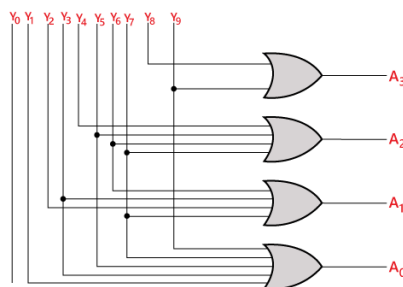
$$A_3 = Y_9 + Y_8$$

$$A_2 = Y_7 + Y_6 + Y_5 + Y_4$$

$$A_1 = Y_7 + Y_6 + Y_3 + Y_2$$

$$A_0 = Y_9 + Y_7 + Y_5 + Y_3 + Y_1$$

- The circuit diagram of octal to binary encoder is as follows :





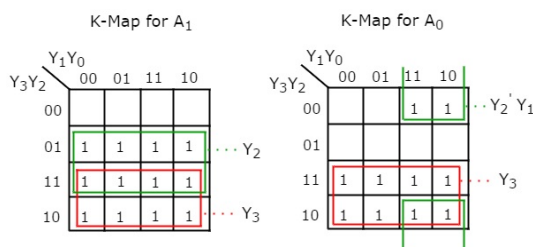
3.5 PRIORITY ENCODER

-For Video

- A 4 to 2 priority encoder has four inputs Y_3, Y_2, Y_1 & Y_0 and two outputs A_1 & A_0 .
- Here, the input, Y_3 has the highest priority, whereas the input, Y_0 has the lowest priority.
- In this case, even if more than one input is '1' at the same time, the output will be the binary code corresponding to the input, which is having higher priority.
- We considered one more output, V in order to know, whether the code available at outputs is valid or not.
- **Note :**
 1. If at least one input of the encoder is '1', then the code available at outputs is a valid one. In this case, the output, V will be equal to 1.
 2. If all the inputs of encoder are '0', then the code available at outputs is not a valid one. In this case, the output, V will be equal to 0.
- The Truth table of 4 to 2 priority encoder is shown below.

Inputs				Outputs		V
Y_3	Y_2	Y_1	Y_0	A_1	A_0	V
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

- Use 4 variable K-maps for getting simplified expressions for each output.



- The simplified Boolean functions are

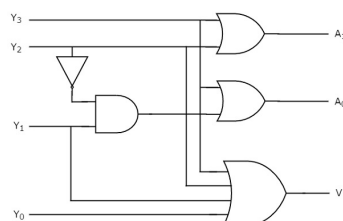
$$A_1 = Y_3 + Y_2$$

$$A_0 = Y_3 + Y_2'Y_1$$

- Similarly, we will get the Boolean function of output, V as

$$V = Y_3 + Y_2 + Y_1 + Y_0$$

- The circuit diagram of 4 to 2 priority encoder is shown below.

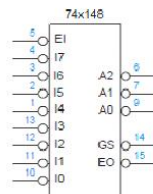




3.6 IC 74X148

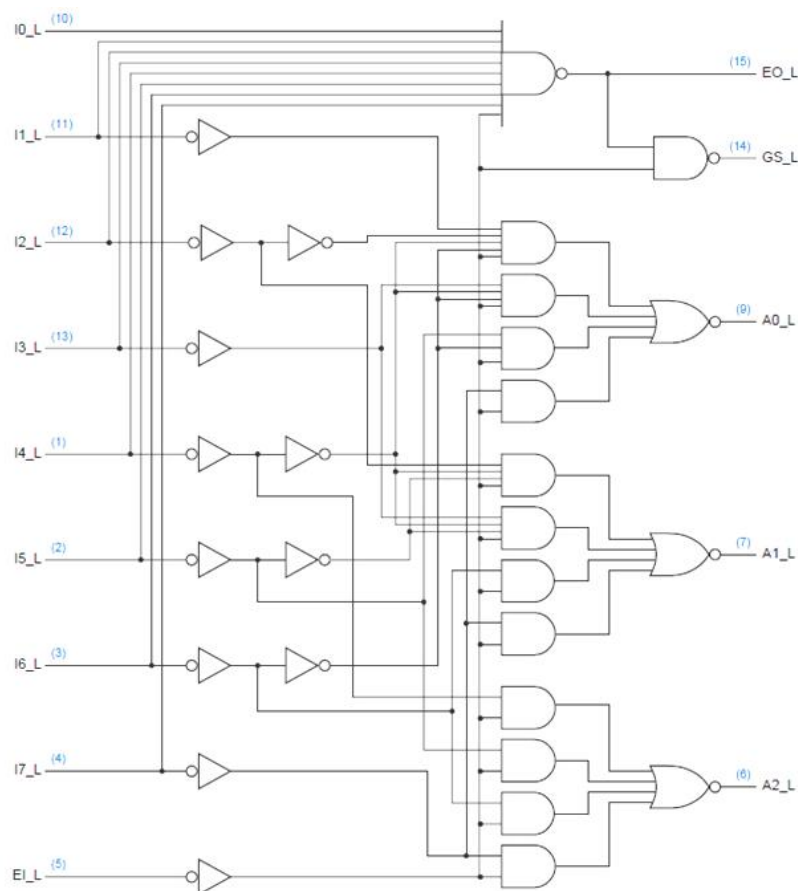
[-For Video](#)

- The main difference between this IC and the “generic” priority encoder of Figure 5 is that its inputs and outputs are active low.
- It has an enable input, EI_L, that must be asserted for any of its outputs to be asserted.
- The 74x148 is a commercially available, MSI 8-input priority encoder.
- Logic symbol for the 74x148 8-input priority encoder and its Truth Table :



Inputs									Outputs				
/EI	/I0	/I1	/I2	/I3	/I4	/I5	/I6	/I7	/A2	/A1	/A0	/GS	/EO
1	x	x	x	x	x	x	x	x	1	1	1	1	1
0	x	x	x	x	x	x	x	0	0	0	0	0	1
0	x	x	x	x	x	x	0	1	0	0	1	0	1
0	x	x	x	x	x	0	1	1	0	1	0	0	1
0	x	x	x	x	0	1	1	1	0	1	1	0	1
0	x	x	x	0	1	1	1	1	1	0	0	0	1
0	x	x	0	1	1	1	1	1	1	0	1	0	1
0	x	0	1	1	1	1	1	1	1	1	0	0	1
0	0	1	1	1	1	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0

- Logic diagram for the 74x148 8-input priority encoder, including pin numbers for a standard 16-pin dual in-line package



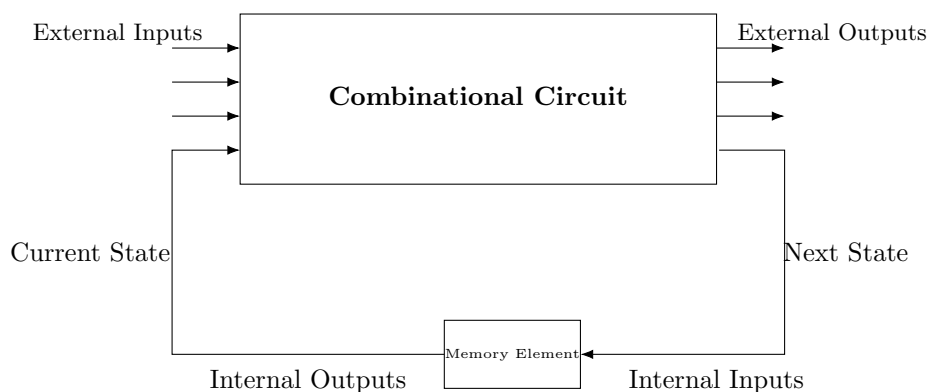


UNIT-4

4.1 SEQUENTIAL CIRCUITS

[-For Video](#)

- Sequential circuits are digital circuits that store and use the previous state information to determine their next state.
- Unlike combinational circuits, which only depend on the current input values to produce outputs, sequential circuits depend on both the current inputs and the previous state stored in memory elements.
- Sequential circuits are commonly used in digital systems to implement state machines, timers, counters, and memory elements.
- The memory elements in sequential circuits can be implemented using flip-flops, which are circuits that store binary values and maintain their state even when the inputs change.
- The following figure shows the block diagram of sequential circuit.



Block Diagram of Sequential Circuit

- This sequential circuit contains a set of inputs and outputs
- The outputs of sequential circuit depends not only on the combination of present inputs but also on the previous outputs
- Previous output is nothing but the present state. Therefore, sequential circuits contain combinational circuits along with memory storage elements.
- Some sequential circuits may not contain combinational circuits, but only memory elements.

4.2 COMBINATIONAL vs SEQUENTIAL CIRCUITS

[-For Video](#)

Combinational Circuits	Sequential Circuits
Outputs depend only on present inputs.	Outputs depend on both present inputs and present state.
Feedback path is not present.	Feedback path is present.
Memory elements are not required.	Memory elements are required.
Clock signal is not required.	Clock signal is required.
Easy to design.	Difficult to design.



4.3 TYPES OF SEQUENTIAL CIRCUITS

[-For Video](#)

- There are two types of Sequential circuits.
 1. Asynchronous sequential circuits
 2. Synchronous sequential circuits

1. Asynchronous sequential circuit :

- If some or all the outputs of a sequential circuit do not change or affect with respect to active transition of clock signal, then that sequential circuit is called as Asynchronous sequential circuit.
- That means, all the outputs of asynchronous sequential circuits do not change or affect at the same time. Therefore, most of the outputs of asynchronous sequential circuits are not in synchronous with either only positive edges or only negative edges of clock signal.

2. Synchronous sequential circuit :

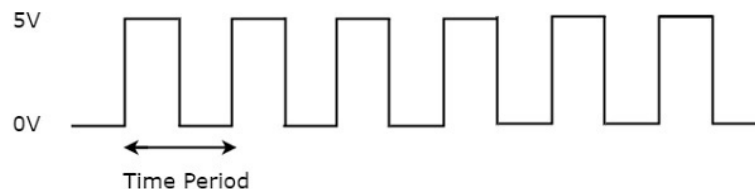
- If all the outputs of a sequential circuit change or affect with respect to active transition of clock signal, then that sequential circuit is called as Synchronous sequential circuit.
- That means, all the outputs of synchronous sequential circuits change or affect at the same time. Therefore, the outputs of synchronous sequential circuits are in synchronous with either only positive edges or only negative edges of clock signal.

4.4 CLOCK SIGNAL AND TRIGGERING

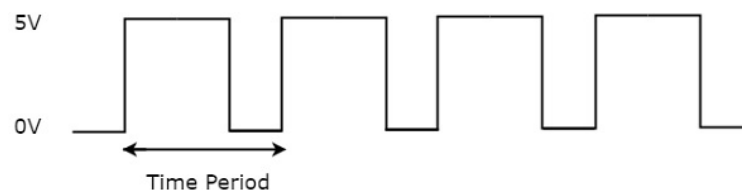
[-For Video](#)

Clock Signal :

- Clock signal is a periodic signal and its ON time and OFF time need not be the same.
- We can represent the clock signal as a square wave, when both its ON time and OFF time are same. This clock signal is shown in the following figure.



- In the above figure, square wave is considered as clock signal. This signal stays at logic High 5V for some time and stays at logic Low 0V for equal amount of time.
- This pattern repeats with some time period. In this case, the time period will be equal to either twice of ON time or twice of OFF time.
- We can represent the clock signal as train of pulses, when ON time and OFF time are not same. This clock signal is shown in the following figure.





- In the figure, train of pulses is considered as clock signal. This signal stays at logic High 5V for some time and stays at logic Low 0V for some other time.
- This pattern repeats with some time period. In this case, the time period will be equal to sum of ON time and OFF time.
- The reciprocal of the time period of clock signal is known as the **frequency of the clock signal**.
- All sequential circuits are operated with clock signal. So, the frequency at which the sequential circuits can be operated accordingly the clock signal frequency has to be chosen.

Types of Triggering :

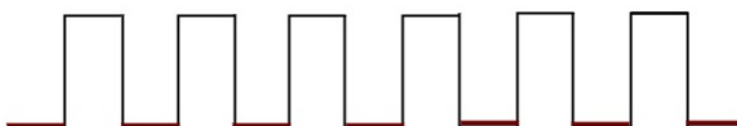
- Following are the two possible types of triggering that are used in sequential circuits.
 1. Level triggering
 2. Edge triggering

1. Level triggering :

- There are two levels, namely logic High and logic Low in clock signal.
- Following are the two types of level triggering.
 1. Positive level triggering
 2. Negative level triggering
- If the sequential circuit is operated with the clock signal when it is in Logic High, then that type of triggering is known as Positive level triggering. It is highlighted in below figure.

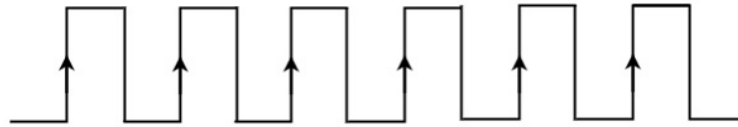


- If the sequential circuit is operated with the clock signal when it is in Logic Low, then that type of triggering is known as **Negative level triggering** .

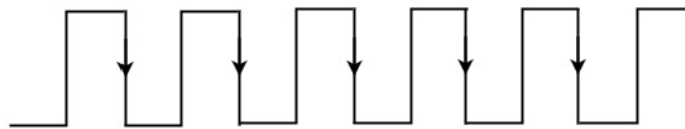


2. Edge triggering :

- There are two types of transitions that occur in clock signal. That means, the clock signal transitions either from Logic Low to Logic High or Logic High to Logic Low.
- Following are the two types of edge triggering based on the transitions of clock signal.
 1. Positive edge triggering
 2. Negative edge triggering
- If the sequential circuit is operated with the clock signal that is transitioning from Logic Low to Logic High, then that type of triggering is known as **Positive edge triggering**.
- It is also called as **rising edge triggering**. It is shown below.

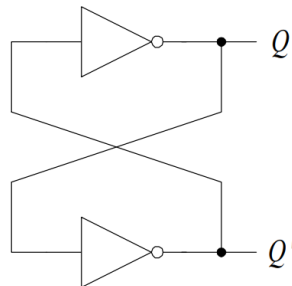


- If the sequential circuit is operated with the clock signal that is transitioning from Logic High to Logic Low, then that type of triggering is known as **Negative edge triggering**.



4.5 MEMORY ELEMENTS IN SEQUENTIAL CIRCUITS [-For Video](#)

- There are two types of memory elements based on the type of triggering that is suitable to operate it.
 1. Latches
 2. Flip-flops
- Latches operate with enable signal, which is level sensitive. Whereas, flip-flops are edge sensitive.
- They are made of logic gates. One latch or flip-flop can store a 1-bit of information.
- The simplest sequential circuit or storage element is a **bistable element**, which is constructed with two inverters connected sequentially in a loop as shown below. It has no inputs and two outputs labeled Q and Q' .



- Since the circuit has no inputs, we cannot change the values of Q and Q' . Assume that $Q = 0$ when we switch on the power. Since Q is also the input to the bottom inverter, Q' , therefore, Q' is a 1. 1 going to the input of the top inverter will produce a 0 at the output Q , which is what we started off with. Similarly, if we start the circuit with $Q = 1$, we will get $Q' = 0$, and again we get a stable situation.
- Bistable element can be also implemented by using NOR and NAND gates.

1. LATCHES

- Latches are digital circuits that store a single bit of information and hold its value until it is updated by new input signals.
- A latch is an example of a bistable multivibrator, that is, a device with exactly two stable states. These states are high-output and low-output.



- They are used in digital systems as temporary storage elements to store binary information.
- Latches can be implemented using various digital logic gates, such as AND, OR, NOT, NAND, and NOR gates.
- There are basically four main types of latches and flip-flops: SR, D, JK, and T.

2.FLIPFLOPS

- The flip-flops are basically the circuits that maintain a certain state unless and until directed by the input for changing that state. We can construct a basic flip-flop using four-NOR and four-NAND gates.
- Flip flop is popularly known as the basic digital memory circuit. It has its two states as logic 1(High) and logic 0(low) states.
- A flip flop is a sequential circuit which consist of single binary state of information or data. The digital circuit is a flip flop which has two outputs and are of opposite states. It is also known as a Bistable Multivibrator.
- The flip-flops are of the following types:
 1. S-R Flip Flop
 2. J-K Flip Flop
 3. T Flip Flop
 4. D Flip Flop

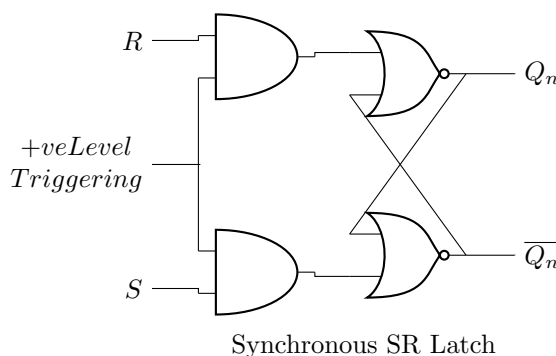
4.6 SR LATCH/FLIPFLOP

[-For Video](#)

- An SR Flip Flop (also referred to as an SR Latch) is the most simple type of flip flop. It has two inputs S and R and two outputs Q and $\bar{Q}$.
- The state of this latch is determined by the condition of Q. If Q is 1 the latch is said to be **SET** and if Q is 0 the latch is said to be **RESET**. This SR Latch or Flip flop can be designed either by two cross-coupled NAND gates or two-cross coupled NOR gates.
- When we design this latch by using NOR gates, it will be an active high S-R latch. That means it is SET when S = 1. When we design this latch by using NAND gates, it will be an active low S-R latch. That means it is SET when S = 1. SR Flip Flop is also called SET RESET Flip Flop.

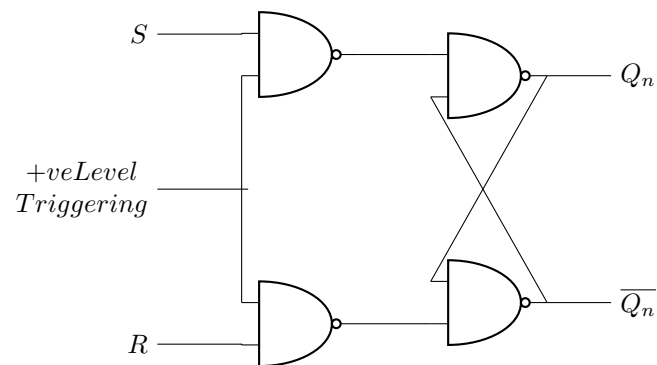
Construction of SR Latch/Fliflop :

- There are following two methods for constructing a SR flip flop
 1. By using NOR latch
 2. By using NAND latch
- 1.By using NOR gates :





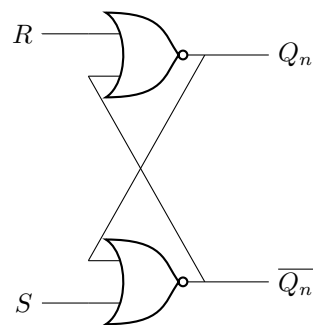
- 2.By using NAND gates :



Synchronous SR Latch

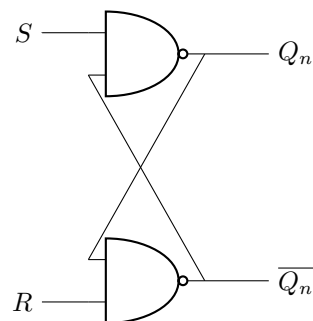
Asynchronous SR-Flipflop :

- 1.By using NOR gates :



Asynchronous SR Latch

- 2.By using NAND gates :



Asynchronous SR Latch



• Truth table for SR latch/Flip Flop

S	R	Q _n (Present State)	Q _{n+1} (Next State)	States and Conditions
0	0	0	0	Latch State condition S = R = 0
0	0	1	1	
0	1	0	0	Reset State condition S =0, R=1
0	1	1	0	
1	0	0	1	Set State condition S =1, R =0
1	0	1	1	
1	1	0	φ	Undefined State condition S = R = 1
1	1	1	φ	

• Characteristic Equation :

S	RQ _n	R'Q _n			
		00	01	11	10
s' 0		0	1	1	2
s 1		1	1	x	x
		4	5	7	6

• $Q_{n+1} = (SR + SR') (Q_n + Q'_n) + Q_n (S'R' + SR')$
 $Q_{n+1} = S + Q_n R'$

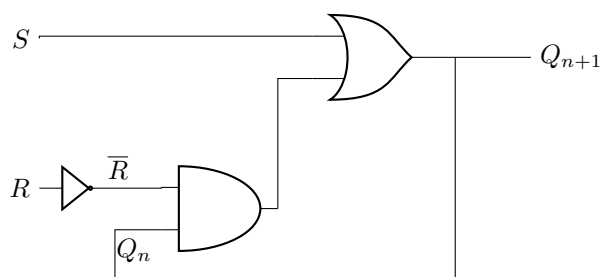
• Excitation table :

Q _n	Q _{n+1}	S	R
0	0	0	φ
0	1	1	0
1	0	0	1
1	1	φ	0

• Function table :

S	R	Q _{n+1}
0	0	Q _n
0	1	0
1	0	1
1	1	φ

• CIRCUIT DIAGRAM :





- **Advantages :**

1. **Simplicity :** The SR flip-flop is a simple and easy-to-understand circuit.
2. **Speed :** The SR flip-flop is relatively fast, which makes it suitable for use in high-speed digital systems.
3. **Low power consumption :** The SR flip-flop consumes very little power, which makes it suitable for use in battery-powered devices.
4. **Bi-stable operation :** The SR flip-flop can be set and reset, making it a bi-stable circuit that can hold a state indefinitely until it is changed by an input signal.

- **Dis-Advantages :**

1. **Race condition :** The SR flip-flop is susceptible to race conditions, which occur when the output state changes unpredictably due to variations in the timing of input signals.
2. **Invalid states :** If both the set and reset inputs are activated at the same time, the SR flip-flop can enter an invalid state where both outputs are high or both are low. This can lead to unpredictable behavior in digital systems.
3. **Limited scalability :** The SR flip-flop can be difficult to scale up to more complex digital systems, as it can lead to increased complexity and the potential for errors.

- **Applications :** Some of the applications of SR flip-flop in real-world includes

1. Control systems
2. Memory storage
3. Digital counters
4. Data synchronization
5. Oscillators

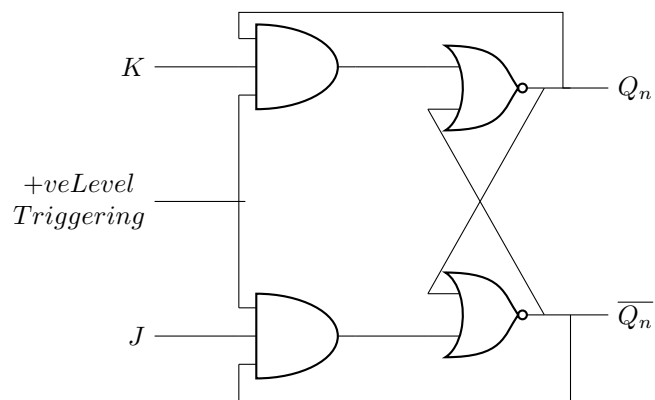
4.7 JK FLIPFLOP

[-For Video](#)

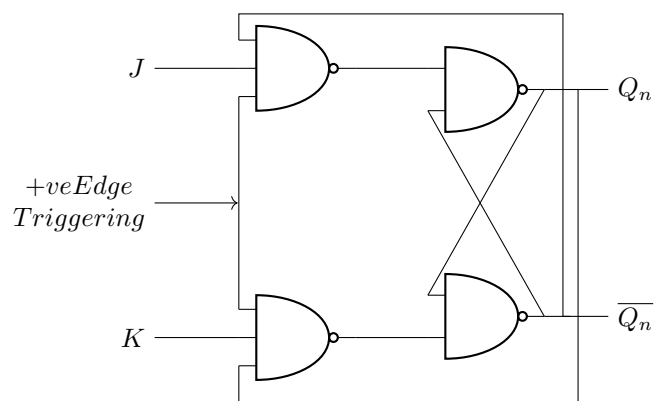
- The SR Flip Flop or Set-Reset flip flop has lots of advantages. But, it has the following switching problems.
 1. When Set 'S' and Reset 'R' inputs are set to 1, this condition is always avoided.
 2. When the Set or Reset input changes their state while the enable input is 1, the incorrect latching action occurs.
- The JK Flip Flop removes above drawback of SR Flip Flop. The JK flip flop is a universal flip flop having two inputs 'J' and 'K'.
- Jack Kilby flip-flop(JK Flipflop) is very prominent as it is very versatile and can be used as a basic memory element.
- The JK flip flop work in the same way as the SR flip flop work. The JK flip flop has 'J' and 'K' flip flop instead of 'S' and 'R'.
- The only difference between JK flip flop and SR flip flop is that when both inputs of SR flip flop is set to 1, the circuit produces the invalid states as outputs, but in case of JK flip flop, there are no invalid states even if both 'J' and 'K' flip flops are set to 1.

Construction of JK Latch/Fliflop :

- There are following two methods for constructing a JK flip flop
 1. By using NOR latch
 2. By using NAND latch



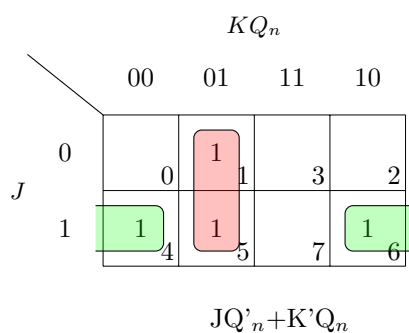
Synchronous JK Latch



Synchronous JK Flipflop

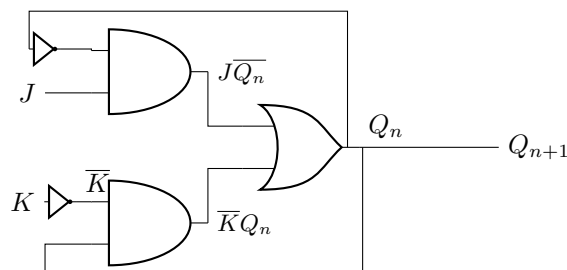
• Truth table for JK latch/Flip Flop

J	K	Q_n (Present State)	Q_{n+1} (Next State)	States and Conditions
0	0	0	0	Latch State condition $S = R = 0$
0	0	1	1	
0	1	0	0	Reset State condition $S = 0, R = 1$
0	1	1	0	
1	0	0	1	Set State condition $S = 1, R = 0$
1	0	1	1	
1	1	0	1	Complement State condition $S = R = 1$
1	1	1	0	





- **Circuit Diagram :**



- **Excitation table :**

Q_n	Q_{n+1}	J	K
0	0	0	ϕ
0	1	1	ϕ
1	0	ϕ	1
1	1	ϕ	0

- **Function table :**

J	K	Q_{n+1}
0	0	Q_n
0	1	0
1	0	1
1	1	Q'_n

Drawbacks :

1. **Complexity :** Compared to other types of flipflops(D,T, SR), JK flipflop requires additional logic gates to implement which consumes extra memory resources and increases complexity to operate.
2. **Propagation Delay :** This is the major problem present in JK-FF. Propagation delay results a timing delay in certain application which are time-flow sensitive.
3. **Race Problem :** This issue arises when the clock input's timing pulse isn't given enough time to turn "Off" before the output Q's state is altered.

Race Around Condition :

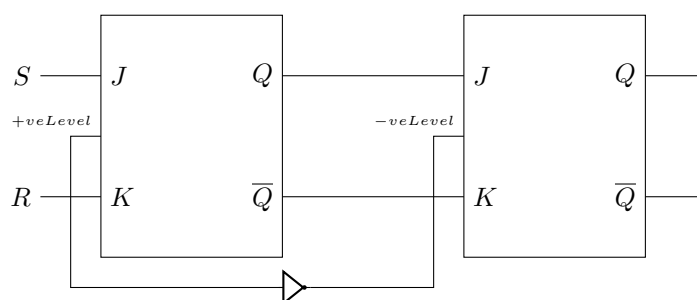
- The problem of the race-around condition does not exist in the flip-flops where the inputs do not change during the presence of clock pulse. But, in the case of JK flip-flops, the inputs change during the clock pulse due to the feedback path present between inputs and outputs. Hence, in JK flip-flops, the race around condition is a major problem.
- The problem of race-around condition and the uncertainty of output can be avoided by increasing the delay of the flip-flops. For that, the delay of the flip-flops must be greater than the duration of the clock signal, i.e. $\delta t > T$. In another way, the duration of the applied clock signal (T) must be reduced to make it less than the delay of the flip-flops (δt).
- However, the increase in the delay of the flip-flops is not a good practice because it decreases the speed of the system. On the other hand, it is also quite difficult to decrease the duration of the clock pulse (T) beyond the delay of the flip-flops (δt). This is because, the delay of the JK flip-flops (δt) is of the order of nanoseconds.
- Hence, the most practical way to solve the problem of race-around condition in JK flip-flops is to use the JK flip-flops in the Master and Slave Mode. In the master-slave mode of JK flip-flops, two JK flip-flops are cascaded.



4.8 MASTER SLAVE FLIPFLOP

[-For Video](#)

- In "JK Flip Flop", when both the inputs and CLK set to 1 for a long time, then Q output toggle until the CLK is 1. Thus, the uncertain or unreliable output produces. This problem is referred to as a race-round condition in JK flip-flop and avoided by ensuring that the CLK set to 1 only for a very short time.
- The master-slave flip flop is constructed by combining two JK flip flops. These flip flops are connected in a series configuration. In these two flip flops, the 1st flip flop work as "master", called the master flip flop, and the 2nd work as a "slave", called slave flip flop.
- The master-slave flip flop is designed in such a way that the output of the "master" flip flop is passed to both the inputs of the "slave" flip flop. The output of the "slave" flip flop is passed to inputs of the master flip flop.
- In "master-slave flip flop", apart from these two flip flops, an inverter or NOT gate is also used. For passing the inverted clock pulse to the "slave" flip flop, the inverter is connected to the clock's pulse. In simple words, when CP set to false for "master", then CP is set to true for "slave", and when CP set to true for "master", then CP is set to false for "slave".

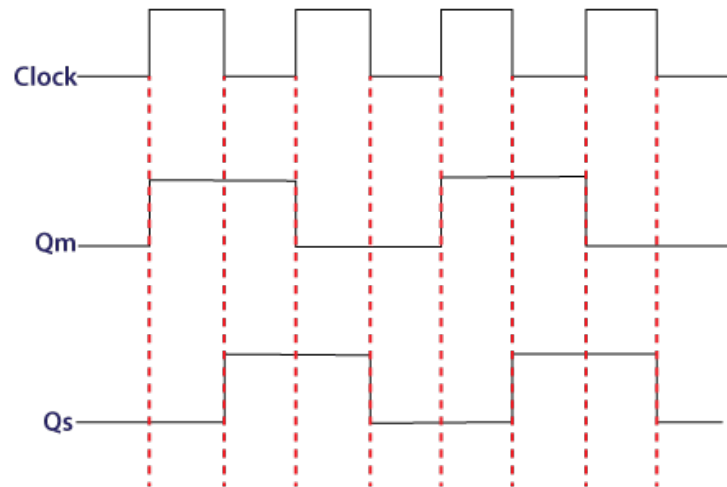


Working :

- When the clock pulse is true, the slave flip flop will be in the isolated state, and the system's state may be affected by the J and K inputs. The "slave" remains isolated until the CP is 1.
- When the CP set to 0, the master flip-flop passes the information to the slave flip flop to obtain the output.
- The master flip flop responds first from the slave because the master flip flop is the positive level trigger, and the slave flip flop is the negative level trigger.
- The output $Q'=1$ of the master flip flop is passed to the slave flip flop as an input K when the input J set to 0 and K set to 1. The clock forces the slave flip flop to work as reset, and then the slave copies the master flip flop.
- When $J=1$, and $K=0$, the output $Q=1$ is passed to the J input of the slave. The clock's negative transition sets the slave and copies the master.
- The master flip flop toggles on the clock's positive transition when the inputs J and K set to 1. At that time, the slave flip flop toggles on the clock's negative transition.
- The flip flop will be disabled, and Q remains unchanged when both the inputs of the JK flip flop set to 0.



Timing Diagram of a Master Flip Flop :



- When the clock pulse set to 1, the output of the master flip flop will be one until the clock input remains 0.
- When the clock pulse becomes high again, then the master's output is 0, which will be set to 1 when the clock becomes one again.
- The master flip flop is operational when the clock pulse is 1. The slave's output remains 0 until the clock is not set to 0 because the slave flip flop is not operational.
- The slave flip flop is operational when the clock pulse is 0. The output of the master remains one until the clock is not set to 0 again.
- Toggling occurs during the entire process because the output changes once in the cycle.

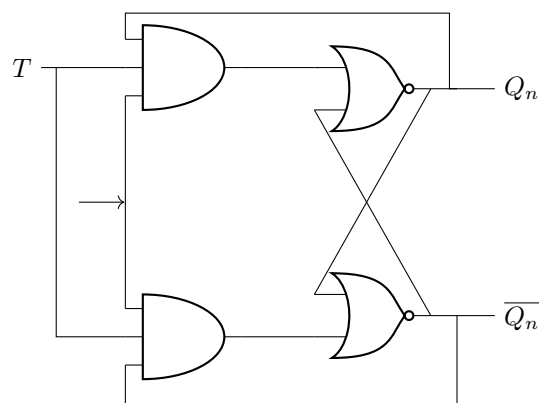
4.9 T-FLIPFLOP

[-For Video](#)

- The T flip-flop is also called toggle flip-flop. It is a change of the JK flip-flop. The T flip flop is received by relating both inputs of a JK flip-flop. The T flip-flop is received by relating the inputs 'J' and 'K'. When $T = 0$, both AND gates are disabled. Therefore, there is no change in the output. When $T = 1$, the output toggles.
- In T flip flop, "T" defines the term "Toggle". In T Flip Flop, we provide only a single input called "Toggle" or "Trigger" input to avoid an intermediate state occurrence. Now, this flip-flop work as a Toggle switch. The next output state is changed with the complement of the present state output. This process is known as "Toggling".
- We can construct the "T Flip Flop" by making changes in the "JK Flip Flop". The "T Flip Flop" has only one input, which is constructed by connecting the input of JK flip flop. This single input is called T. In simple words, we can construct the "T Flip Flop" by converting a "JK Flip Flop". Sometimes the "T Flip Flop" is referred to as single input "JK Flip Flop".

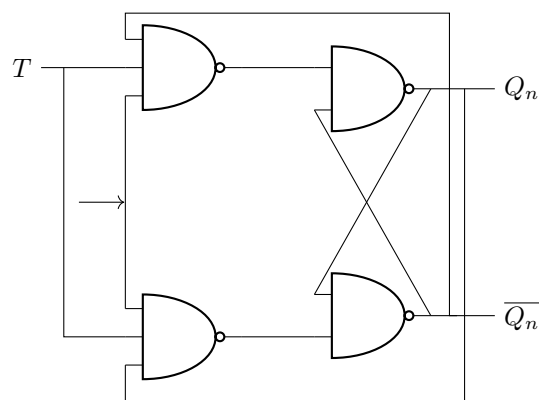


- T-FlipFlop using NOR gates :



Synchronous T Flipflop

- T-FlipFlop using NAND gates :



Synchronous T Flipflop

- Characteristic table :

T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

- Excitation table :

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

- Function table :

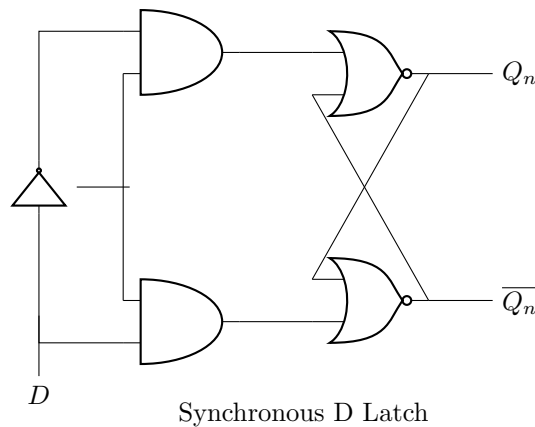
T	Q_{n+1}
0	Q_n
1	Q'_n



4.10 D-FLIPFLOP

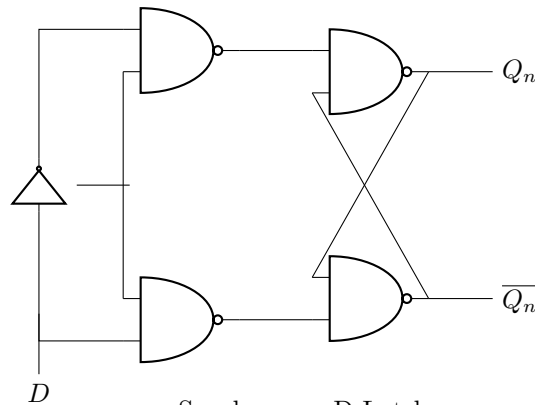
[-For Video](#)

- The D flip flop is the most important flip flop from other clocked types. It ensures that at the same time, both the inputs, i.e., S and R, are never equal to 1.
- The Delay flip-flop is designed using a gated SR flip-flop with an inverter connected between the inputs allowing for a single input D(Data).
- This single data input, which is labeled as "D" used in place of the "Set" input and for the complementary "Reset" input, the inverter is used.
- Thus, the level-sensitive D-type or D flip flop is constructed from a level-sensitive SR flip flop.
- **D-FlipFlop using NOR gates :**



Synchronous D Latch

- **D-FlipFlop using NAND gates :**



Synchronous D Latch

- **Characteristic table :**

D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1



- **Excitation table :**

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	0
1	1	1

- **Function table :**

D	Q_{n+1}
0	0
1	1

4.11 CONVERSION OF FLIPFLOPS

[-For Video](#)

- Earlier we discussed the four flip-flops, namely SR flip-flop, D flip-flop, JK flip-flop & T flip-flop. We can convert one flip-flop into the remaining three flip-flops by including some additional logic. So, there will be total of twelve flip-flop conversions.
- Follow these steps for converting one flip-flop to the other.

Steps :

1. Consider the characteristic table of desired flip-flop.
2. Fill the excitation values inputs of given flip-flop for each combination of present state and next state. The excitation table for all flip-flops is shown below.

Present State	Next State	SR inputs	D flip-flop input	JK inputs	T input
Q_n	Q_{n+1}	SR	D	JK	T
0	0	0 x	0	0 x	0
0	1	1 0	1	1 x	1
1	0	0 1	0	x 1	1
1	1	x 0	1	x 0	0

3. Get the simplified expressions for each excitation input. If necessary, use Kmaps for simplifying.
4. Draw the circuit diagram of desired flip-flop according to the simplified expressions using given flip-flop and necessary logic gates.

1. SR Flip-Flop to other Flip-Flop Conversions :

- Following are the three possible conversions of SR flip-flop to other flip-flops.
 1. SR flip-flop to D flip-flop
 2. SR flip-flop to JK flip-flop
 3. SR flip-flop to T flip-flop

SR flip-flop to D flip-flop conversion :

- Here, the given flip-flop is SR flip-flop and the desired flip-flop is D flip-flop. Therefore, consider the following characteristic table of D flip-flop.

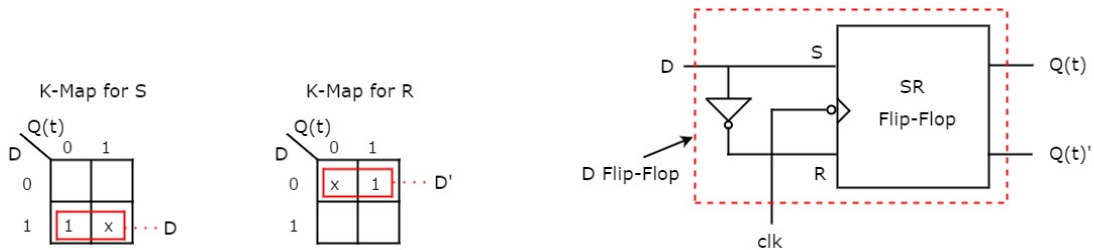
D flip-flop input	Present State	Next State
D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1



- We know that SR flip-flop has two inputs S & R. So, write down the excitation values of SR flip-flop for each combination of present state and next state values.
- The following table shows the characteristic table of D flip-flop along with the excitation inputs of SR flip-flop.

D flip-flop input	Present State	Next State	SR flip-flop inputs	
D	Q_n	Q_{n+1}	S	R
0	0	0	0	x
0	1	0	0	1
1	0	1	1	0
1	1	1	x	0

- So, we got $S = D$ & $R = D'$ after simplifying. The k-maps & circuit diagram of D flip-flop are shown below.



- This circuit consists of SR flip-flop and an inverter. This inverter produces an output, which is complement of input, D. So, the overall circuit has single input, D and two outputs Q_n & Q_{n+1}

2. D Flip-Flop to other Flip-Flop Conversions

- Following are the three possible conversions of D flip-flop to other flip-flops.
 1. D flip-flop to T flip-flop
 2. D flip-flop to SR flip-flop
 3. D flip-flop to JK flip-flop

D flip-flop to T flip-flop conversion :

- Here, the given flip-flop is D flip-flop and the desired flip-flop is T flip-flop. Therefore, consider the following characteristic table of T flip-flop.

T flip-flop input	Present State	Next State
T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

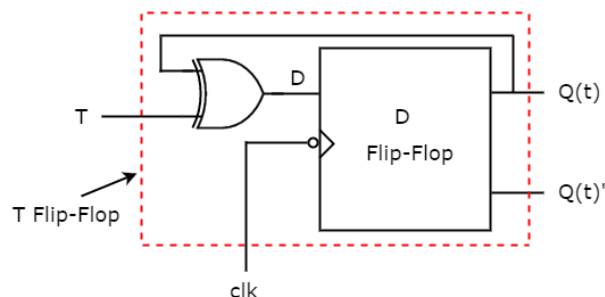
- We know that D flip-flop has single input D. So, write down the excitation values of D flip-flop for each combination of present state and next state values.
- The following table shows the characteristic table of T flip-flop along with the excitation input of D flip-flop.

T flip-flop input	Present State	Next State	D flip-flop input
T	Q_n	Q_{n+1}	D
0	0	0	0
0	1	1	1
1	0	1	1
1	1	0	0

- From the above table, we can directly write the Boolean function of D as below.

$$D = T \oplus Q(n)$$

- So, we require a two input Exclusive-OR gate along with D flip-flop. The circuit diagram of T flip-flop is shown below.



- This circuit consists of D flip-flop and an Exclusive-OR gate. This Exclusive-OR gate produces an output, which is Ex-OR of T and Q_n . So, the overall circuit has single input, T and two outputs Q_n & Q_{n+1} . Hence, it is a T flip-flop.

3. JK Flip-Flop to other Flip-Flop Conversions :

- Following are the three possible conversions of JK flip-flop to other flip-flops.
 - JK flip-flop to D flip-flop
 - JK flip-flop to SR flip-flop
 - JK flip-flop to T flip-flop

JK flip-flop to T flip-flop conversion :

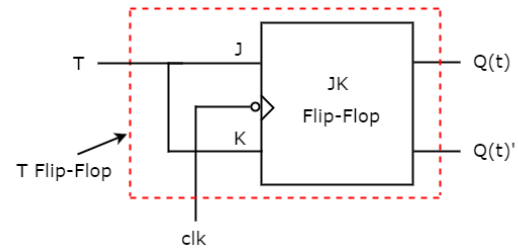
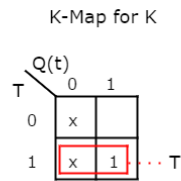
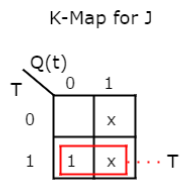
- Here, the given flip-flop is JK flip-flop and the desired flip-flop is T flip-flop. Therefore, consider the following characteristic table of T flip-flop.

T flip-flop input	Present State	Next State
T	Q_n	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

- We know that JK flip-flop has two inputs J & K. So, write down the excitation values of JK flip-flop for each combination of present state and next state values.
- The following table shows the characteristic table of T flip-flop along with the excitation inputs of JK flip-flop.

T flip-flop input	Present State	Next State	JK flip-flop inputs	
T	Q_n	Q_{n+1}	J	K
0	0	0	0	x
0	1	1	x	0
1	0	1	1	x
1	1	0	x	1

- We got, $J = T$ & $K = T$ after simplifying. The k-maps & circuit diagram of T flip-flop are shown below.



- This circuit consists of JK flip-flop only. It doesn't require any other gates. Just connect the same input T to both J & K. So, the overall circuit has single input, T and two outputs Q_n & Q_{n+1} . Hence, it is a T flip-flop.

4. T Flip-Flop to other Flip-Flop Conversions

- Following are the three possible conversions of T flip-flop to other flip-flops.
 1. T flip-flop to D flip-flop
 2. T flip-flop to SR flip-flop
 3. T flip-flop to JK flip-flop

T flip-flop to D flip-flop conversion :

- Here, the given flip-flop is T flip-flop and the desired flip-flop is D flip-flop. Therefore, consider the characteristic table of D flip-flop and write down the excitation values of T flip-flop for each combination of present state and next state values.
- The following table shows the characteristic table of D flip-flop.

D flip-flop input	Present State	Next State
D	Q_n	Q_{n+1}
0	0	0
0	1	0
1	0	1
1	1	1

- Write down the excitation values of T flip-flop for each combination of present state and next state values.
- The following table shows the characteristic table of D flip-flop along with the excitation input of T flip-flop.

D flip-flop input	Present State	Next State	T flip-flop input
D	Q_n	Q_{n+1}	T
0	0	0	0
0	1	0	1
1	0	1	1
1	1	1	0

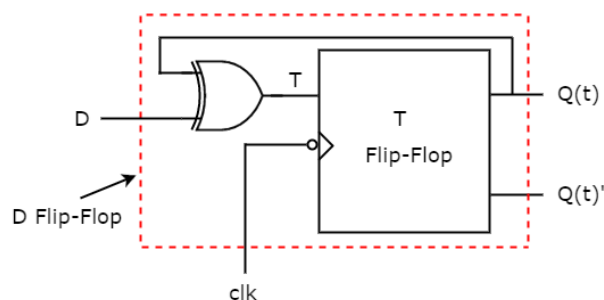
- From the above table, we can directly write the Boolean function of D as below.

$$T = D \oplus Q(n)$$

- So, we require a two input Exclusive-OR gate along with T flip-flop.



- The circuit diagram of D flip-flop is shown below.

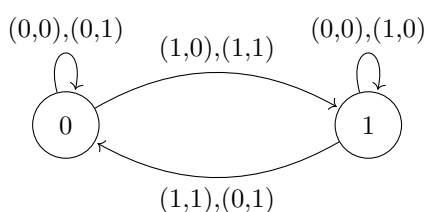


- This circuit consists of T flip-flop and an Exclusive-OR gate. This Exclusive-OR gate produces an output, which is Ex-OR of D and Q_t . So, the overall circuit has single input, D and two outputs Q_n & Q_{n+1} . Hence, it is a D flip-flop.

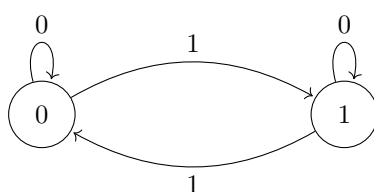
4.12 STATE DIAGRAMS FOR FLIPFLOPS

[-For Video](#)

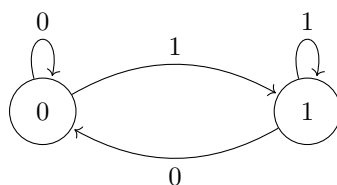
- JK FlipFlop :



- T FlipFlop :



- D FlipFlop :





UNIT-5

5.1 COUNTERS

[-For Video](#)

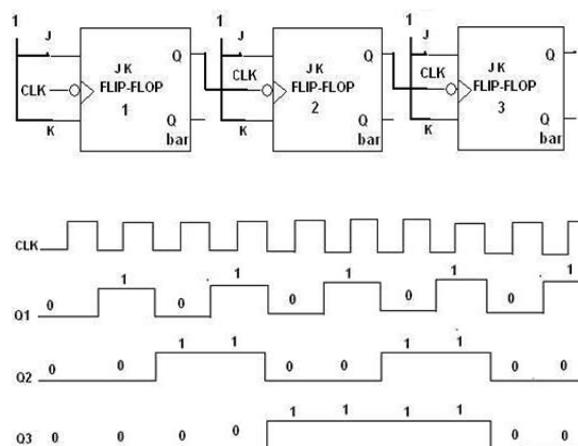
- A special type of sequential circuit used to count the pulse is known as a counter, or a collection of flip flops where the clock signal is applied is known as counters.
- The counter is one of the widest applications of the flip flop. Based on the clock pulse, the output of the counter contains a predefined state. The number of the pulse can be counted using the output of the counter.

Types of Counters :

- The Counters in digital electronics are broadly classified into two main categories :

1. Asynchronous Counters :

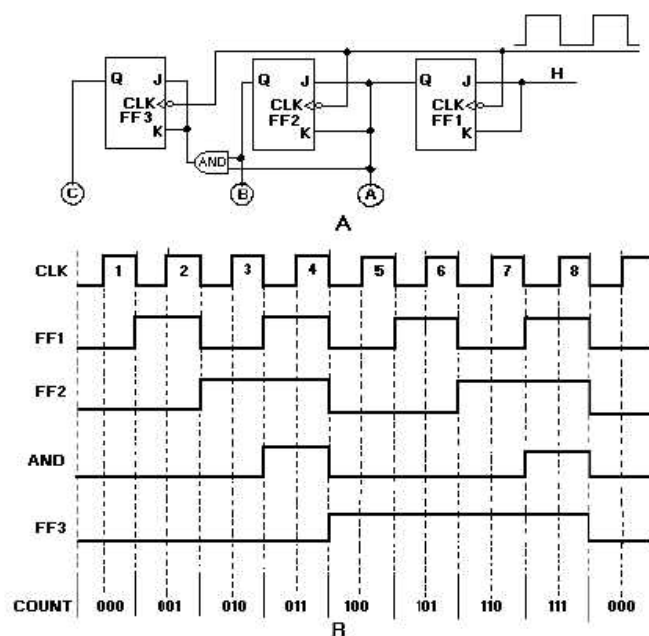
- The asynchronous counter is also called as Ripple counter and is made up of a series of flip-flops.
- If the flip-flops do not receive the same clock signal, the counter is referred to as asynchronous.
- Only the first flip-flop receives a clock signal from the system clock. The remaining flip-flops receive the clock signal from the previous stage flip-output.



- The timing diagram clearly shows that Q_1 changes as soon as the rising edge of the clock pulse is encountered, Q_2 changes when the rising edge of Q_1 is encountered (because Q_1 is like the clock pulse for the second flip-flop), and so on.
- A ripple counter is a cascaded configuration of flip-flops in which the output of one flip-flop drives the clock input of the flip-flop after it.

2. Synchronous Counters :

- The synchronous counter is a counter also known as the parallel counter, operation is fast if we compare it with asynchronous counters.
- The synchronous counter has a single global clock that drives each flip-flop, allowing output to change in parallel.
- The synchronous counter can operate at higher frequencies, In the synchronous counter, all of the flip flops clock inputs use the same source and produce the output at the same time.
- The synchronous counter produces fewer errors than the asynchronous counter.



Note :

- **Modulus Counters**, or simply MOD counters, are defined based on the number of states that the counter will sequence through before returning back to its original value. For example, a 2-bit counter that counts from 00 to 11 in binary, that is 0 to 3 in decimal, has a modulus value of 4 (00 → 01 → 10 → 11, and return back to 00) so would therefore be called a modulo-4, or mod-4 counter. Note that it has taken four clock pulses to get from 00 to 11.
- A 'Mod- 2^N ' counter will require 'N' number of flip-flops connected together to count a single data bit while providing 2^n different output states, (n is the number of bits). Note that 'N' is always a whole integer value.

5.2 DIFFERENCE BETWEEN SYNCHRONOUS AND ASYNCHRONOUS COUNTERS

[-For Video](#)

Synchronous	Asynchronous
All flip flops are triggered simultaneously by same clock.	Flip-flops are activated with different clocks.
Operate more quickly than asynchronous counters.	These operate more slowly than synchronous.
There are no decoding issues using Synchronous Counter.	Asynchronous Counter produces decoding errors.
Another name is Parallel Counter	Serial Counter is another name for this.
Due to growing no.of states, creating is difficult	Designing & implementing is relatively simple.
Examples are Johnson and Ring counters.	Ripple UP and DOWN counters are two examples.

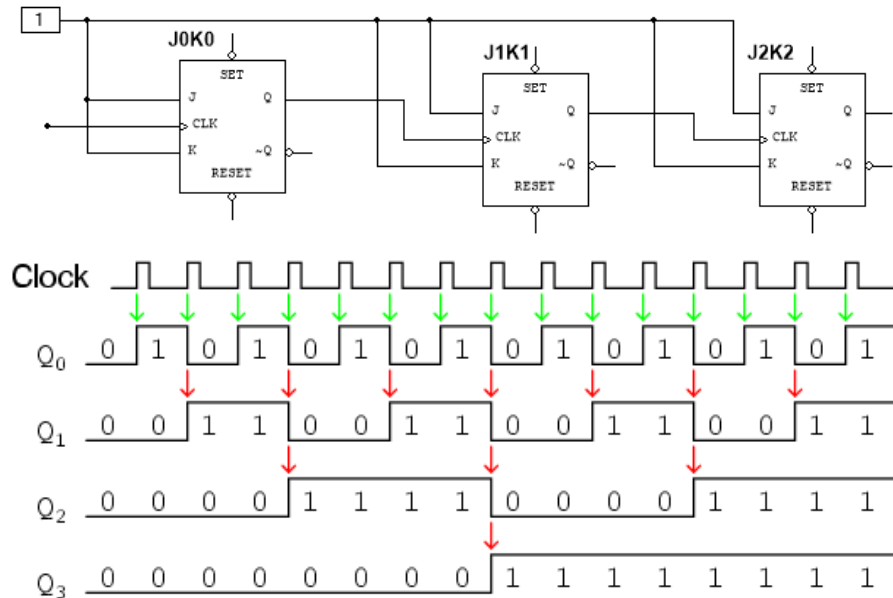
Advantages of Counters :

1. Counting
2. Time Measurement
3. Analog to digital converter
4. Frequency division



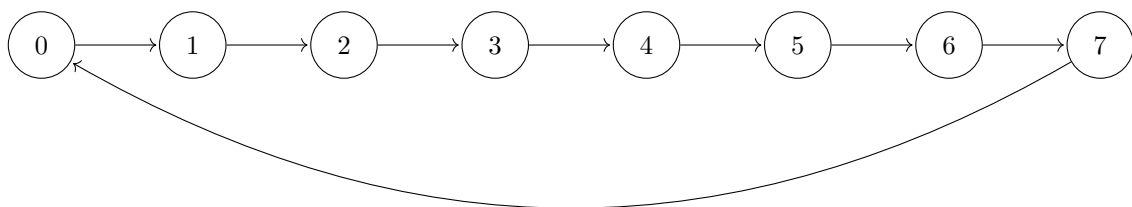
5.3 THREE-BIT ASYNCHRONOUS UP COUNTER

- It needs 3 flipflops and it is mod-8 up counter.
- Values are from 0 to 7. $[2^3-1]$



CLK	Q_A	Q_B	Q_C
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

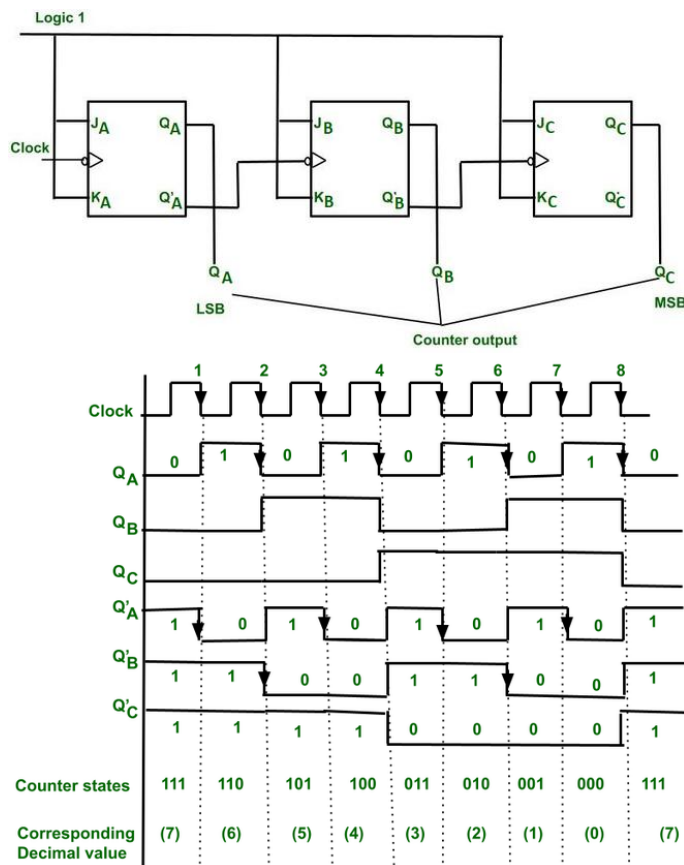
State Diagram of 3-bit up counter :





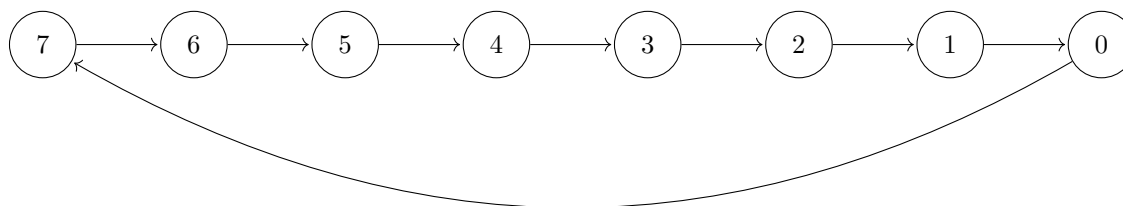
5.4 THREE-BIT ASYNCHRONOUS DOWN COUNTER

- It needs 3 flipflops and it is mod-8 down counter.
- Values are from 7 to 0. $[2^3-1]$



CLK	Q_A	Q_B	Q_C
0	1	1	1
1	1	1	0
2	1	0	1
3	1	0	0
4	0	1	1
5	0	1	0
6	0	0	1
7	0	0	0

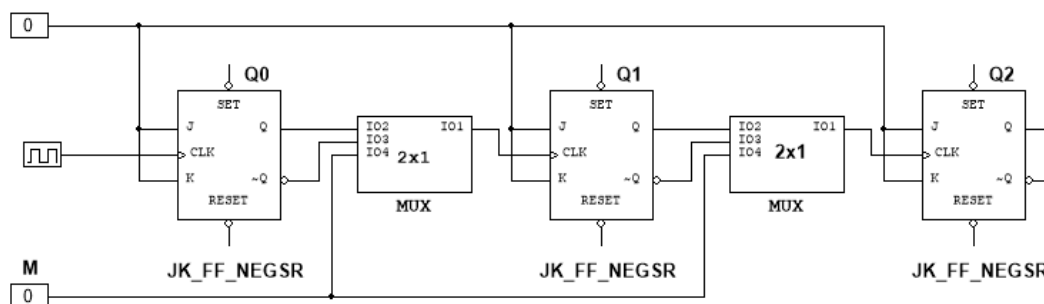
State Diagram of 3-bit up counter :





5.5 THREE-BIT ASYNCHRONOUS UP/DOWN COUNTER [-For Video](#)

- Up/Down counter is the combination of both the counters in which we can perform up or down counting by changing the Mode control input.
- It is used more than separate up or down counter.
- In this a mode control input (say M) is used for selecting up and down mode.
- A combinational circuit is required between each pair of flip-flop to decide whether to do up or do down counting.
- It needs 3 flipflops and it is mod-8 up-down counter.
- Values are from 7 to 0 or 0 to 7 [2^3-1]



CLK	Q ₂	Q ₁	Q ₀	Q' ₂	Q' ₁	Q' ₀
0	0	0	1	1	1	1
1	0	0	1	1	1	0
2	0	1	0	1	0	1
3	0	1	1	1	0	0
4	1	0	0	0	1	1
5	1	0	1	0	1	0
6	1	1	0	0	0	1
7	1	1	1	0	0	0

- When $M = 0$, then $Y = Q$, therefore it will perform Up counting (As discussed above).
- When $M = 1$, then $Y = Q'$ therefore it will perform Down counting (As discussed above).

1. Case 1 – When $M=0$, then $M' = 1$.

- Put this in $Y = M'Q + MQ' = Q$ So Q is acting as clock for next FFs.
- Therefore, the counter will act as Up counter.
- Explanation of Up counter –
 - (a) The 1st FF is connected to logic 1. Therefore, it will toggle for every falling edge.
 - (b) The 2nd FF input is connected to Q_0 . Therefore it changes its state when $Q_0 = 1$ and there is falling edge of clock.
 - (c) Similarly, 3rd FF is connected to Q_1 . Therefore, it changes its state when $Q_1 = 1$ and there is falling edge of clock.
 - (d) By this we can generate counting states of Up counter.
 - (e) After every 8th falling edge the counter is again reaching to state 0 0 0.
 - (f) Therefore, it is also known as divide by 8 circuit or mod 8 counter.



2. Case 2 – When $M=1$, then $M' = 0$.

- (a) Put this in $Y = M'Q + MQ' = Q'$. So Q' is acting as clock for next FFs.
- (b) Therefore, the counter will act as Down counter. Explanation of Down counter –
 - i. The 1st FF is connected to logic 1. Therefore, it will toggle for every falling edge.
 - ii. The 2nd FF input is connected to Q'_0 . Therefore it changes its state when $Q'_1 = 1$ and there is falling edge of clock.
 - iii. Similarly, 3rd FF is connected to Q'_1 . Therefore, it changes its state when $Q'_2 = 1$ and there is falling edge of clock.
 - iv. By this we can generate counting states of down counter.
 - v. After every 8th falling edge the counter is again reaching to state 111.
 - vi. Therefore, it is also known as divide by 8 circuit or mod 8 counter.

5.6 MOD-6 UP COUNTER

[-For Video](#)

- Mod-6 counter represents that the counter will have 6 states. Thus, $N = 6$.
- The number of flip-flops used for counter design is determined using the formula, $2^n \geq N$.
- The value of n is found to be 3. That is the number of flip-flops, $n = 3$. Hence the 6 counter states are 000, 001, 010, 011, 100, 101.
- Let us design the mod-6 asynchronous counter with JK flip-flops.
- The truth table for mod-6 asynchronous counter is as follows :

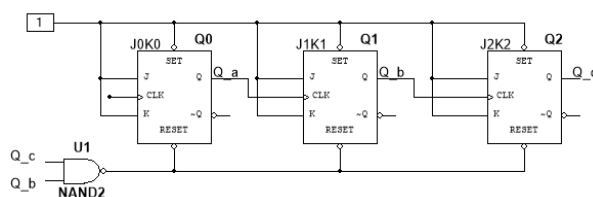
CLK	Q_C	Q_B	Q_A
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1

- The above truth table for Mod-6 counter is implemented in a Karnaugh map to get the reset logic function.

		$Q_C Q_A$			
		00	01	11	10
Q_C	0	1	1	1	1
	1	1	1	0	0

$Y = \overline{Q}_C + \overline{Q}_B$

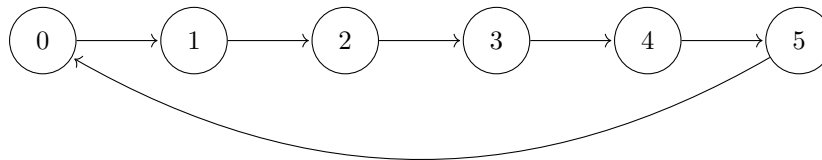
- The logic circuit diagram for mod-6 asynchronous/ripple counter is drawn as below.





- It is drawn with 3 flip-flops. Since it is an asynchronous counter, the clock pulse is given only to the first flip-flop. For the other flip-flops, the output of the previous flip-flop is given as the clock pulse for the next flip-flop.
- The expression obtained from the K-map is drawn using combinational circuits. The output Y is given as RESET for all the flip-flops.

State Diagram :



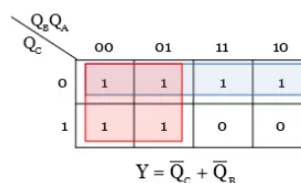
5.7 MOD-6 DOWN COUNTER

[-For Video](#)

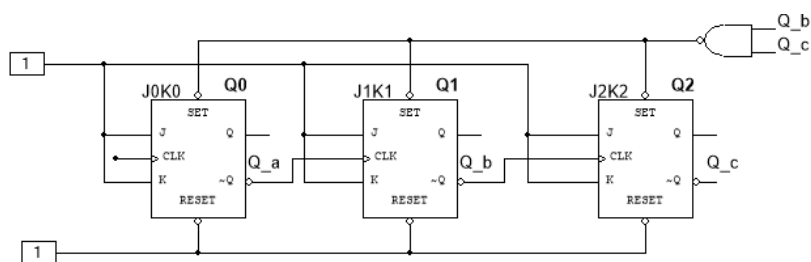
- Mod-6 counter represents that the counter will have 6 states. Thus, $N = 6$.
- The number of flip-flops used for counter design is determined using the formula, $2^n \geq N$.
- The value of n is found to be 3. That is the number of flip-flops, $n = 3$.
- Hence the 6 counter states are 111, 110, 101, 100, 011, 010.
- Let us design the mod-6 asynchronous counter with JK flip-flops.
- The truth table for mod-6 asynchronous counter is as follows :

CLK	Q_C	Q_B	Q_A
0	1	1	1
1	1	1	0
2	1	0	1
3	1	0	0
4	0	1	1
5	0	1	0

- The above truth table for Mod-6 counter is implemented in a Karnaugh map to get the reset logic function.

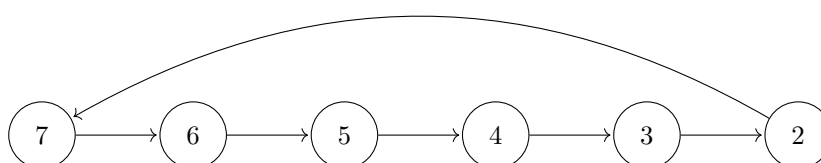


- The logic circuit diagram for mod-6 asynchronous/ripple counter is drawn as below.
- It is drawn with 3 flip-flops. Since it is an asynchronous counter, the clock pulse is given only to the first flip-flop. For the other flip-flops, the output of the previous flip-flop is given as the clock pulse for the next flip-flop.



- The expression obtained from the K-map is drawn using combinational circuits. The output Y is given as SET for all the flip-flops.

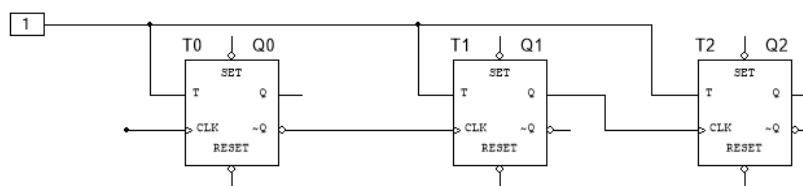
State Diagram :



5.8 RANDOM COUNTER

[-For Video](#)

- The number of flip-flops used for counter design is determined using the formula, $2^n \geq N$.
- The value of n is found to be 3. That is the number of flip-flops, $n = 3$.
- Let us design the random asynchronous counter with T flip-flops. The logic circuit diagram for random asynchronous/ripple counter is drawn as below.



- The truth table for random asynchronous counter is as follows :

CLK	Q_2	Q_1	Q_0	Q_{2N}	Q_{1N}	Q_{0N}
0	0	0	0	0	0	1
1	0	0	1	1	1	0
2	0	1	0	0	1	1
3	0	1	1	0	0	0
4	1	0	0	1	0	1
5	1	0	1	0	1	0
6	1	1	0	1	1	1
7	1	1	1	1	0	0

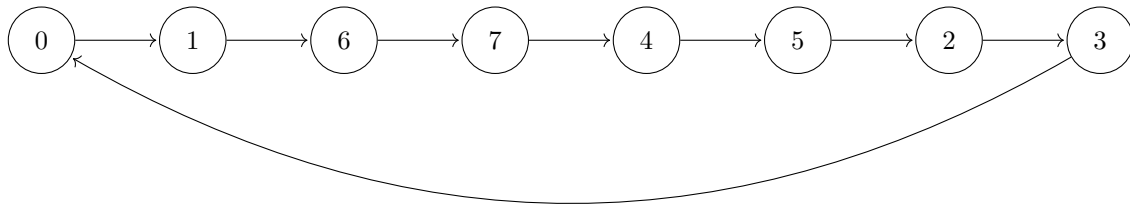
- The above truth table for random counter is obtained as follows :
 $Q_{0N} = \bar{Q}_0$



$$Q_{1N} = \bar{Q}_1 \text{ where } Q_0 : 0 \Rightarrow 1 \text{ (or) } Q_0 : 1 \Rightarrow 0$$

$$Q_{2N} = \bar{Q}_2 \text{ where } Q_1 : 0 \Rightarrow 1$$

State Diagram :



5.9 TWO-BIT SYNCHRONOUS UP COUNTER

[-For Video](#)

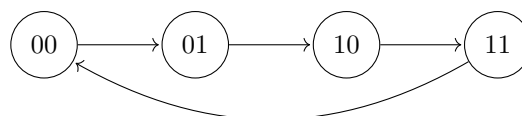
Using T-FlipFlop :

- These are the following steps to design the 2-bit synchronous up counter using T-FlipFlop.
Step-1 : To design a synchronous up counter first we need how many flipflops are required. Here we require 2 T-flipflops.

Step-2 : Draw the excitation table for the T-flipflop.

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

Step-3 : Draw the state diagram for 2-bit synchronous up counter as follows.

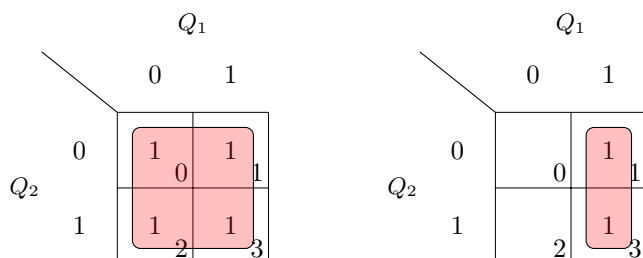


Step-4 : Now we need to construct a truth table using excitation table.

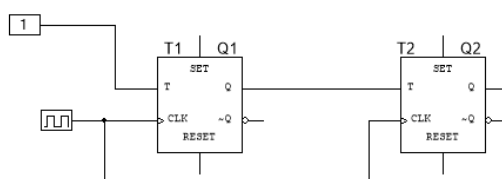
Q_2	Q_1	Q_{2N}	Q_{1N}	T_2	T_1
0	0	0	1	0	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	0	0	1	1



Step-5 : After making the excitation table next we need to obtain k-map for the design of the counter. So for T_1 and T_2 we got 1 and Q_1 respectively.



Step-6 : Lastly according to the equation got from K-map create the design for 2 bit synchronous up counter.



Using JK-FlipFlop :

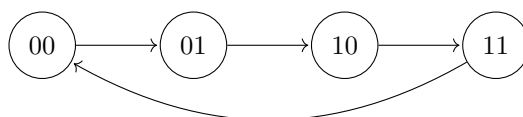
- These are the following steps to design the 2-bit synchronous up counter using JK-FlipFlop.

Step-1 : To design a synchronous up counter first we need how many flipflops are required. Here we require 2 JK-flipflops.

Step-2 : Draw the excitation table for the JK-flipflop.

Q_n	Q_{n+1}	J	K
0	0	0	ϕ
0	1	1	ϕ
1	0	ϕ	1
1	1	ϕ	0

Step-3 : Draw the state diagram for 2-bit synchronous up counter as follows.

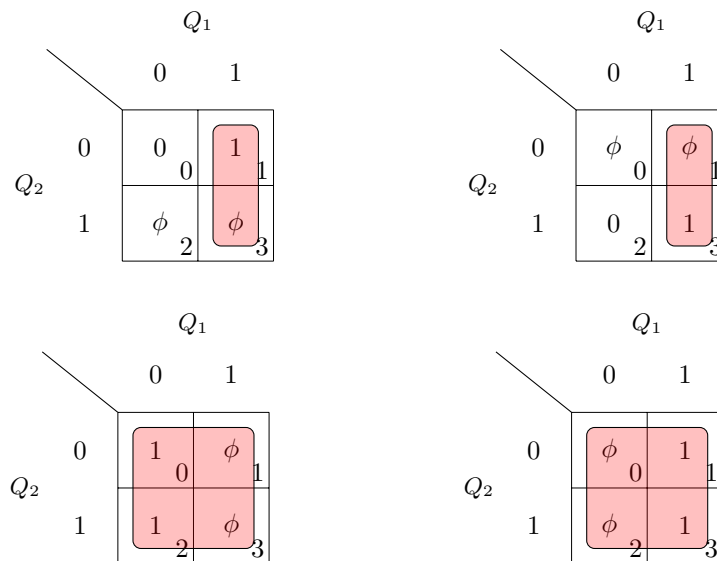


Step-4 : Now we need to construct a state table using excitation table.

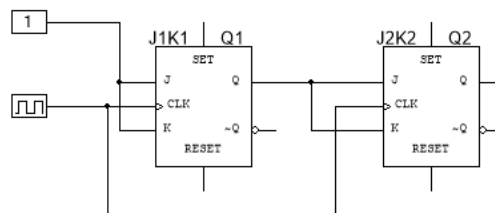
Q_2	Q_1	Q_{2N}	Q_{1N}	J_2	K_2	J_1	K_1
0	0	0	1	0	ϕ	1	ϕ
0	1	1	0	1	ϕ	ϕ	1
1	0	1	1	ϕ	0	1	ϕ
1	1	0	0	ϕ	1	ϕ	1



Step-5 : After making the excitation table next we need to obtain k-map for the design of the counter.



Step-6 : Lastly according to the equation got from K-map create the design for 2 bit synchronous up counter.



5.10 TWO-BIT SYNCHRONOUS DOWN COUNTER [-For Video](#)

Using T-FlipFlop :

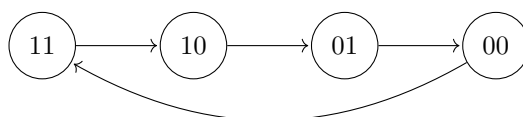
- These are the following steps to design the 2-bit synchronous down counter using T-FlipFlop.

Step-1 : To design a synchronous down counter first we need how many flipflops are required. Here we require 2 T-flipflops.

Step-2 : Draw the excitation table for the T-flipflop.

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

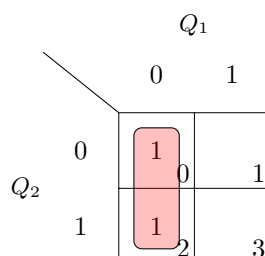
Step-3 : Draw the state diagram for 2-bit synchronous down counter as follows.



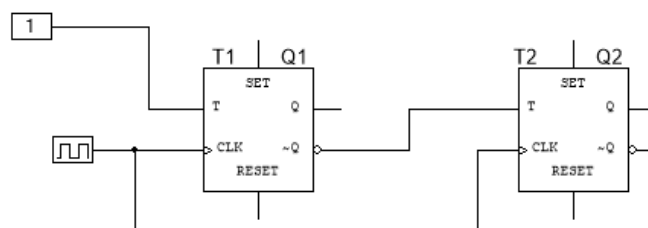
Step-4 : Now we need to construct a state table using excitation table.

Q_2	Q_1	Q_{2N}	Q_{1N}	T_2	T_1
0	0	1	1	1	1
0	1	0	0	0	1
1	0	0	1	1	1
1	1	1	0	0	1

Step-5 : After making the excitation table next we need to obtain k-map for the design of the counter. So for T_1 and T_2 we got 1 and Q'_1 respectively.



Step-6 : Lastly according to the equation got from K-map create the design for 2 bit synchronous down counter.



5.11 TWO-BIT SYNCHRONOUS UP-DOWN COUNTER [-For Video](#)

Using T-FlipFlop :

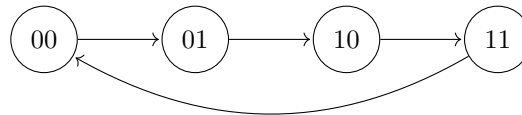
- These are the following steps to design the 2-bit synchronous Up-down counter using T-FlipFlop.
- Step-1 :** To design a synchronous up counter first we need how many flipflops are required. Here we require 2 T-flipflops.



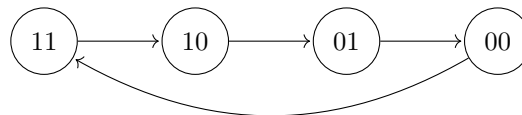
Step-2 : Draw the excitation table for the T-flipflop.

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

Step-3 : Draw the state diagram for 2-bit synchronous up counter as follows.



Step-4 : Draw the state diagram for 2-bit synchronous down counter as follows.



Step-5 : Now we need to construct a state table using excitation table.

M	Q_2	Q_1	Q_{2N}	Q_{1N}	T_2	T_1
0	0	0	0	1	0	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	0	0	1	1
1	0	0	1	1	1	1
1	0	1	0	0	0	1
1	1	0	0	1	1	1
1	1	1	1	0	0	1

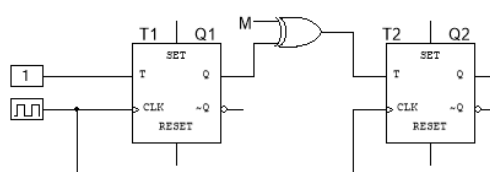
Step-5 : After making the excitation table next we need to obtain k-map for the design of the counter. So for T_1 and T_2 we got 1 and Q_1 respectively.

		Q_2Q_1			
		00	01	11	10
M	0	0	1	1	2
	1	1			1



		Q_2Q_1			
		00	01	11	10
M	0	1 0	1 1	1 3	1 2
	1	1 4	1 5	1 7	1 6

Step-6 : Lastly according to the equation got from K-map create the design for 2 bit synchronous up counter.



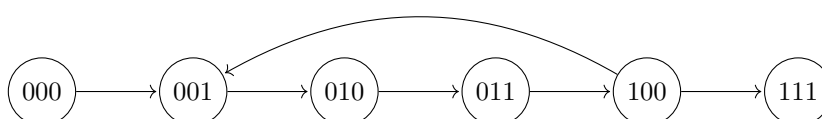
5.12 SELF-STARTING COUNTER

[-For Video](#)

- A counter is said to be self-starting if it is possible to enter a counter loop irrespective of the initial state.
- It is known that all counters generate a sort of sequence of numbers (with each flip-flop representing one bit in a number).
- Now if the longest loop in the sequence (or the main loop) can be traversed from any state, only then is the counter said to be self-starting.
- Self-starting counters are made so that “trap” states can be avoided. A trap state is a state that is accessed due to some error in the operation of the counter.
- Self-starting counters are made with certain modifications in the circuit so that if the counter goes into a trap state, it can automatically move out of it and come back into the main counting loop.

Examples :

1. In this example, the subsequence 001->010->011->100 forms the main counting loop. However, it can be seen that the main counting loop can be reached irrespective of the state we choose to start in. Hence this counter is self-starting.





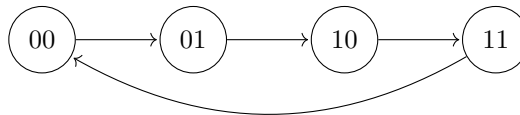
5.13 FREE RUNNING COUNTER

[-For Video](#)

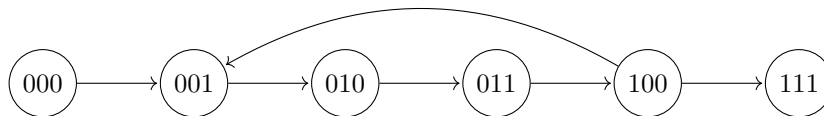
- A counter is said to be free-running if it contains all possible states in the counter loop.
- The longest loop in the sequence of numbers generated by the counter is said to be the main counting loop.
- If the main counting loop extends and covers all the states, only then is the counter said to be free-running. In other words, the main counting loop should cover the entire sequence.
- Free-running counters are counters which operate without needing any external interference at any point in time.
- If at least one point lies outside the main counting loop then some sort of extra effort is needed to force it to come back to the main loop.

Examples :

1. In this example, the main counting loop is the entire sequence – it covers all the states in the sequence. Hence it is a free-running counter.



2. This counter is not free-running.



Note : Every free running is self starting but not vice versa.

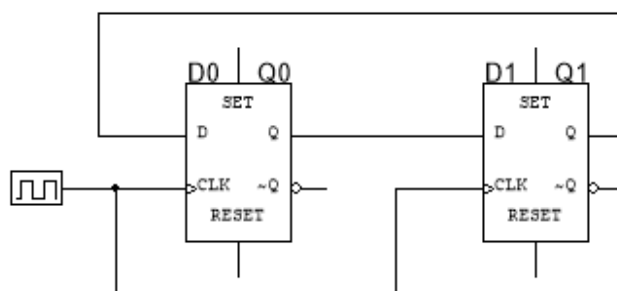
5.14 SHIFT COUNTERS

[-For Video](#)

- A synchronous counter that consists of clocked flip-flops arranged as a shift register.
- Data is propagated from left to right (or from right to left) between the flip-flops by the application of a clock or count pulse.
- These are two types :
 1. Ring Counters
 2. Johnson Counters

1. Ring Counter [Mod-2] :

- The ring counter is one type of shift counter. Here the output of the last flip-flop is connected to the input of the first flip-flop in the case of the ring counter but in the case of the shift register it is taken as output.
- **No. of states in Ring counter = No. of flip-flop used**
- The diagram for Mod-2 Ring Counter is as follows :

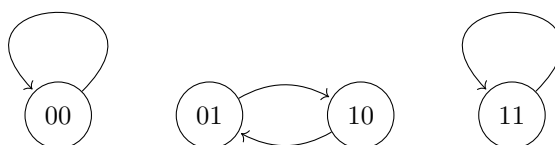


- State table for D-flipflop :

Q_1	Q_0	Q_{1N}	Q_{0N}
0	0	0	0
0	1	1	0
1	0	0	1
1	1	1	1

Here $Q_{1N}=D_1=Q_0$ and $Q_{0N}=D_0=Q_1$

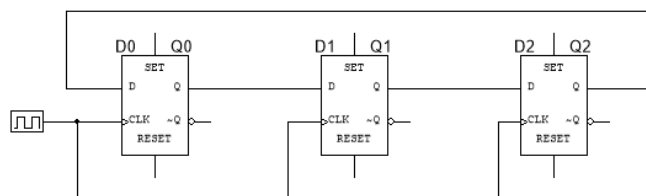
- State diagram for Mod-2 Ring counter is as follows :



Ring Counter [Mod-3] :

[-For Video](#)

- The diagram for Mod-3 Ring Counter is as follows :



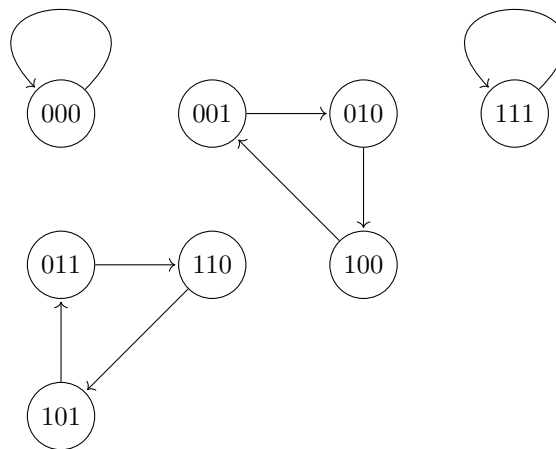
- State table for D-flipflop :

Q_2	Q_1	Q_0	Q_{2N}	Q_{1N}	Q_{0N}
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	0	1	1
1	1	0	1	0	1
1	1	1	1	1	1

Here $Q_{2N}=D_2=Q_1$, $Q_{1N}=D_1=Q_0$ and $Q_{0N}=D_0=Q_2$



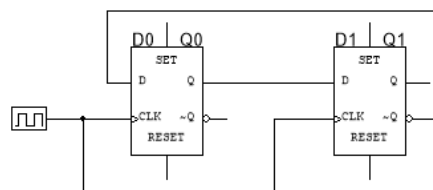
- State diagram for Mod-3 Ring counter is as follows :



2. Johnson Counter [2-bit] :

[-For Video](#)

- The Johnson counter is similar to the Ring counter. The only difference between the Johnson counter and the ring counter is that the outcome of the last flip flop is passed to the first flip flop as an input.
- But in Johnson counter, the inverted outcome Q' of the last flip flop is passed as an input. The remaining work of the Johnson counter is the same as a ring counter.
- The Johnson counter is also referred to as the Creeping counter.

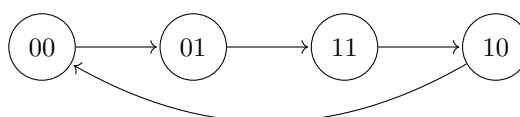


- State table for D-flipflop :

Q_1	Q_0	Q_{1N}	Q_{0N}
0	0	0	1
0	1	1	1
1	0	0	0
1	1	1	0

Here $Q_{1N}=D_1=Q_0$ and $Q_{0N}=D_0=Q_1'$

- State diagram is as follows :

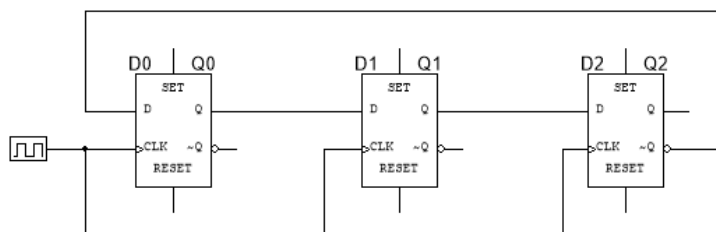




Johnson Counter [Mod-6] :

[-For Video](#)

- The diagram for Mod-6 Johnson Counter is as follows :

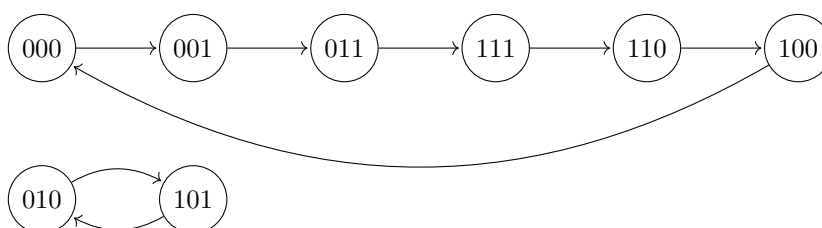


- State table for D-flipflop :

Q_2	Q_1	Q_0	Q_{2N}	Q_{1N}	Q_{0N}
0	0	0	0	0	1
0	0	1	0	1	1
0	1	0	1	0	1
0	1	1	1	1	1
1	0	0	0	0	0
1	0	1	0	1	0
1	1	0	1	0	0
1	1	1	1	1	0

Here $Q_{2N}=D_2=Q_1$, $Q_{1N}=D_1=Q_0$ and $Q_{0N}=D_0=Q'_2$

- State diagram for Mod-3 Ring counter is as follows :



****LEARNING NEVER ENDS****